

The Ledger

Specification, Version 15.0

R. Delloye

July 2026

Contents

1	The Objective and the Commitments	5
1.1	One record, or reconciliation	5
1.2	The six commitments	5
1.3	The discipline of the document	6
2	The Picture: Map, Then Fold	7
2.1	The six threads	7
2.2	The world presents events	7
2.3	The map: each event becomes a transaction	8
2.4	The fold: the transactions become the ledger	8
2.5	Data, functions, and machines	8
2.6	The clock, and proposal versus authority	8
2.7	One instrument walks the pipeline	9
2.8	Not event sourcing	9
3	The Objects	10
3.1	Units	10
3.2	Wallets	11
3.3	The Atomic Move	12
3.4	The Transaction	12
3.5	The Immutable Log	13
3.6	Closure by Virtual Wallets	13
3.7	The Coordinate Vector	13
3.8	Term Sheets as Graphs	13
4	The Machines	15
4.1	The Event Monitor	15
4.2	The Events Executor	16
4.3	The Transaction Executor	16
4.4	One event, three machines, one write	17
4.5	The watch handshake	17
4.6	The trust boundary	18
5	Smart Contracts	18
5.1	The dividend: three firings, one contract	19
5.2	The trigger classes differ in what they read, never how they run	20
5.3	The future: settlement prints and the expiry unwind	20
5.4	The one-touch: one firing, one transaction, and idempotence	21

5.5	The variance swap: a contract never remembers	21
5.6	Faithfulness: every right and obligation is a unit	22
6	State: The Three Homes	22
6.1	The three homes	22
6.2	One fact, one home: the future exercises all three	23
6.3	The homes suffice	23
6.4	Two more facts finding their homes	24
6.5	Rebuildable by replay	25
7	Valuation and Profit and Loss	25
7.1	Net Asset Value	25
7.2	Profit and loss is the variation in NAV	26
7.3	Pricing is parameterised by state read from the ledger	27
7.4	Deposits do not move profit and loss	27
7.5	Mid-life valuation is read from the record	28
7.6	Dual valuation: two price vectors over one position set	29
7.7	The pricing-stack requirements	29
8	Ingestion, Corporate Actions, and the Market Data Operator	30
8.1	The boundary, and corporate actions on it	30
8.2	The observation door	30
8.3	The datum-kind registry	31
8.4	The market data operator and adjustment-schedule totality	32
8.5	The operator menu: the split	33
8.6	State-aware reads: consistency of reference and cum/ex	33
8.7	Recompose and the aggregate weld	34
8.8	Failure regimes at the boundary	34
8.9	The boundary-convention slot	35
8.10	What an auditor checks	36
9	Collateral, Margin, and Lending	36
9.1	One representation, keyed by declared regime	36
9.2	The two guards	37
9.3	Title transfer: owned re-books; the claim is a unit	38
9.4	Pledge: marker mass under the coverage bound	39
9.5	Cash margin: four cases, one machine	39
9.6	Line valuation	40
9.7	Entitlements: determination and payment	40
9.8	The knock while pledged	41
9.9	The lent plane: securities lending	42
9.10	Retirement, and the supervised write-off	43
10	Virtual Ledgers, Strategies, and the Total Return Swap	43
10.1	The strategy as a smart contract	44
10.2	Two disciplines	44
10.3	Real execution: orders, benchmark, and slippage	45
10.4	The total return swap	45
11	The Settlement Interface	46
11.1	Settlement as a projection	46
11.2	Four episodes	47

11.3	Settlement state and the market claim	48
12	Presentation and Reporting Projections	50
12.1	The projection principle	51
12.2	The presentation projection	51
12.3	The attribution convention is total	52
12.4	The reconciled surface	53
12.5	Episode: the pledged one-touch in the encumbrance projection	53
12.6	Verification and the audit gate	54
13	Alignment with the Common Domain Model	54
13.1	Unit maps to product; a bespoke instrument maps to the trade	54
13.2	Business event maps to transaction	55
13.3	Eligibility as declared data restates the CDM collateral model	56
13.4	Closed enumerations and the product graph	56
13.5	Gaps	56
14	The Invariant Catalogue	57
14.1	Conservation per (unit, coordinate)	57
14.2	The log's structural invariants	58
14.3	Consistency of reference	59
14.4	Writer discipline	60
14.5	Time travel and replay	60
14.6	The coverage invariant and the two-guard contrast	60
14.7	The post-and-return metamorphic property	62
14.8	What the catalogue buys	62
15	Testability and the Executable-Check Regime	62
15.1	Two layers of correctness	62
15.2	The generator universe and the adversarial scheduler	63
15.3	The test surface: the invariant catalogue and the product graph	64
15.4	The three binding conditions as executable gates	64
15.5	What an auditor checks	67
16	Minimum Requirements	67
16.1	The Event Monitor and the Events Executor	67
16.2	Valuation semantics	68
16.3	The data boundary	69
16.4	Verification	70
17	Scope	70
17.1	In scope	71
17.2	Out of scope	71
17.3	The open-problems index and the constitutional-parking rule	72
17.4	Authority	73

Abstract

Reconciliation exists because many systems store the same facts, and independently maintained copies eventually disagree. The Ledger removes the cause: every fact is recorded exactly once, in an immutable log of atomic transactions, and every other number — balances, valuations, profit and loss, reports — is a deterministic projection of that single record, reproducible by any party and reconstructible at any past date. This specification defines the system in full, rated one-way against the Ledger Framework Constitution v1.1: the objects the system manipulates and the signed coordinate vector every balance generalises to; the three machines that evaluate the pipeline — the Event Monitor, the Events Executor, and the Transaction Executor, three roles and one writer; smart contracts as pure, stateless transcriptions of legal terms; the three homes of state; valuation and profit and loss; ingestion, corporate actions, and the market data operator; collateral, margin, and lending under the declared legal regime of the governing agreement; virtual ledgers and the total return swap; the settlement interface; presentation and reporting projections; alignment with the Common Domain Model; and the invariant catalogue and executable-check regime that make every guarantee a property a machine can run. Six instruments with fixed terms and fixed quantities recur through every chapter, so that each claim is exercised on the same worked facts it governs.

Contents

1 The Objective and the Commitments

This chapter discharges §1 and §2 of the constitution.

1.1 One record, or reconciliation

The objective of the Ledger is a clean post-trade and risk-management platform: one system of record from which positions, valuations, profit and loss, collateral, and reports are all derived — exactly, deterministically, and reproducibly.

The obstacle is the architecture of the infrastructure the industry runs today. Post-trade processing is not one system but many — trading systems, risk engines, general ledgers, custodians, regulatory engines — and each holds its own copy of positions, cash balances, corporate-action adjustments, and lifecycle states, updated on its own schedule by its own logic. The copies diverge, and reconciliation — detecting and repairing the divergences — is one of the largest operational costs and risks in the industry. The root cause is architectural: **the same fact is stored more than once**. Two stores of one fact maintained by independent logic will eventually disagree, and no amount of process removes a failure mode the architecture itself provides.

The Ledger removes the failure mode at its root. There is exactly one canonical record — an ordered, append-only stream of atomic transactions — and every other number is computed on demand as a projection of that stream: balances, valuations, profit-and-loss statements, balance sheets, regulatory reports. Two projections of one record cannot disagree about any fact the record contains. That sentence is the design’s entire ambition; the rest of this specification is what it forces. Its scope — the recording, valuation, and lifecycle of trading-book positions held at fair value, and the projection of committed activity into settlement — is fixed component by component in Chapter 17, together with the external authorities the ledger reconciles against at its boundary and never replaces.

1.2 The six commitments

Six commitments are non-negotiable, and every design decision in this specification is tested against them. Each is stated here with the one consequence it forces on the rest of the document.

Full auditability. Every number is traceable, mechanically and completely, to the recorded events that produced it. History is never rewritten: a mistake is repaired by a recorded correction that names what it supersedes, so the error and its repair both survive. This commitment forces the immutable, hash-chained log of Chapter 2 and Chapter 3, and the repair discipline of Chapter 15: a wrong booking is corrected by a compensating transaction through the same admission door, in the open, never by amendment of the past.

Full reproducibility. Given the same record, any party — the firm, a counterparty, an auditor, a regulator — recomputes the same state, the same views, the same contract outputs, to the last minor unit. This commitment forces every arrow of Chapter 2 to be a deterministic function of recorded inputs and every quantity in this document to be an exact integer in minor units; the single input not on the record, the clock, is confined to one machine, whose emissions are themselves recorded (Chapter 4).

Time travel embedded in the design. The exact state of the system at any past moment is reconstructible from the record alone, at any time, forever — independently of whether the software that produced the events still exists. This commitment forces replay to re-read admitted transactions and never to re-execute smart contracts; Chapter 14 states time travel as a theorem of the fold, not a feature bolted onto it.

Correctness. Illegal states are made unrepresentable wherever possible: a check can be forgotten or bypassed; a type cannot. Where wrongness cannot be prevented — a contract can always compute the wrong economics — it is made mechanically detectable and deterministically

repairable. Defaults fail closed: the system stops rather than improvises. This commitment forces the single admission door of Chapter 4, the typed lifecycle graphs of Chapter 3, and the two layers of correctness of Chapter 15 — structural correctness guaranteed at admission, economic correctness made checkable by recomputation.

Testability. The ledger is correct because it is built to be tested. Every step is a pure, versioned function with explicit inputs, explicit outputs, and strong types — typing alone eliminates whole classes of defects before a single test runs — and every core invariant is expressed as an executable check over generated products, events, and histories, never defended only in prose. Correctness then composes: because the steps are pure and their composition is fixed — the map, then the fold — verifying each part verifies the whole. This commitment forces the invariant catalogue of Chapter 14 and the executable-check regime of Chapter 15, and it forces a rule of method: a property that cannot be tested mechanically is redesigned until it can be.

Clarity. One name per component and one component per name; behaviour carried as declared data rather than hidden in code; every claim stated so that it can be checked by a reader with no prior exposure. This commitment fixes the vocabulary of this document — unit, wallet, balance, move, transaction, watch, the immutable log, projection, the Event Monitor, the Events Executor, the Transaction Executor, smart contract, the market data operator, the three homes, virtual wallet, virtual ledger — and forces the declared-data discipline that recurs in every chapter: terms, watches, market data operators, and collateral eligibility are data the record carries, never behaviour a reader must infer from code.

The commitments are ordered by what arbitrates a conflict: a design that is auditable and reproducible but untestable is redesigned; a design that is elegant but cannot be replayed is rejected. No design decision in this document rests on custom or typical practice; each is derived from the commitments and then, where an external standard binds, shown to satisfy it. One boundary bounds all six: they secure one consistent internal record, not its correspondence with the world outside — internal unbreakability is not external correctness, and whether a projection matches the market, the custodian, or the counterparty is reconciled at the boundary (Chapter 8), never assumed.

1.3 The discipline of the document

The direction of authority is one-way. This specification is rated against the Ledger Framework Constitution v1.1 and against nothing else; each chapter opens by naming the constitution sections it discharges, and a genuine conflict with the constitution is never drafted around — it is recorded in Chapter 17’s open-problems index with the exact amendment it would require.

Six instruments run through the document as threads, introduced as a cast in Chapter 2: a cash-settled future, a one-touch option, a cash dividend, a stock split, an OTC variance swap, and a total return swap. Their terms, parties, dates, and quantities are fixed once and never vary, so a number met in one chapter can be checked against the same number in another. Each visit is an *episode* in a fixed five-part shape: the trigger; the contract that catches it; the transactions it produces, as numbered rows with exact quantities; what the door checks; and the design point it substantiates. All quantities are exact integers in minor units; amounts stated in whole currency are exact in cents.

Two conventions govern notation. Category-theoretic language appears only as a *second telling*, and no knowledge of category theory is assumed of the reader: every concept is first fully explained in plain terms with a concrete example, and a marked box may then restate it categorically to remove residual ambiguity for the reader who wants it — nothing later depends on any such box, and deleting them all leaves the specification complete. Code appears only as short typed signatures and data declarations; the reference implementation lies outside this document entirely.

2 The Picture: Map, Then Fold

This chapter discharges §3 of the constitution.

Everything this framework does is a *map* followed by a *fold* — two words borrowed from programming, worth a line each. A *map* processes a list element by element: given a way to turn an **a** into a **b**, it carries $[a]$ to $[b]$, one element at a time. A *fold* accumulates a list into a single result: each element is absorbed into a running value, $[a]$ to **b**. The whole architecture is these two steps over one list. The world presents a list of events, $E = [e]$; a map carries it to a list of transactions, $T = [tx]$; a fold reduces that list to the ledger. Four terms complete the picture, each defined in Chapter 3 and used here only at a gloss: a **unit** is anything that can be held or owed; a **wallet** is where units are held; a **transaction** is the atomic bundle of changes the ledger records; and a **projection** is any view computed from the record. Every number the ledger reports — a balance, a valuation, a profit and loss, a report — is a projection: a further fold of that same one list.

2.1 The six threads

Six instruments recur through this specification as threads. Their terms, parties, and quantities are fixed here once and never vary, so a number met in one chapter is the same number met in another. This chapter follows one of them — the one-touch OT-1 — through the whole pipeline; the rest are worked where they bite.

	Instrument, kind, parties	Fixed terms and headline numbers
T1	FUT-IDX — cash-settled future on IDX; W-ALPHA / the CCP	multiplier 10, USD, settled-to-market; buy 1 at 995.00; settlement prints 1,000.00 / 990.00 / 1,005.00; cumulative VM USD 100 = PnL
T2	OT-1 — one-touch on stock OMEGA; W-BANK writes, W-ALPHA holds	barrier breached when OMEGA's official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch; payout USD 1,000,000; spot 95.00 at inception; knock at close 79.00
T3	ACME — cash dividend; issuer / W-ALPHA	1.50 per share; W-ALPHA holds 10,000 shares; entitlement USD 15,000
T4	ACME — 2-for-1 split; issuer / W-ALPHA	10,000 → 20,000 shares; price frame 100.00 → 50.00; declared dividend figure 1.50 → 0.75
T5	VS-1 — OTC variance swap on IDX; bilateral	one year, 252 fixings, strike 400 variance points, USD 1,000 per point; realised variance 441; settlement USD 41,000
T6	TRS-1 — total return swap; reference W-QIS in virtual ledger V-1 (strategy S-QIS)	notional USD 10,000,000; financing 4% p.a., quarterly; first reset NAV 10,300,000; net move USD 200,000

2.2 The world presents events

The world presents the system with a list of events. Some arrive from outside: trades executed in the market, corporate-action notices, fixings and other observations, captured and recorded at the boundary. Others are not received but noticed. Every unit declares, from its terms, the conditions its life requires watching — its **watches** (Chapter 3) — and the Event Monitor evaluates those watches against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. The future FUT-IDX declares such watches at registration: one for each daily settlement print, one for its expiry; the one-touch OT-1 declares one, its underlying's recorded close against the barrier. A coupon date arriving, an expiry falling due, a close printing below a barrier — each becomes an event the same way a trade does, and once emitted the two kinds are indistinguishable: both are on the record.

2.3 The map: each event becomes a transaction

Each event is mapped to a transaction, and the responsibility for the map divides in two. The **smart contracts** are the mapping functions: each is a stateless piece of code that, given one event and the current state of the ledger, computes the transaction the event requires (Chapter 5). The **Events Executor** applies the map over the list, $E = [e] \rightarrow T = [tx]$: for each event it fires the responsible contract, hands it the event together with the current projection of the ledger, and collects the proposed transaction. Many contracts, one Events Executor; the contracts carry all the product knowledge, the Events Executor carries none (Chapter 4).

The map is not blind to state. To turn a dividend event into cash moves, the contract must read who holds the stock at that moment — the fold of everything admitted so far. Map and fold therefore advance together, one event at a time, in a single total order: the transaction for event e is computed against the ledger that the transactions for all earlier events have already built.

2.4 The fold: the transactions become the ledger

The resulting list of transactions is folded — reduced, one transaction at a time — into the state of the system: balances, and the three homes of unit and position state (Chapter 6, *State: The Three Homes*). The **Transaction Executor** performs the fold, $T = [tx] \rightarrow \text{Ledger}$: it serialises, validates, and applies each transaction, and the immutable log is precisely the list being folded. Nothing else writes. And because the log is one list, every other view is a further fold of it: a balance sums the moves that touch a wallet, a valuation folds the composition against a price vector (Chapter 7), a report folds it into a presentation (Chapter 12). One record, and many folds of it — never many records.

2.5 Data, functions, and machines

Three kinds of thing appear in this picture, and they are never confused: data, functions, and machines. The data are Event, Transaction, and Ledger. The functions are the arrows between the data. The machines are what evaluate the arrows — one machine per arrow, and they are the subject of Chapter 4. Typed:

```
watch    : (Record, Clock)      -> [Event]      -- the Event Monitor
contract : (Event, Ledger)      -> Transaction  -- the Events Executor
apply    : (Transaction, Ledger) -> Ledger       -- the Transaction Executor

step (ledger, e) = apply (contract (e, ledger), ledger) -- map, then fold
final ledger     = fold step over E = [e]
```

The three arrows differ in what they read. **watch** reads the Record and the clock and yields events; **contract** and **apply** consume the Ledger — the folded state — and never the clock. One **step** is a map (fire the contract) immediately followed by a fold (apply the result), and the whole run is that step folded over $E = [e]$. Map, then fold: the phrase is the architecture, not a metaphor for it.

2.6 The clock, and proposal versus authority

Two properties of the picture carry most of its weight.

The clock is the single input that is not on the record, and the Event Monitor is the single place it is consulted. The event the Monitor emits is itself recorded, so everything downstream of the emission is a function of the record alone — which is exactly why a late firing produces the identical transaction, and why, given the record and the clock, any party can recompute which events should have been emitted, and when (Chapter 4).

The map is proposed; the fold is authoritative. A contract’s output is a proposal, not a fact: a transaction joins the list only when the Transaction Executor admits it. The smart contracts, the Event Monitor, and the Events Executor sit outside the trust boundary and may only propose; the one Transaction Executor writes. Either arrow may refuse — a contract may find nothing due; the door rejects a malformed transaction and writes nothing — and a refusal is a returned value, never a half-applied state. Because both the events and the transactions are on the record, economic correctness is checkable after the fact: re-run the map on the recorded event and compare the result against the recorded transaction (Chapter 15, *Testability and the Executable-Check Regime*). Verification lives with the events; authority lives with the transactions.

2.7 One instrument walks the pipeline

The one-touch OT-1 pays USD 1,000,000 when OMEGA’s official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch; its spot at inception is 95.00. It is written by W-BANK and held by W-ALPHA, and at registration its terms declared one watch: OMEGA’s recorded official closes against the barrier.

Trigger (watch). OMEGA’s official close of 79.00 is recorded. The declared condition — at or below 80.00 — is met, and the Event Monitor emits the touch event onto the record. No ledger state has changed: the Monitor emits events, and an event is an input to the map, not a write.

Contract (map). The Events Executor fires OT-1’s contract, handing it the touch event and the current projection of the ledger. The contract reads the terms (payout USD 1,000,000), the holder (W-ALPHA), and the unit’s state (*live*), and returns one transaction:

	Kind	Content
1	unit-state change	OT-1: UnitStatus <i>live</i> → <i>triggered</i>
2	move	owned(USD) 1,000,000: W-BANK → W-ALPHA

Apply (fold). The Transaction Executor admits it — the move’s paired legs leave conservation undisturbed, the cause-derived identifier names the touch event, and the writer is OT-1’s own contract — and the fold advances: the ledger now shows the cash with W-ALPHA and the option *triggered*. Where OT-1 stands encumbered under a collateral agreement the payment leg routes differently, and Chapter 9 works that same knock start to finish; the pipeline is the one drawn here either way.

One list of events, one map, one fold: the option’s whole knock is a single event carried across the three arrows, and nothing off the record ever touches it.

2.8 Not event sourcing

The resemblance to event sourcing is real, and the difference matters. The Ledger is event-driven because finance is: trades execute, prices print, coupons mature, and barriers are touched whether or not the software cares, so events are recorded because reality presents them, not because a pattern was chosen. The difference is where authority sits. In event sourcing the event stream is the canonical history, state is whatever the current code derives from it, and reinterpreting history through new code is a feature. Here events are inputs to the map, never the accounting record: contracts transform them into proposed transactions, the Transaction Executor admits or refuses — an admission door event sourcing has no equivalent of — and the admitted transaction log alone is the canonical history. Replay re-reads transactions and never re-executes contracts (Chapter 4), so history cannot be silently reinterpreted; a wrong interpretation is repaired by a compensating transaction, in the open (Chapter 15). And every view is a projection of that one log, not a fold over per-aggregate streams stitched back together:

there is a single total order, and two projections of it cannot disagree. The events stay on the record for one purpose — so the map can be re-run and verified.

A second telling, in categorical language; nothing later depends on it.

A fold with a fixed step is a *catamorphism*: the unique map out of the list $E = [e]$ determined by one rule for the empty list and one rule for prepending an element — here **step**. Because the step is fixed and the list carries its own order, the ledger is that unique image, and every projection — balance, valuation, report — is another catamorphism over the same list. “Map, then fold” names the composite; that the fold is a catamorphism is why one record admits many views and no two of them can disagree.

3 The Objects

This chapter discharges §4 of the constitution as amended, including Amendment 2 (the mass-never-value sentence) and Amendment 3 (the signed coordinate basis).

At the highest level a ledger is three things and a record: units — what can be held; wallets — where it is held; balances — how much of each unit each wallet holds; and the immutable log that records every change to them. Each object is defined here once; nothing later in the specification introduces another. The chapter ends where a unit’s terms begin: with the product graph, the declared structure from which a unit’s watches, its state schema, and its contract’s actions are all derivable.

3.1 Units

A **unit** is anything that can be held or owed inside the ledger: a currency (USD in cents), an equity security identified by its ISIN, a bond, a listed or OTC derivative, a securities loan, a managed-account mandate, a margin obligation under a collateral agreement, or any other contractual instrument. Units are first-class and extensible: a new instrument type is a new element of the universe of units, never a modification of the machinery. Because every unit — asset, liability, or contractual obligation — is represented uniformly, one move machinery and one conservation discipline cover them all, without special cases.

Units divide into **valued** and **non-valued**. A unit is valued exactly when it represents an economic claim whose value exists independently of the governance or continued operation of the instrument that created it: a stock, a bond, a client’s redemption claim on a managed account, a total return swap. A unit is non-valued when it is the governance itself: a mandate’s rulebook, a portfolio-swap reset mechanism, a quantitative strategy’s governance shell. The price of a non-valued unit is undefined — not zero — because pricing the governance structure beside the claim it governs would count the same economic reality twice. Valuation (Chapter 7) sums over the valued units only.

Every unit carries three things: its **terms** — the contractual text, versioned and append-only; its **state** — where it stands in its lifecycle; and its **smart contracts** — the executable form of those terms. From the terms follow its **watches**: the dates, observations, and conditions the Event Monitor must observe on its behalf. Declaring them is the unit’s duty — at registration, and again whenever a later transaction creates a new obligation — and the declaration is exhaustive: the Monitor does exactly what the units have asked of it, nothing more and nothing less. Section 3.8 shows that this duty is not a discipline to remember but a consequence of the form the terms take.

Episode — the future (T1): terms declare the watches. *Trigger.* The exchange lists FUT-IDX, and on day 0 W-ALPHA’s purchase of one contract at 995.00 executes; both facts are captured and recorded at the boundary. *Contract.* Each recorded event is mapped to its

transaction by the responsible contract (Chapter 2): the listing to a registration, the execution to a trade booking. *Transactions.* Two, in order:

1. *Registration* (moveless). ProductTerms FUT-IDX version 1: cash-settled future on index IDX, multiplier 10, USD, settled-to-market — the regime a declared term of the CCP agreement unit (Chapter 9). Unit-state change: watches declared — one per daily settlement-price publication, and one for the expiry.
2. *Trade* (day 0). One move: 1 FUT-IDX, owned, from the CCP’s virtual wallet to W-ALPHA. One position-state change: W-ALPHA’s traded price 995.00 recorded.

The door. Both transactions carry cause-derived identifiers, so a duplicated capture commits once; the move’s paired legs make conservation automatic; and no cash moves — a settled-to-market future exchanges nothing at trade, the first variation-margin move being day 1’s (Chapters 5 and 9). *Design point.* *The watches follow from the terms and are declared at registration: the future’s whole life — every print, the expiry — is asked for before the first event arrives.*

Episode — the variance swap (T5): the declaration is exhaustive. *Trigger.* VS-1 is executed and its confirmation captured at the boundary: a one-year OTC variance swap on IDX, variance strike 400 variance points, variance notional USD 1,000 per point, no cap, 252 scheduled fixings against the recorded IDX official closes — the same observation source FUT-IDX settles against. *Contract.* The confirmation maps to the registration transaction that installs the unit. *Transactions.* One, moveless:

1. ProductTerms VS-1 version 1: the terms above, the fixing schedule carried as declared data.
2. Unit-state change: 252 fixing watches declared — one per scheduled fixing date, each a watch on the recorded IDX official close; the 252nd is the final fixing, on whose firing settlement is computed (Chapter 7).

The door. A moveless transaction is admitted like any other: idempotent under its cause-derived identifier; nothing to conserve, everything to record. *Design point.* *Exhaustive means exhaustive: all 252 conditions the swap’s life requires are on the record before the first fixing prints, and no watch is discovered mid-life.*

Episode — the TRS (T6): a non-valued unit beside an ordinary one. *Trigger.* Two registrations are captured at the boundary: the strategy S-QIS, and the swap TRS-1 written on the strategy’s wallet. *Contract.* Each confirmation maps to its registration transaction. *Transactions.* Two, both moveless:

1. ProductTerms S-QIS version 1: the strategy’s rulebook, the complete terms of a *non-valued* unit whose contract manages wallet W-QIS in virtual ledger V-1 (Chapter 10). S-QIS has no price: undefined, not zero.
2. ProductTerms TRS-1 version 1: notional USD 10,000,000; reference leg the NAV of W-QIS; financing leg fixed 4% per annum, quarterly. Unit-state change: watches declared — each quarterly reset date, at which the contract reads the stamped NAV observation then on the record.

The door. Consistency of reference: TRS-1’s terms name a wallet and an observation source that exist on the record; the registrations commit once under their cause-derived identifiers. *Design point.* *The strategy’s governance shell is non-valued because pricing it beside the wallet it governs would count the same economics twice; the TRS that references it is an ordinary valued unit of the true ledger — referencing a virtual ledger makes nothing special.*

3.2 Wallets

A **wallet** is a named logical partition of position space — a container, and deliberately nothing more. Its state at any moment is, for each unit it holds, a signed quantity: its **balance**. A wallet names two parties: a **manager**, authorised to perform transactions on it, and a **beneficial**

owner, legally accountable for the book. Both are declared at creation, and the distinction is load-bearing: authorisation is checked against the manager, accountability and reporting attach to the beneficial owner. Wallets carry no smart contract and no logic of their own; where a wallet appears to need behaviour — the rules of a managed account — that behaviour is a unit (the mandate), and the wallet stays a container. W-QIS above is exactly this shape: the rulebook lives in the non-valued unit S-QIS, and the wallet only holds.

3.3 The Atomic Move

An **atomic move** is the sole operation that changes a balance: it transfers a positive quantity of exactly one unit from a source wallet to a destination wallet in a single indivisible step — the same quantity debited at the source and credited at the destination. Because a move only transfers, it can never create or destroy quantity; issuance and retirement are themselves paired-leg moves against the issuer’s wallet (Chapter 14 states the conservation law this yields and proves it by induction on the log).

Quantities are exact integers in minor units: a USD amount is an integer count of cents, a holding an integer count of shares or contracts, a recorded statistic an integer at its declared scale. No quantity in the ledger is a floating-point number. Where an event divides unevenly — a per-share figure across a holding it does not divide, a pro-rata allocation with an odd cent — the rounding convention is part of the declared terms, and the remainder is booked explicitly as its own move: the convention says where the odd cent goes, and the odd cent’s journey is on the record. Nothing is ever silently dropped, which is what makes conservation exact rather than approximate.

3.4 The Transaction

A **transaction** is an atomic, all-or-nothing bundle of four kinds of change: moves, unit-state changes, position-state changes, and terms changes. The four align one-for-one with the four components of the ledger’s state: balances, and the three homes of Chapter 6 — UnitStatus, PositionState, ProductTerms. A transaction applies in full or not at all, and some transactions carry no moves: the registration of a new unit and the recording of a barrier breach are complete transactions with empty move lists. The episodes above have already shown three of the four kinds; the split shows the fourth.

Episode — the split (T4): a terms change is a transaction component. *Trigger.* 2026-06-01: ACME’s 2-for-1 split becomes effective. The recorded split notice had declared the effective-date watch; the condition is met and the Event Monitor emits the event. *Contract.* ACME’s contract fires with the event and the current projection of the ledger: W-ALPHA holds 10,000 shares. *Transactions.* One, atomic:

1. Move: 10,000 ACME, owned, from the issuer’s virtual wallet to W-ALPHA — the holding becomes 20,000 by a paired-leg move, one such move per holder, never by an edit.
2. Terms change: ProductTerms ACME version 2, carrying the 2-for-1 event. How the recorded price frame 100.00 reads as 50.00 after the event — and the declared per-share dividend figure 1.50 as 0.75 — is the market data operator’s work, Chapter 8.

The door. Atomicity: the move and the new terms version apply in full or not at all — no instant exists at which 20,000 shares stand beside pre-split terms (Chapter 14’s consistency of reference begins here). Conservation: the issuer wallet’s balance falls by 10,000 as W-ALPHA’s rises. *Design point.* A terms change is one of the four components of a transaction, atomic with the moves it accompanies; terms are versioned and append-only, never edited.

3.5 The Immutable Log

The **immutable log** is the ordered, append-only, hash-chained record of every admitted transaction — hash-chained so that any alteration of history is detectable. It is the sole canonical source of truth. Three words are used precisely throughout this specification. The **log** is the recorded sequence itself. The **ledger** is the state obtained by folding the log: balances and the three homes. The **record** is everything that has been recorded — the log and all it carries: transactions, events, observations. “On the record” means present there, and a **projection** is any view computed from the record — which is why, in Chapter 2’s typed picture, the watch arrow reads the Record while contract and apply consume the Ledger.

3.6 Closure by Virtual Wallets

Every counterparty outside the system is represented by a **virtual wallet** inside it: the CCP a future faces, the issuer of a stock, the writer of an option each hold a wallet, and every move has both legs on the book. The system is therefore closed, and conservation holds by construction, not by check. The split episode shows the closure working: the 10,000 new shares credited to W-ALPHA are debited from the issuer’s virtual wallet in the same move — quantity appears nowhere from nothing, and the sum over all wallets, virtual ones included, stays at zero.

3.7 The Coordinate Vector

When a balance must carry more information than a single number, it generalises to a signed vector of coordinates: (**owned, lent, posted**). The five constitutional names are the rays of this basis — *lent out* and *borrowed* are the two signs of lent; *posted as collateral* and *received as collateral* are the two signs of posted — and the outright holding is the degenerate case with all mass on owned. A quantity earns a coordinate only when a distinct real-world action can change that coordinate independently of ownership: lending changes possession without changing ownership, pledging encumbers without transferring, and nothing else in scope passes the test, so there is no fourth coordinate. A move writes exactly one coordinate of one unit, paired legs as always, so conservation holds per (unit, coordinate), not merely per unit.

Only the owned coordinate carries economic value and drives profit and loss. The remaining coordinates carry mass, never value: they record possession and encumbrance under a named agreement, no quantity reaches them without reference to the agreement unit that governs it, and a lifecycle event extinguishes value, never mass — a unit leaves the ledger only from the zero vector. Anything computable from the coordinates — availability, encumbrance, possession — is a projection, computed when needed and never stored.

This section fixes the definition and nothing else. Which coordinate a given inflow writes, the agreement semantics of the non-owned planes, the guards checked at the door, and the regime key that decides among them are Chapter 9’s.

3.8 Term Sheets as Graphs

A unit’s terms are not free text with a contract bolted on; they have a shape, and the shape is a graph. Take the one-touch OT-1 of the thread map: payout USD 1,000,000, its barrier breached when OMEGA’s official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch. Its life has three states — live, triggered, expired. From live, exactly two things can happen: a recorded OMEGA official close at or below 80.00 (the unit becomes triggered and the payout falls due), or the expiry date arrives untouched (the unit becomes expired, worthless). From triggered and from expired, nothing further happens. Drawn, that is a graph: three nodes, two edges leaving one of them.

Definition 3.1 (Product graph). *The terms of a unit define its product graph: the nodes are the unit’s lifecycle states, a set closed and known at registration; each node carries a payload — the sufficiency facts of constitution §7, held in the three homes of Chapter 6 (the variance swap’s accrued fixing statistic, the future’s last settlement applied); each edge carries a guard — an event kind the Event Monitor can watch, together with a predicate — and an action — the transaction template the traversal emits.*

The structure is declared data: nodes, edges, guards, and payload schemas are recorded with the terms and readable without running anything. The arithmetic inside an action is not data and does not pretend to be: it is a small, named, versioned pure function — the variance swap’s realised-variance accumulator, the future’s margin difference — referenced from the edge that uses it. Declared structure carries behaviour, per the constitution’s Clarity commitment (§2); named pure functions carry computation; and nothing is hidden in code the graph does not point to.

The graph is identical with the watch discipline of this chapter. The watch list of a unit is exactly the out-edge set of its current node, so declaring the graph declares the watches — the exhaustiveness demanded of units above is not a rule to remember but a projection of the graph. Arming is edge registration; firing is traversal; and a traversal expires its sibling edges: when OT-1’s touch edge fires, the expiry edge dies with the live node that carried it. The declared $\rightarrow$ armed $\rightarrow$ fired/expired lifecycle of Chapter 4’s handshake is a graph fact, not a bookkeeping convention. The variance swap reads the same way: 252 fixing edges, and the 252 declared watches of its episode are their guards, exhaustive at registration because the graph is closed at registration.

Because the node set is closed at registration, each state can be a distinct type and each edge a function between those types — and the edges are the only constructors those types have:

```
data Live      = Live      { accrued  :: Payload }
data Triggered = Triggered { touchRef :: EventId }
data Expired   = Expired

touch  :: Fixing -> Live -> Triggered  -- the only constructor of Triggered
expire :: Expiry  -> Live -> Expired
-- Triggered -> Live: no such function exists, so no program un-knocks a
-- note; a case analysis that forgets a node does not compile.
```

An illegal transition is not refused; it is unrepresentable — there is no program that writes it. Totality is compiler-checked: exhaustive case analysis over a closed node set is exactly what these types force, so a contract that forgets the expired state is a compile error, not a production incident.

One requirement makes the identification normative rather than decorative.

Invariant 3.2 (Graph consistency). *For every unit there exists a product graph from which its watches, its state schema, and its contract’s actions are all derivable, and their mutual consistency holds: (i) at every commit, the unit’s watch list equals the out-edge set of its current node; (ii) every fired event matches exactly one out-edge and lands the unit on that edge’s target node; (iii) every transaction the contract emits equals the edge’s declared action applied to the recorded inputs; (iv) a traversal expires exactly the declared sibling edges, and no others.*

The four properties are executable, not defended in prose: their executable forms join the invariant catalogue of Chapter 14 and the test regime of Chapter 15, where they are property-tested over generated products, events, and histories.

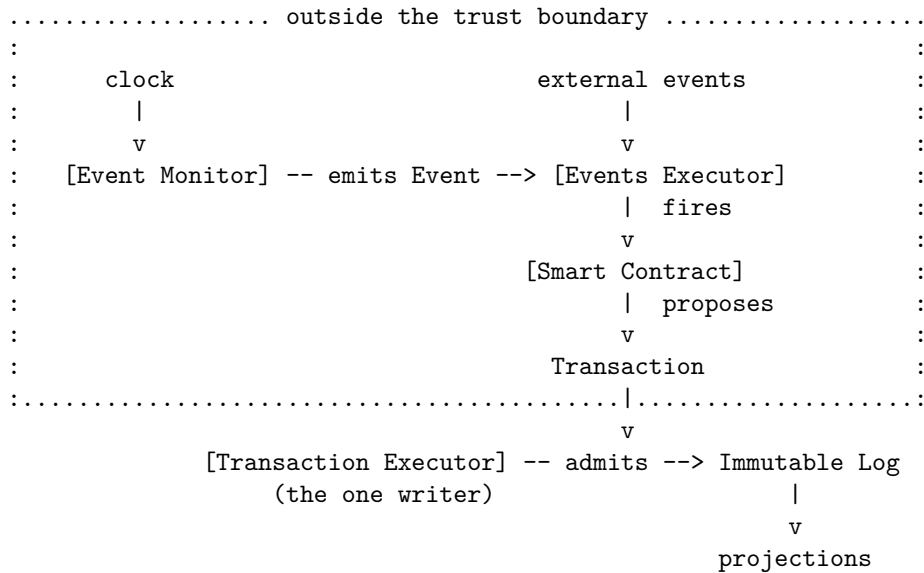
A second telling, in categorical language; nothing later depends on it.

The product graph is one object, presented by its generators (nodes and edges) and the relations the invariant states. The Event Monitor’s watch list, the typed state schema, and the contract’s action table are three interpreters of that object — structure-preserving maps out of the same graph — and the consistency invariant says exactly that the three maps agree on it. Nothing interprets a term sheet except through its graph.

4 The Machines

This chapter discharges §5 of the constitution.

The framework has three machines, one for each arrow of the typed picture of Chapter 2 (*The Picture: Map, Then Fold*): the Event Monitor evaluates **watch**, the Events Executor evaluates **contract**, and the Transaction Executor evaluates **apply**. All three are generic machinery. No machine holds a line of product knowledge: what a dividend, a touch, or an expiry *means* lives in the smart contracts (Chapter 5); the machines only notice, deliver, and admit. And only the last of the three ever writes. The Event Monitor emits events; the Events Executor collects proposals; the Transaction Executor alone changes the state of the ledger. Three roles, one writer. In one picture — machines in brackets, data on the arrows, the dotted line the trust boundary:



4.1 The Event Monitor

The Event Monitor owns time and attention. It holds every declared watch — the dates, observations, and conditions each unit’s terms require watching (Chapter 3) — and evaluates them against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. It holds no private list: the watches it evaluates are on the record, declared by the units, so the Monitor’s workload is itself a projection. And it emits events, never transactions: nothing the Monitor does can place so much as one move before the door.

The clock is the single input to the whole system that is not on the record, and the Event Monitor is the single place it is consulted. The event the Monitor emits *is* recorded, so everything downstream of the emission is a function of the record alone. Two consequences follow, and both are load-bearing. A late firing produces the identical transaction: the contract reads the recorded event and the recorded state, never the wall clock, so delay costs timeliness and nothing else. And emission itself is verifiable: because watches are declared data and the Monitor is

deterministic, any party holding the record and the clock recomputes which events should have been emitted, and when — the Monitor is auditable against its own inputs.

The relationship between the units and the Monitor is a recorded handshake. Each declared watch is acknowledged by the Monitor, and the acknowledgement — an event like any other, flowing through the same pipeline — is written into the unit’s state. A watch is thus visibly *declared*, then *armed*, then *fired* or *expired*, and each step is on the record. A declared watch that is never armed is therefore a visible gap, never a silent one: it stands in the unit’s state as declared-but-unarmed, an overdue item any reader can query. The full chain can be walked in either direction — register a unit, the conditions it declares, the acknowledgements, the events, the transactions, the state updates — which is the auditability commitment (§2) applied to attention itself.

The Monitor can affect timeliness and liveness; it cannot affect the correctness of ledger state, because it only emits events, and an event changes nothing until a contract’s proposal passes the door.

4.2 The Events Executor

The Events Executor owns delivery. It does two things and nothing else. First, it captures the events arriving from outside — trades executed in the market, corporate-action notices, fixings and other observations — and records them at the boundary. Second, for each event, arrived or emitted, it fires the responsible smart contract, handing it the event together with the current projection of the ledger, and collects the proposed transaction. Many contracts, one Events Executor: the contracts carry all the product knowledge; the Events Executor carries none.

A trigger carries timing, not data: the firing says *when*, and everything the event means is read from the record. The Events Executor runs the contracts; it never writes the ledger; its output is only proposals. A proposal is not a fact — it becomes one only if admitted.

4.3 The Transaction Executor

The Transaction Executor owns the record. It is the gatekeeper of the ledger door: the single component through which every transaction must pass, and the only component permitted to change the ledger’s state. On each proposal it validates structure. Conservation is automatic — a move is paired-leg by construction, so the per-(unit, coordinate) sums cannot be disturbed (Chapter 3); authorisation, idempotence, consistency of reference, and writer discipline are enforced as checks at the door. If the proposal is admissible, the Executor appends it to the immutable log; if not, it refuses, and the refusal is a returned value, never a half-applied state. It creates nothing and runs nothing. There is no other way to change the ledger’s state.

Idempotence is the door’s own defence, owed to no one upstream: every proposed transaction carries an identifier derived from its cause — the recorded event that produced it — so a retried, duplicated, or late proposal for an already-processed event is recognised and committed exactly once.

The single-writer discipline is a correctness requirement, not an implementation preference. The Transaction Executor presents to the log a strictly monotonic sequence of committed transactions: every projection folds the same total order, which is what makes two projections of one record unable to disagree. Where two simultaneous events on the same instrument propose transactions that do not commute and whose terms declare no precedence, the Executor refuses rather than guesses: an order the record cannot justify is an order nobody can recompute, and defaults fail closed (§2). The refusal is visible, and resolution — a declared precedence, a corrected proposal — arrives through the same door.

4.4 One event, three machines, one write

The one-touch OT-1 pays USD 1,000,000 when OMEGA’s official close is at or below the barrier 80.00 — a discretely-monitored (closing-price) one-touch; it is written by W-BANK and held by W-ALPHA, and its terms declared the watch at registration: the recorded official closes of OMEGA against the barrier.

Trigger. OMEGA’s official close of 79.00 is recorded. The declared condition is met, and the Event Monitor emits the touch event onto the record. No ledger state has changed: the Monitor emits events and cannot write state.

Contract. The Events Executor fires OT-1’s smart contract, handing it the touch event and the current projection. The contract reads the terms (payout USD 1,000,000), the holder (W-ALPHA), and the unit’s state (*live*), and returns one transaction.

Transaction.

	Kind	Content
1	unit-state change	OT-1: UnitStatus <i>live</i> → <i>triggered</i>
2	move	owned(USD) 1,000,000: W-BANK → W-ALPHA

Where the unit stands encumbered under a collateral agreement, the payment leg routes through that agreement’s conditional obligation; Chapter 9 works this same knock start to finish. The machines are identical in both tellings.

The door. The Transaction Executor checks the paired legs (the per-(unit, coordinate) sums are undisturbed), the cause-derived identifier (the touch event, not yet processed), and writer discipline (OT-1’s state is written by OT-1’s contract), and admits. One append; the fold advances; the ledger now shows the cash with W-ALPHA and the option *triggered*.

Duplicate emission. Emission is at-least-once (Chapter 16), so the Monitor may emit the touch again. The Events Executor fires the contract again; the contract finds UnitStatus already *triggered* and proposes nothing. Had a duplicate proposal been produced anyway, its identifier names an already-processed cause and the door commits it zero further times. Two independent defences, either one sufficient.

One event, three machines, one write — and the recorded state is exactly what makes a duplicate harmless.

4.5 The watch handshake

Trigger. On 2026-05-04 the issuer of stock ACME announces a cash dividend of 1.50 per share: record date 2026-05-15, payment date 2026-05-22. The announcement is an external event; the Events Executor captures and records it at the boundary.

Contract. The announcement fires ACME’s smart contract once — not to pay, but to register the watches the dividend creates. Nothing is owed yet and nothing moves; what the dividend has created is two future obligations of attention, and the unit declares them exhaustively, exactly as it declared its watches at registration.

Transaction. One moveless transaction:

	Kind	Content
1	unit-state change	ACME: record-date watch declared, 2026-05-15
2	unit-state change	ACME: payment-date watch declared, 2026-05-22

The door. A moveless transaction is well-formed: a transaction is a bundle of four kinds of change and may carry no moves (Chapter 3). The identifier derives from the recorded announcement; conservation is undisturbed because nothing moved; the declaring writer is ACME’s own contract.

The handshake. The Monitor acknowledges each declared watch; each acknowledgement is an event like any other, flows through the same pipeline, and lands as a unit-state update: both watches now visibly *armed*. On 2026-05-15 the Monitor emits the record-date event and that watch is *fired*; on 2026-05-22 the payment-date watch fires likewise. What the two firings do — record each holder’s entitlement movelessly, then pay against the recorded entitlements — is Chapter 5’s episode, and the entitlement’s home is Chapter 6’s; the machines’ contribution is that neither date could have passed silently. Had either watch remained unacknowledged beyond its arming window, the unit’s state would show a declared, unarmed watch: an overdue item, never an absence.

A watch is visibly declared, then armed, then fired or expired; a date that matters cannot pass silently, because attention itself is on the record.

4.6 The trust boundary

The three roles sit at different levels of trust, and the dotted line of the opening picture is exact: everything above it is untrusted. The Transaction Executor is the trusted component — the ledger’s integrity rests on it and on nothing else. The smart contracts, the Event Monitor, and the Events Executor sit outside the boundary, untrusted in three different ways.

A contract can be economically wrong. Its proposal can be structurally impeccable and still book the wrong economics, and the door cannot know that. The defence is not prevention but recomputation: the events and the transactions are both on the record, so verification re-runs the map on the recorded event and compares (Chapter 15); a discrepancy is a defect of the contract, not of the ledger, and it is repaired by a compensating transaction through the same door.

The Event Monitor and the Events Executor cannot corrupt the ledger at all. A late emission or a late firing produces the identical transaction, because everything downstream reads the record and not the clock. A duplicate is committed once, under the cause-derived identifier. A spurious firing finds nothing due — the one-touch’s second touch above — and proposes nothing, or proposes something the door refuses. Every failure these two machines can commit maps to delay or noise, never to wrong state.

What the untrusted machines do carry is liveness. Integrity owes them nothing; timeliness owes them everything: watches must actually arm, events must actually be emitted, contracts must actually be fired, obligations must actually be discharged by their deadlines. Those duties are stated as minimum requirements — durable watches and timers, at-least-once emission, acknowledged watches, triggers carrying timing not data, replayable orchestration, no write privilege, obligation liveness — in Chapter 16, and each degrades, when unmet, to a visible overdue item rather than to silence.

5 Smart Contracts

This chapter discharges §6 of the constitution.

The ledger admits well-formed transactions and records them; it performs no product-specific logic of its own. What a dividend, an expiry, a barrier knock, or a settlement print means for a particular instrument lives in that instrument’s *smart contract*. An event arrives from the outside or is emitted by the Event Monitor; the ledger can do nothing with it until it is a transaction; the smart contract is the one function that performs the conversion. Since the ledger is a transaction processor and nothing more, everything reaching it from outside must be turned into a transaction, and the contract is the converter.

One signature states the entire interface.

```
contract :: Event -> Ledger -> Transaction
```

```

-- what a contract is NOT (each of these would be a defect):
--   contract :: Event -> State Memory Transaction    -- it may not remember
--   contract :: Event -> IO Transaction              -- it may not reach out
--   contract :: Event -> Clock -> Transaction        -- it may not read time

```

Read the signature as a claim about what a contract may touch. Its only inputs are the event and the ledger as it currently stands: the declared terms of the instrument, the recorded observations, and the visible unit and position state at the moment of firing. There is no clock among the inputs — the clock is consulted once, by the Event Monitor of Chapter 4, and never again downstream — no private store, and no channel to the world. A smart contract is therefore a pure, deterministic function of its inputs, and, because none of those inputs is private to it, a stateless one. Each contract is exactly this: a stateless, machine-readable transcription of the legal terms and conditions of the instrument it governs. Purity is what makes economic verification possible at all: to check a recorded transaction, re-run the same contract on the same recorded event and compare (Chapter 15). A function that read a clock, a store, or the network could not be re-run to the same answer, and the guarantee would be lost.

Two principles govern every contract, and the rest of this chapter is their working-out.

Principle 5.1 (Everything a contract consumes is on the record). *Every input a smart contract reads — the instrument’s terms, the observations it depends on, the positions it acts upon — is present on the record at the moment of firing. A contract reads nothing that is not recorded, so its output is a function of the record alone.*

Principle 5.2 (A contract never waits and never remembers). *The waiting is done by the watch, which sits in the record, and by the Event Monitor, which notices when the watch’s condition is met; the contract is invoked only after its event has fired, runs once, and returns. Whatever a firing must carry forward to a later firing it writes into the record — into one of the three homes of Chapter 6 — and the later firing reads it back. A contract that had to remember would need private state, and private state is a second store of the very facts the ledger already holds: the one thing this architecture exists to abolish.*

5.1 The dividend: three firings, one contract

The dividend shows both principles in one instrument fired three times. Wallet W-ALPHA holds 10,000 shares of the stock ACME.

The trigger. On 2026-05-04 the issuer announces a cash dividend of USD 1.50 per share on ACME. The announcement is captured at the boundary and recorded — an external event, like a trade.

The contract that catches it. The stock’s smart contract fires on the announcement, and fires again on each date the announcement creates. The three firings are one contract, invoked three times on three different events; it carries nothing between them.

The transactions it produces.

Firing	Date	Event	Transaction emitted
1	2026-05-04	announcement	arm the record-date and payment-date watches; <i>no move</i>
2	2026-05-15	record date	PositionState: W-ALPHA entitlement = 10,000 × USD 1.50 = USD 15,000.00; <i>no move</i>
3	2026-05-22	payment date	move USD 15,000.00, issuer → W-ALPHA; PositionState: entitlement discharged

The first firing pays nothing; it registers the two watches the dividend creates and writes them into the unit’s state. The second reads the owned positions as the ledger then stands and

records each holder’s entitlement — a moveless transaction: no cash changes hands, but the position state now carries what each holder is owed. The third reads those recorded entitlements and emits the cash moves against them. Nothing links the firings but the record: firing 2 does not remember firing 1, it reads the armed watch; firing 3 does not remember firing 2, it reads the recorded entitlement.

What the door checks. On firing 3 the Transaction Executor checks conservation — the USD 15,000.00 debited from the issuer wallet equals the amount credited to W-ALPHA, a paired-leg move that can neither create nor destroy quantity — and idempotence: the payment proposal carries an identifier derived from its cause, so a retried or late firing commits once (Chapter 4).

Where an agreement encumbers the holder — the shares pledged, the cash trapped — the payment routes through a conditional obligation unit instead of paying directly; that refinement is Chapter 9’s, and it changes what firing 3 emits, never that the contract is stateless and fired three times.

Three firings, three transactions, one stateless contract; between them the contract remembers nothing — the armed watches, the recorded entitlement, and the positions are all on the record.

5.2 The trigger classes differ in what they read, never how they run

The three firings of the dividend were reached three different ways. The announcement arrived from outside. The record date and the payment date were calendar conditions the Event Monitor noticed against the clock. A fourth way remains: a predicate on a recorded observation — a settlement price printing, a close falling below a barrier. These are the trigger classes, and they classify what a contract reads, never how it runs.

```
data Trigger
  = External Observation -- captured at the boundary (a dividend announcement)
  | Calendar Date       -- a date arriving (a record, payment, or expiry date)
  | Condition Predicate -- a predicate on a recorded observation (a print, a
                        touch)
  -- every trigger, whatever its class, fires the same contract above.
```

The class decides which watch the Monitor evaluates and which inputs the firing reads; the signature above is unchanged across all of them. A future exercises two of the classes on one instrument.

5.3 The future: settlement prints and the expiry unwind

FUT-IDX is a cash-settled future on the index IDX, multiplier 10, settled to market. W-ALPHA holds one contract, entered at 995 against the clearing house wallet. Its terms declare a watch on each daily settlement print and a watch on its expiry.

The trigger and the contract. On each recorded settlement print the Monitor emits and the future’s contract fires. It reads two facts, each already on the record: the day’s settlement level, a UnitStatus fact identical for every holder, and W-ALPHA’s last applied level, a PositionState fact. From the two it computes the day’s variation margin and emits it. On the third print, which is also the expiry, the same contract additionally unwinds the position — in one atomic transaction.

Day	Print	Variation margin	Move	PositionState
1	1,000.00	$(1000 - 995) \times 10 = +50$	USD 50, CCP $\rightarrow$ W-ALPHA	level $\leftarrow$ 1,000.00
2	990.00	$(990 - 1000) \times 10 = -100$	USD 100, W-ALPHA $\rightarrow$ CCP	level $\leftarrow$ 990.00
3	1,005.00	$(1005 - 990) \times 10 = +150$	USD 150, CCP $\rightarrow$ W-ALPHA	position unwound

The contract never remembers yesterday’s level; it reads it from PositionState, computes against today’s print, and writes today’s level back for tomorrow’s firing. Cumulative variation

margin is $+50 - 100 + 150 = +100 = (1005 - 995) \times 10$ — exactly the position’s profit, arrived at one recorded firing at a time.

What the door checks. The margin move is conserved — what W-ALPHA receives the clearing house pays, and conversely — and each firing multiplies the multiplier by *the recorded settlement level*, never a stale one, so no firing books a quantity against a price from a different state (Chapter 14).

The daily print and the expiry unwind are the same contract fired on different events; the settlement print and the dividend announcement differ in what the contract reads, never in how it runs.

5.4 The one-touch: one firing, one transaction, and idempotence

OT-1 is a one-touch on the stock OMEGA that pays USD 1,000,000 the first time OMEGA’s official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch. It is written by W-BANK and held by W-ALPHA; its terms declare one watch, on OMEGA’s recorded official close against the barrier — and because the barrier is observed at that close, the watch is exactly the barrier edge’s declared guard, so the term is transcribed faithfully (Chapter 3’s product-graph consistency).

The trigger and the contract. OMEGA closes at 79.00; the close is recorded, the condition is met, and the Monitor emits the touch. The contract fires and reads exactly two things from the record: the declared payout and who holds the option. It returns one transaction — a single move of USD 1,000,000 from W-BANK to W-ALPHA, together with one unit-state change, $OT-1 \rightarrow triggered$.

What the door checks. Conservation on the payout, and idempotence. Idempotence is not something the contract defends: should the Monitor emit the touch a second time — barriers are often crossed on many later days — the next firing reads the unit and finds it already *triggered*, so it proposes nothing, and even a duplicate of the original proposal carries the same cause-derived identifier and commits once at the door.

Idempotence is guaranteed not by the contract’s vigilance but by the recorded state and the cause-derived identifier; the touch contract is a pure function of the record — terms and holder in, one transaction out.

5.5 The variance swap: a contract never remembers

VS-1 is a one-year variance swap on IDX with 252 scheduled fixings against the same recorded official closes the future settles on. Each scheduled fixing is a declared watch; on fixing k the Monitor emits and the contract fires. To extend the realised variance the firing needs the previous close and the running accrued statistic — the fixings $1 \dots k-1$. It does not hold them: the previous close is a recorded observation, and the accrued statistic is a UnitStatus fact that firing $k-1$ wrote for exactly this purpose (Chapter 6). Firing k reads both, forms the return, adds its square, and writes the updated accrued statistic back for firing $k+1$. A firing 200 days into the swap’s life reconstructs its entire history from the record and holds nothing of its own between firings.

Ordering. The fold obeys the log’s order, not the wire’s: the immutable log’s total order for a unit’s fixings is the recorded observation index — the fixing sequence k — not the order the prints happen to arrive, so replay is deterministic. A duplicate print is inert, absorbed once under its cause-derived identifier; a late or out-of-order print is not a no-op — it is re-sequenced to its observation position k , and the accrued-statistic tail $A_k, A_{k+1}, \dots$ is recomputed from k forward. The recomputed result is identical to timely arrival because the fold is over the ordered sequence, and any figure already posted against a superseded tail is corrected by a compensating transaction, never left standing (Chapter 15).

Firing k finds fixings $1 \dots k-1$ on the record; a contract never remembers.

5.6 Faithfulness: every right and obligation is a unit

Purity and statelessness say how a contract runs. One further demand says what it must produce: the alignment with the legal contract runs deep enough that every right and obligation the legal contract creates is represented in the ledger as a unit. A subscription is booked as an exchange — cash against the client’s redemption claim — never as a transfer for nothing. A dividend not yet paid is a recorded entitlement, seen above. A payout trapped by a collateral agreement is a conditional obligation unit (Chapter 9). Because each such right and obligation is a unit or a declared term of one, the contract that transcribes the instrument has, at every firing, a home on the record for everything it must carry — which is why it can afford to remember nothing. Statelessness and faithfulness are the same discipline seen from two sides: the contract keeps no private state precisely because the record already holds, as units and their declared terms, every fact its next firing will need.

6 State: The Three Homes

This chapter discharges §7 of the constitution.

A unit’s state is not one thing. Some of it is a property of the unit alone — the settlement level a future has printed, the fixings a variance swap has accrued, whether a note’s barrier has knocked — the same fact for every holder. Some of it is a joint property of the unit and a particular wallet — the variation margin this holder applied yesterday, the dividend that holder is owed. And some of it is the contractual text itself — a multiplier, a strike, a fixing schedule — versioned and append-only. Three kinds of fact, and the state model gives each its own home: **UnitStatus**, **PositionState**, and **ProductTerms**. With the balances of Chapter 3 these are the four components of the ledger’s state, and the four kinds of change a transaction bundles (Chapter 3) write them one for one.

6.1 The three homes

ProductTerms holds the versioned, append-only contractual terms of an instrument: the future’s multiplier, the option’s barrier and payout, the variance swap’s strike and 252-fixing schedule. A new version is appended, never an edit; the current terms are the latest version, and any past state is read against the version in force then. Its facts are properties of the instrument’s terms, and a corporate action that changes those terms writes a new version (the split below).

UnitStatus holds the facts that are properties of the unit as a whole — the same for every holder: the last recorded settlement level, the running accrued sum of squared returns, the live / triggered / expired lifecycle node. It is a materialised projection of the unit-state changes on the log, written by the instrument’s contract as its life unfolds.

PositionState holds the facts that are joint properties of a unit and a wallet: this holder’s last applied settlement level, this holder’s recorded dividend entitlement. A fact lands here exactly when it depends on which wallet holds the unit — when one unit-level event yields a different value to each holder.

None of the three holds a balance — that is the coordinate vector of Chapter 3, the transaction’s fourth kind of change — and none holds a view. The keys distinguish them: **ProductTerms** and **UnitStatus** are keyed by the unit, **PositionState** by the wallet-and-unit pair.

```
data Homes = Homes
  { productTerms  :: Unit          -> [TermsVersion]  -- append-only; latest is current
  , unitStatus    :: Unit          -> UnitFacts       -- properties of the unit alone
```

```

    , positionState :: (Wallet, Unit) -> PosFacts          -- joint unit-and-wallet
      facts
    }
  -- each field is a fold of the log; discard and replay reconstructs it.

```

Two disciplines make the homes trustworthy, and both are checkable.

Principle 6.1 (One fact, one home). *Every non-balance fact has exactly one home, determined by what the fact is a property of: the unit alone (UnitStatus), the unit-and-wallet pair (PositionState), or the contract’s terms (ProductTerms). No fact is kept in two homes, so no two homes can disagree about it.*

Invariant 6.2 (One legal writer). *Every non-balance fact has exactly one legal writer — the smart contract of the instrument the fact belongs to — and no other contract may write it. The Transaction Executor enforces this at the door: a transaction that writes a fact outside its writer’s authority is refused. The invariant joins the structural catalogue of Chapter 14 with constitution §11’s writer discipline.*

6.2 One fact, one home: the future exercises all three

The daily life of a cash-settled future carries all three kinds of fact, so it visits all three homes. FUT-IDX is a future on index IDX, multiplier 10, USD, settled-to-market; W-ALPHA holds one contract. On day 1 the official close prints 1,000.00, the Event Monitor emits, and the future’s contract fires. Three facts are in play, and each has one home.

Fact	Property of	Home	Legal writer
Settlement print 1,000.00	the unit alone	UnitStatus	the print firing
W-ALPHA’s applied level; VM USD 50	the unit–wallet pair	PositionState	the print firing
Multiplier 10	the contract’s terms	ProductTerms	registration (v. 1)

The print is one number for every holder: it is a UnitStatus fact. The variation margin depends on the level W-ALPHA last applied, a fact of the wallet-and-unit pair: it is a Position-State fact, and the same print yields a different margin to a holder who entered at a different level. The multiplier was fixed at registration in ProductTerms and changes only by a new version. The print firing writes the first two facts in one transaction — two facts, two homes, one writer — and reads yesterday’s applied level back from PositionState rather than holding it (Chapter 5).

Each fact has exactly one home, fixed by what it is a property of; the future, carrying three kinds of fact, exercises all three homes.

6.3 The homes suffice

The homes are not bookkeeping conveniences. Their reason for existing is a sufficiency requirement, and it is the load the rest of the architecture rests on.

Principle 6.3 (The homes suffice). *The three homes, together with the balances, carry all the information the smart contracts require in order to execute. Every fact a firing must carry forward to a later firing is written into a home by an earlier firing, so the later firing finds its entire context on the record; if a contract would ever need to remember something, that something has a home.*

This is the precise reason a contract is stateless and never remembers (Chapter 5): statelessness is affordable only because every fact a firing would want to remember already has a

home the firing can read. The sufficiency is what turns “never remembers” from an aspiration into a consequence.

Minimality is its other half. A home holds a *fact* — an input a later firing reads — never a *view*. NAV, profit and loss, availability, and encumbrance are projections, computed on demand from the homes and the log and never materialised into one (Chapters 7 and 12). The homes carry every fact a contract must read and no figure a valuation or a report recomputes — which is exactly why no store of “realised PnL” exists anywhere to be reconciled against the position record.

The variance swap exercises the requirement at full stretch.

Episode — the variance swap (T5): sufficiency, exercised. *Trigger.* The 127th scheduled IDX close is recorded and the Event Monitor emits VS-1’s 127th fixing. VS-1 is a one-year OTC variance swap on IDX, 252 scheduled fixings against the recorded IDX official closes — the same series FUT-IDX settles on. *Contract.* VS-1’s contract fires. To extend the realised variance it needs the whole path so far: the previous close and the running sum of squared returns over fixings $1, \dots, 126$. It holds neither, and it is forbidden to. *What it finds on the record.* The previous close is a recorded observation. The accrued statistic is a `UnitStatus` fact of VS-1 — a property of the unit alone, the realised path as the swap has seen it — written by the 126th firing for exactly this reader. At the mid-life checkpoint (fixing 126, half of the 252 fixings elapsed) it stands at $A_{126} = 1,805,000$, an integer at scale 10^{-8} : the 126 accrued squared returns compressed into one number. The firing reads A_{126} and the previous close, forms the 127th return, adds its square, and writes A_{127} into `UnitStatus` for the 128th firing. *The coupling.* The IDX close at fixing 126 is 1,005.00 — the same recorded close FUT-IDX takes as its day-3 final settlement; fixings 124, 125, 126 are the future’s three settlement days, closes 1,000, 990, 1,005. One recorded observation, two consumers: the future writes it as a settlement level into FUT-IDX’s `UnitStatus`, the swap folds it into VS-1’s accrued statistic in VS-1’s `UnitStatus`, and neither reads the other’s home. *Design point.* *Firing 127 finds its entire context on the record — 126 fixings compressed into one UnitStatus integer $A_{126} = 1,805,000$ — so a contract that spans a year of history remembers nothing between firings. The homes suffice.*

6.4 Two more facts finding their homes

The dividend and the split add one home each: the entitlement is per-holder, the split is a terms change.

Episode — the dividend (T3): an entitlement in `PositionState`. On 2026-05-15, the record date, ACME’s contract fires and reads the owned positions as the ledger then stands: W-ALPHA holds 10,000 ACME. It records the entitlement $10,000 \times \text{USD } 1.50 = \text{USD } 15,000.00$. A dividend entitlement is a property of the unit-and-wallet pair — the same USD 1.50 per share yields a different amount to each holder — so it lands in `PositionState`. The transaction is moveless: no cash moves on the record date, but the position state now carries what W-ALPHA is owed, for the payment-date firing to discharge (Chapter 5). *Design point.* *An entitlement owed but unpaid is a fact with a home; the record date writes it into PositionState movelessly, and the payment date reads it back.*

Episode — the split (T4): a new version in `ProductTerms`. On 2026-06-01 ACME’s 2-for-1 split becomes effective. In one atomic transaction the holding moves from 10,000 to 20,000 shares (a paired-leg move, Chapter 3) and ACME’s terms gain a new version: `ProductTerms` ACME version 2 carries the split, and the recorded price frame reads 100.00 as 50.00 after it (the market data operator’s work, Chapter 8). A split is a property of the contract’s terms, so it lands in `ProductTerms` and nowhere else. The terms are not edited; a version is appended. *Design point.* *Terms change only by appending a version; the split is one ProductTerms fact, atomic with the move it accompanies, and the append-only discipline is what lets any past state be read against the terms in force then.*

6.5 Rebuildable by replay

None of the three homes is an independent store. Each is a deterministic projection of the immutable log, so it can be discarded and rebuilt at any time without loss.

Principle 6.4 (Homes are projections). *Each home is a deterministic projection of the log — `ProductTerms` of the terms changes, `UnitStatus` of the unit-state changes, `PositionState` of the position-state changes. Any home may be discarded and rebuilt by replaying the log, without loss of information, so no home is a store that can drift from the record.*

This is the precise sense in which the homes are *materialised projections* and not second copies. A second copy is dangerous because two stores of one fact, maintained by independent logic, eventually disagree — the failure the framework exists to remove (Chapter 1). A home cannot play that role: it has exactly one legal writer (the one-writer invariant above), it holds each fact in exactly one place (one fact, one home), and it is a pure fold of the one log everything else also folds. Materialising the accrued statistic into `UnitStatus` is therefore not the reintroduction of a store to reconcile; it is a cached read of the log, provably equal to the log it came from. What is materialised is a fact a contract must read; what is never materialised is a view a projection can recompute. Between them the homes carry everything the contracts need and nothing they do not.

The accrued statistic is order-dependent: each firing reads the previous accrual and extends it, so its projection is defined by an order, and that order is fixed. The immutable log’s total order for a unit’s fixings is the recorded observation index k — the fixing sequence — not the order in which prints arrive; replay folds the accrual in that order and is deterministic. A late or out-of-order print is inserted at its observation position k , and the accumulator tail $A_k, A_{k+1}, \dots$ is recomputed from k forward — identical to timely arrival because the fold is over the ordered sequence, while the downstream daily CCP-cash figures at positions after k — the future’s variation margin on the same recorded closes, not anything the once-settled variance swap pays — are corrected by compensating transactions, not left as posted. Rebuilding a home is therefore a re-fold in observation-index order, never a replay in arrival order.

7 Valuation and Profit and Loss

This chapter discharges §8 of the constitution, as amended by Amendment 1.

Net Asset Value (NAV) has an operational meaning before it has a formula. The initial NAV of a wallet is the cash it took to build the portfolio at inception; the current NAV is the cash you would hold if you unwound every position at prevailing prices; and the profit and loss (PnL) is the difference between the two. Nothing in this chapter is a new store: every number it produces is a projection of the immutable log, computed on demand and never written back.

7.1 Net Asset Value

The operational account becomes a formula in two steps. First, replay the log to recover the wallet’s current composition — the signed coordinate vector, per unit, that the fold of Chapter 2 produces. Second, multiply that composition by the current price vector supplied by the pricing data layer, summing over the valued units only.

Definition 7.1 (Net Asset Value). *The NAV of a wallet w at a log position is*

$$\text{NAV}(w) = \sum_{u \text{ valued}} \text{owned}(w, u) \cdot P(u, \sigma(u)),$$

where $\text{owned}(w, u)$ is the owned coordinate of w ’s balance in unit u (Chapter 3), $\sigma(u)$ is the current lifecycle state of u read from the ledger, and P is the price the pricing data layer supplies for that unit in that state.

Only the owned coordinate enters. The lent and posted coordinates carry mass, never value (Chapter 3): a security posted as collateral is still owned by the poster and is already counted once, on its owned coordinate; counting the posted marker as well would value one economic reality twice. Non-valued units — a strategy rulebook, a mandate’s governance shell — have no price and enter no sum; their value is undefined, not zero.

Two disciplines are visible in the definition and are carried by the machinery of Chapters 3 and 8. The composition is one number per coordinate, obtained from one fold of one log, so no two readers can disagree about what is held. And a price is never combined with a quantity across a corporate-action event: after a two-for-one split, 2,000 shares are multiplied by the post-split frame 50.00, never by the stale 100.00 — consistency of reference, catalogued in Chapter 14.

```

type Points = Integer                -- money in minor units, exact
price :: Unit -> State -> PriceVector -> Points  -- pricing is state-
parameterised
nav    :: Composition -> PriceVector -> Points
nav c pv = sum [ q * price u (stateOf c u) pv      -- owned coordinate, valued
units
              | (u, q) <- ownedBalances c, valued u ]

```

7.2 Profit and loss is the variation in NAV

Principle 7.2 (PnL is NAV variation). *The profit and loss of any wallet between two log positions is exactly the variation in its NAV over that interval. PnL is therefore path-independent, and no second copy of “realised PnL” or “book cost” is ever maintained alongside the position record.*

The principle is a consequence of the definition, not an additional convention. NAV is a function of the composition and the price vector alone; its variation between two positions depends only on the endpoints, never on the sequence of transactions that connected them. A stored realised-PnL figure would be a second record of a fact the log already determines — precisely the redundancy the framework exists to remove. Realised and unrealised PnL remain available as *projections*, cut from the same log by a reporting convention (Chapter 12), but neither is a source of truth.

Episode — the future. FUT-IDX is a cash-settled future on index IDX, multiplier 10, USD, settled-to-market. Wallet W-ALPHA buys one contract at 995 on day 0. The official settlement prints on the three settlement days are 1,000, then 990, then 1,005; the contract’s smart contract fires on each print and emits the day’s variation-margin (VM) move.

Settlement day	Print	VM move (owned cash)	Running NAV change
Day 1	1,000	+50	+50
Day 2	990	−100	−50
Day 3	1,005	+150	+100

Each VM is a settlement — an owned-cash move that extinguishes the day’s exposure, with no return obligation, because none exists in law for a settled-to-market contract (§8, case 1; developed in Chapter 9). The cumulative VM is $50 - 100 + 150 = 100$, and the position’s PnL is $(1,005 - 995) \times 10 = 100$. The two agree, and the second computation never touches the intermediate prints 1,000 and 990: the path is irrelevant, only the endpoints 995 and 1,005 matter. *PnL is the variation in NAV, and it is path-independent.*

7.3 Pricing is parameterised by state read from the ledger

Because a unit's value depends on where it stands in its lifecycle, the price function is parameterised not only by the unit but by its state — and that state is obtained from the ledger, from `UnitStatus` and `PositionState` (Chapter 6), never from a store the pricing layer keeps of its own. The pricing layer computes; the ledger is the sole authority for the state it computes in.

Episode — the one-touch. OT-1 is a one-touch on stock OMEGA, barrier 80.00 observed at the official close (a discretely-monitored closing-price one-touch), payout USD 1,000,000, held by W-ALPHA. While live, the pricing layer marks it at USD 400,000. When OMEGA's official close prints 79.00, the Event Monitor emits the touch and the option's contract records the lifecycle transition: `UnitStatus` $\rightarrow$ *triggered*. The pricing layer, reading that state from the ledger, now prices the same unit in state *triggered* at 0: the contingent claim has resolved, its economic value realised as the cash payout that Chapter 9 discharges.

The mark on OT-1 collapses from USD 400,000 to 0, but the mass does not move: one unit of OT-1 remains on W-ALPHA's owned coordinate, and any posted marker over it remains on its posted coordinate, until the payout completes and the unit retires from the zero vector. *A lifecycle event extinguishes value, never mass: coordinates carry mass, prices carry value, and the price turns on a state that lives on the record.*

7.4 Deposits do not move profit and loss

For the variation in NAV to be exactly profit, an inflow of value must not, by itself, create profit. The constitution's guarantee (§8, as amended) is that every inflow of value is one of three things, and which one is not a matter of asset class but of the legal regime declared once on the governing agreement unit.

- (1) **An exchange paired with an equal-valued outflow.** Settled-to-market margin is this case: an owned-cash move that settles the day's exposure with no return obligation, because the settlement extinguishes the exposure outright. The future's VM above is the worked instance.
- (2) **A contribution or financing received against an equal obligation created in the same transaction.** Collateral received under title transfer, cash included, is this case: the receiver owns the inflow and owes an equivalent-return obligation that is itself a unit.
- (3) **Value held in custody without being owned.** Collateral received under a security interest without right of use, segregated cash included, is this case: it is recorded on the received-collateral coordinate and never touches owned.

Case 1 adds equal and opposite owned value; case 3 adds no owned value at all. Case 2 is the one that could leak profit, and does not, for a stated reason.

Proposition 7.3 (Deposit-neutrality of title-transfer financing). *The return obligation minted in case 2 is a valued unit, priced at the inflow amount — par plus accrued at the declared rate. This is a theorem, and not an approximation, exactly when the declared financing rate equals the fair rate for the term, so that par-plus-accrued is the obligation's fair value; a declared rate away from the fair rate prices the obligation off the inflow amount, and the day-one difference is a financing basis, not a deposit. Under that assumption, at receipt the owned cash and the obligation are equal and opposite in the owned sum, so net owned value is unchanged, and NAV does not move.*

Because the obligation is priced at par-plus-accrued, its value tracks the cash it returns for the life of the financing, and the receipt neither raises nor lowers NAV; only the subsequent

accrual — a genuine financing cost or benefit — moves it, as it should. This is why PnL needs no regime branch. The regime decides *where* an inflow lands — owned plus an obligation unit, a posted marker, or an owned settlement — but once it has landed, PnL is computed uniformly.

Principle 7.4 (No regime branch in PnL). *PnL is $\sum \text{owned} \cdot P$ over the valued units, with no branch on collateral regime. The regime is a property of the transaction shape and the agreement unit's declared terms, not of the valuation.*

A deposit therefore cannot move profit and loss — not because a formula subtracts it, but because, under every case, it cannot change net owned value. The full mechanics of the three regimes, the obligation and claim units, and the entitlement trap are Chapter 9's; this chapter uses only their valuation consequence.

7.5 Mid-life valuation is read from the record

State-parameterised pricing extends past a binary knock to any unit whose value depends on facts its own history has accrued. The point is that those facts have a home on the record (Chapter 6), so the pricing layer reads them; it does not remember them.

Episode — the variance swap. VS-1 is a one-year OTC variance swap on IDX, 252 scheduled fixings against the recorded official closes, variance strike $K = 400$ points, variance notional $N = \text{USD } 1,000$ per point, no cap. Its declared convention annualises over 252 returns, so realised variance in points is $\sigma_R^2 = 10^4 \cdot \sum_{i=1}^{252} R_i^2$, where R_i is the i -th log return of the close series. Each firing writes the running sum-of-squared-returns statistic into UnitStatus as an integer at scale 10^{-8} , so firing k finds fixings $1, \dots, k-1$ already on the record.

VS-1 fixes on the same recorded IDX closes the future settles against. The future's three settlement days are fixings 124, 125, 126 of the series (closes 1,000, 990, 1,005); the future's final settlement level, 1,005, is therefore the variance swap's mid-life checkpoint close at fixing 126 — one recorded fact, two consumers.

At the mid-life checkpoint (fixing 126, half the fixings elapsed) the accrued statistic is $A_{126} = \sum_{i=1}^{126} R_i^2 = 0.018050$, stored as the integer 1,805,000. Expressed in points and annualised to date this is $10^4 \cdot A_{126} \cdot (252/126) = 361$ points — a realised 19-vol over the first half. The mid-life mark to the variance receiver is that accrued statistic plus the remaining implied variance, both taken from the record: with remaining implied variance $I = 484$ points (22-vol) over the remaining half-year fraction $126/252$,

$$V_{126} = N \left[10^4 \cdot A_{126} + I \cdot \frac{126}{252} - K \right] = 1,000 [180.5 + 242 - 400] = \text{USD } 22,500.$$

The accrued term (180.5 points) is a projection of a UnitStatus integer; the implied term (242 points) is a market observation with provenance (Chapter 8); the strike is a declared term. No figure is stored as a mark. When the year completes, the remaining fixings accrue $10^4 \cdot \sum_{i=127}^{252} R_i^2 = 260.5$ points, the realised variance is $180.5 + 260.5 = 441$ points, and the settlement is $1,000 \times (441 - 400) = \text{USD } 41,000$ to the receiver — the mid-life mark and the endpoint cut from one accruing statistic. *A mid-life value is not a stored quantity; it is the record, priced.*

Episode — the total return swap. TRS-1's reference leg is defined as the NAV of a designated wallet, W-QIS, in a virtual ledger. Valuing that leg requires no new machinery: it is the identical fold, $\sum \text{owned} \cdot P$ over W-QIS, that values any wallet. The stamped NAV enters the true ledger as an observation, never a move (Chapters 8 and 10), and the TRS contract reads it at each reset. *The reference leg of a swap is a NAV projection, folded like any other.*

7.6 Dual valuation: two price vectors over one position set

A wallet is valued for two purposes that demand different prices. Settlement, margining, and reconciliation against a counterparty, a CCP, or a custodian demand the price at which value actually changes hands — the mark-to-market price, P^{mk} , which may differ by venue. Risk aggregation and PnL attribution demand one internally consistent price across the book — the mark-to-mid price, P^{md} . The ledger supplies both as projections over one and the same composition.

Principle 7.5 (One position set, two price vectors). *Mark-to-market and mark-to-mid are two price vectors applied to the single composition produced by the fold. Position divergence is impossible: there is one position record, so only two money figures — NAV^{mk} and NAV^{md} — can differ, never the positions themselves.*

The classical source of internal reconciliation failure is that a settlement system and a risk system each keep their own positions and drift apart. Here they cannot: both read the same coordinates from the same log. The two numbers that differ are honest reflections of two questions — what would settle, and what is the consistent risk mark — and their difference is a basis, computed on demand, never a break to be repaired.

Example — three venue marks, one risk mid. A wallet holds 3,000 units of U , sub-custodied across three venues: 1,500 at venue A, 1,000 at venue B, 500 at venue C. Each venue publishes its own official settlement close, used for that venue’s margin and reconciliation: $A = 100.20$, $B = 100.00$, $C = 99.60$. Risk marks the whole holding at one mid, 100.00.

$$\text{NAV}^{\text{mk}} = 1,500(100.20) + 1,000(100.00) + 500(99.60) = \text{USD } 300,100,$$

$$\text{NAV}^{\text{md}} = 3,000(100.00) = \text{USD } 300,000.$$

The two money figures differ by USD 100, the cross-venue basis. The 3,000 is one number from one fold: each venue’s sub-position reconciles against its own statement under P^{mk} , and the book aggregates for risk under P^{md} , but the position cannot diverge because there is only one of it.

7.7 The pricing-stack requirements

The valuation discipline above forces a small set of requirements on the pricing stack and the pricing data layer. They are derived here as consequences of NAV being a projection; Chapter 16 restates them as the minima any implementation must meet.

- P1. **The ledger records pricing outputs and never runs a model.** Model outputs re-enter only as observations carrying sufficient provenance; reproducing a valuation from recorded prices is unconditional, while reproducing a model’s number requires the model itself, whose retention is a governance matter outside scope.
- P2. **Valuation is state-parameterised, and the state comes from the ledger.** $P(u, \sigma(u))$ reads $\sigma(u)$ from UnitStatus and PositionState, never from a pricing-owned store — the one-touch and the variance swap both turn on this.
- P3. **Valuations are reproducible from recorded inputs.** The composition is a fold of the log and the prices are identified on the record, so any party recomputes any NAV to the minor unit.
- P4. **Prices and quantities are never mixed across an event.** Each is taken in the same state of the instrument — consistency of reference.

- P5. **Two price vectors are both projections over one composition.** Mark-to-market and mark-to-mid share the single position record; positions never diverge.
- P6. **Estimates are kept apart from entitlements,** and originals are preserved while adjusted and derived values are computed at read (Chapter 8).

Each is a property that can be checked mechanically over generated products and histories, and each is exercised by a thread episode in this chapter. Valuation adds no store to the ledger: it is the log, folded and priced.

8 Ingestion, Corporate Actions, and the Market Data Operator

This chapter discharges §9 of the constitution.

§9 governs the one place the closed system is not closed: the boundary where external data crosses in. Everything inside the ledger conserves quantity by construction (Chapter 3); value, by contrast, is supplied from outside — prices, dividend figures, index levels, fixings, corporate-action notices, stamped valuations. The conservation properties hold only because the system is closed, so the boundary is where they are won or lost. A datum admitted without provenance, a corporate action that silently re-reads old data in a new frame, a fallback taken without a record: each corrupts the closed-system property without violating any single internal invariant. This chapter holds the boundary watertight, and it does so with two devices — the recorded observation and the market data operator — and no third.

8.1 The boundary, and corporate actions on it

Two things cross the boundary, and they are recorded the same way. *Market and reference data* — a price, a dividend figure, an index level, a fixing, a stamped NAV — crosses as a recorded observation. A *corporate action* — a split, a dividend, a spin-off, a merger — crosses first as a recorded notice and is then, on its effective date, a transaction like any other: a bundle of moves, unit-state changes, and a new version of the instrument’s terms, applied atomically through the one Transaction Executor (Chapter 4). A corporate action is not a privileged operation on a store; it is admitted at the same door as a trade, and it writes ProductTerms, UnitStatus, and balances through the same four-kind transaction (Chapter 3).

A corporate action does one thing no trade does: it changes the meaning of every datum recorded before it. After ACME’s two-for-one split, the recorded close 100.00 still names the same economic fact, but a post-split reader who multiplies it by the post-split count 20,000 books a phantom. To carry a pre-event datum into the post-event frame a transformation must be applied, and the framework gives that transformation a name: the **market data operator**. Three principles govern it, taken from §9 and used throughout this chapter:

- (i) the operator is *intrinsic to the pairing* of a kind of data with a kind of event, declared once when the kind of data is introduced and the corporate action is recorded, never improvised at the moment of reading;
- (ii) the *ledger is the authority* and the pricing data layer merely applies: the operator is declared data on the record, and the pricing data layer computes with it, never around it;
- (iii) *originals are never overwritten*. The observed datum stays on the record exactly as observed; the adjusted value is computed at each read; derived quantities are recomputed from operator-adjusted inputs — until fresh post-event data are recorded, after which the operator has nothing left to adjust.

8.2 The observation door

No datum enters the ledger as a bare read. Every datum crosses as a recorded **observation** carrying its provenance: the source that published it, the time it was observed, and the basis

frame it was observed in. This is the door that Chapters 7 and 10 refer to when a mark or a stamped level “enters through the observation door.” A read against a live feed leaves no record and is therefore unreproducible; a recorded observation is a fact of the log, so every downstream number is a function of the record alone (§12).

```
data Provenance = Provenance { source :: SourceId, observed :: Time, basis ::
    BasisRef }
data Observation = Observation { datum :: Datum, prov :: Provenance }
-- every datum crosses as a recorded Observation; there is no bare read.
-- the door admits or refuses; a refusal is a returned value, never a partial
  write:
data Ingest      = Admitted Observation | Refused IngestError
```

The provenance frame is what makes “prices and quantities are never mixed across an event” (§8) enforceable rather than hoped for: the frame a datum was observed in is on the record beside the datum, so a read that would combine it with a quantity in a different frame is detectable, not silent. That detection is the invariance witness of Section 8.6.

Episode — the variance swap (T5): the fixing source is the record. *Trigger.* VS-1’s scheduled fixing dates arrive; each is a watch on the recorded IDX official close (Chapter 3). *The door.* The fixing is not a second feed; it is a read of the same recorded official-close observation that FUT-IDX settles against. The three settlement days of the future — IDX closes 1,000.00, 990.00, 1,005.00 — are fixings 124, 125, and 126 of the swap’s 252-fixing series. *Design point.* One recorded observation source feeds two consumers; because the source is on the record, the fixing series is deterministic and replayable at the data boundary, and firing 127 recomputes the same statistic any party would (Chapter 7).

Episode — the total return swap (T6): the bridge is an observation. *Trigger.* TRS-1’s first quarterly reset arrives; virtual ledger V-1’s replayable level is stamped at USD 10,300,000. *The door.* That level crosses as an observation with provenance — source the fold of V-1’s own log, observed at the reset date, in the reference-currency frame — and is recorded. No move crosses: a move from a wallet of V-1 to a wallet of the true ledger has no paired leg that conserves in either ledger, so neither Transaction Executor can admit it (Chapter 10). *The door checks.* Provenance present; the crossing is a recorded observation, not a move. *Design point.* The bridge between the virtual ledger and the true ledger is an observation, never a move; the reset and financing moves it triggers are the true ledger’s own (Chapter 10).

8.3 The datum-kind registry

Market data is not one number per name. A dividend forecast is a composite: cash entries that carry a price dimension and scale under a split, and proportional entries that carry no dimension and stand. An operator that treated the datum as a single scalar would either scale the proportional part, which is wrong, or leave the cash part, which is also wrong. The operator must therefore know the *dimension of each component*, and it learns it from the **datum-kind registry**.

A kind is registered once, before any datum of that kind crosses, with a component-wise declaration: each component’s dimension — price-in-basis, proportional, or quantity — and the quotation convention it renders. Registration is immutable: a correction is a new kind plus a retirement of the old, never an edit, so no datum ever changes dimension under a reader’s feet. A datum whose kind is not registered is refused at the door.

```
data Dim      = Price | Proportional | Quantity          -- component dimensions
data Datum    = Datum { kind :: KindId, comps :: [(Dim, Integer)] } -- exact
  minor units
data IngestError = UnknownUnit | UndeclaredConvention | UnregisteredKind
```

-- *an operator acts on Price components; Proportional stands; Quantity is walled off*
 -- *to the move plane (Chapter 3) and is never carried by an operator.*

The wall between **Quantity** and the operators is the same wall as the move plane. A share count is not adjusted by an operator: the two-for-one split moves 10,000 ACME to 20,000 by a paired-leg move from the issuer’s virtual wallet (Chapter 3), because quantity is conserved and only a move may change it. Operators act on value-frame components only. This division is what makes the split’s arithmetic come out invariant, and it is stated as the witness next.

The registry is a named trust assumption, and it is the only one the operators rest on.

TA-KIND. A kind declaration faithfully renders the publisher’s quotation convention — which components are cash and which are proportional, in which basis. *Owner:* the party that governs kind registration (data governance). *Violation:* an operator scales a component it should not, or spares one it should scale, misvaluing across every subsequent corporate action. *Detection:* the invariance witness at the first crossing of a corporate action (Section 8.6), divergence against an independent source (regime W3 below), and §13 recomputation. *Residual:* a misdeclaration that never meets a corporate action is invisible from inside the boundary — a reconciliation matter at the perimeter, the exact sibling of the collateral regime bit of Chapter 9.

8.4 The market data operator and adjustment-schedule totality

An operator is a function of the registered kind and the event kind, declared once and recorded. It carries a value-frame datum from the pre-event basis to the post-event basis; it is the identity on proportional components and undefined on quantity components, which never reach it.

-- *one operator per (event kind, datum kind), declared once, never improvised at read:*
 operator :: EventKind -> KindId -> (BasisFrame -> BasisFrame)

The first mechanism is that this declaration is *total*.

Principle 8.1 (Adjustment-schedule totality). *For every registered datum kind and every event kind the boundary admits, an operator is declared — the identity operator where a kind is genuinely unaffected, but declared, not assumed. There is no (datum kind, event kind) pair for which a read finds the adjustment undefined. A corporate action whose pairing has no declared operator does not cross: it is refused at the door, never silently treated as identity.*

Totality is what turns “never improvised at read” into a checkable property. An improvised adjustment is precisely a pairing with no declared operator; forbidding the improvisation means requiring the declaration in advance, for the whole registered domain. The refusal of an undeclared pairing is failure regime W4 (Section 8.8), and it fails closed, as §2’s Correctness commitment demands: the system stops rather than guesses a frame.

Operators compose. Two corporate actions in sequence adjust a datum by the composition of their operators, and the composition is taken *in log order*, because the operators do not commute: a split then a special dividend is not the same adjustment as the dividend then the split. The log fixes the order, so the composed adjustment is well defined and replayable.

A second telling, in categorical language; nothing later depends on it.

Fix a datum kind. Its bases are the objects of a small category; each recorded event contributes the arrow its operator names, from the pre-event basis to the post-event basis; identity events contribute identity arrows. A chain of corporate actions is then a path, and the adjustment it induces is the composite arrow — associative because composition of functions is, order-sensitive because the arrows do not commute. The map sending each event

sequence to its composed operator is a functor from event paths to basis maps; adjustment totality is the statement that this functor is defined on every arrow of the source. Nothing below depends on this telling.

8.5 The operator menu: the split

The split shows the whole menu at once. On 2026-06-01 ACME’s two-for-one split is effective; the recorded split notice had declared the effective-date watch, the Event Monitor emits the event, and one transaction carries the move and the new terms version (Chapters 3 and 6). What each recorded datum kind reads as, in the new frame, is the operator’s work:

Datum kind	Operator for the 2-for-1 split	Read after the event
Official close (spot)	scale by $\frac{1}{2}$	100.00 $\rightarrow$ 50.00
Cash-per-share dividend figure	scale by $\frac{1}{2}$	1.50 $\rightarrow$ 0.75
Proportional (percentage) figure	identity — stands	1.00% $\rightarrow$ 1.00%
Index divisor (a constituent splits)	adjusted to hold the level	level continuous

The share count is absent from the menu on purpose: 10,000 $\rightarrow$ 20,000 is a move, not an operator (Section 8.3). The spot and the cash-per-share figure are price-dimensioned and halve; the percentage figure is proportional and stands; the index divisor is adjusted so the published level does not jump when a constituent splits. Every original stays on the record; each “read after” is computed at the read, until a fresh post-split close prints and the operator retires for that datum.

Episode — the dividend figure under the split (T3×T4). *Trigger.* On 2026-05-04 ACME’s cash dividend of USD 1.50 per share is announced; the announcement is captured at the boundary as a recorded observation, and its declared per-share figure 1.50 enters the record (the three firings that pay it are Chapter 5’s). *The operator.* The split is effective 2026-06-01, after the payment date but within the window in which the declared forward figure is still read; a read of the 1.50 figure in the post-split frame applies the split’s cash-per-share operator and returns 0.75. *The door checks.* The datum kind is registered as a cash-per-share (price) component, so the operator scales it; a proportional figure beside it would have stood. *Design point.* *One operator, one invariant, visible: the same halving that takes the spot from 100.00 to 50.00 takes the recorded per-share dividend figure from 1.50 to 0.75, so the dividend history reads in current-share terms — the scaling of a real figure is shown, not asserted.*

8.6 State-aware reads: consistency of reference and cum/ex

Every operator read is state-aware: the frame a datum reads in is determined by the corporate-action state of its underlying at the read, and that state is on the record. Two readings of one recorded datum in two frames are two different values, and combining a quantity with a price drawn from a different frame produces a phantom. The check that refuses phantoms is the **invariance witness**: the operator on the value frame and the move on the quantity are bound so their product — total value — is invariant across the event.

For the split the witness is arithmetic. Before: $10,000 \times 100.00 = \text{USD } 1,000,000$. After, read in the post-split frame: $20,000 \times 50.00 = \text{USD } 1,000,000$. The phantom the witness refuses is $20,000 \times 100.00 = \text{USD } 2,000,000$, the new count against the stale frame. This is the consistency-of-reference invariant; it is stated formally and catalogued in Chapter 14, and used here.

The same witness governs the cum/ex boundary of a distribution, where no quantity changes at all. Before the ex date a price is *cum* — it includes the coming distribution; on and after the

ex date it is *ex* — it excludes it. The cum price and the ex price are one datum kind read in two states separated by the ex-date event, whose declared operator subtracts the distribution's value. A cum price consumed as though it were ex, or an ex price consumed as though it were cum, is a phantom of exactly the split's kind — the right number in the wrong frame — and the same invariance witness refuses it, because the state that fixes the frame is the recorded ex-date event, not the wall clock. A read that ignores the ex-date state prices the instrument on the wrong side of its own corporate action, and the witness catches it before the number reaches a valuation.

8.7 Recompose and the aggregate weld

The split adjusts one unit's data in place. Two further mechanisms handle corporate actions that change which units exist: one splits a unit into several, the other merges several into one. Both preserve total value, and both are checked by the same witness.

Recompose (one unit becomes many): a spin-off. When an issuer distributes a new entity to its holders, one position becomes a basket, and the pre-event data of the parent must be carried onto the resulting units. *Recompose* is the operator that does so, per a declared allocation basis recorded with the event. Illustratively: HELIOS trades at 120.00 and spins off LUMEN, one LUMEN share per HELIOS share, with a declared allocation of 75% of pre-event value to the HELIOS stub and 25% to LUMEN. A holder of 1,000 HELIOS holds, after the event, 1,000 HELIOS and 1,000 LUMEN — the new LUMEN mass arriving by a paired-leg move from the issuer, not by an operator. Recompose carries the recorded pre-event close onto the two frames: HELIOS stub 90.00, LUMEN 30.00. The witness holds: $1,000 \times 120.00 = 1,000 \times 90.00 + 1,000 \times 30.00 = \text{USD } 120,000$. The original 120.00 is preserved; the stub and spun-off frames are computed at read until fresh post-event closes print for each.

The aggregate weld (many units become one): an elective merger. When one unit is absorbed into another, the resulting position welds into the acquirer's aggregate, and the two data histories must weld into one continuous series so that valuation and profit and loss run unbroken across the event. The *aggregate weld* is the operator that carries the pre-event data of the absorbed unit onto the acquirer's frame, per the elected terms recorded at the boundary. An elective merger is captured as an observation — the election result is a datum with provenance, like any other. Illustratively: TARGET is acquired by ACQUIRER, each TARGET share electing either USD 50.00 cash or 0.4 ACQUIRER shares; ACQUIRER trades at 125.00. A holder of 1,000 TARGET who elects stock welds to 400 ACQUIRER, and the pre-merger TARGET series is welded onto the ACQUIRER frame at the elected ratio 0.4; the witness holds, $1,000 \times 50.00 = 400 \times 125.00 = \text{USD } 50,000$. A holder who elects cash has the position retired at settlement and no weld occurs — the elected terms, recorded data, decide which mechanism runs.

Two further composite distributions — a dividend forecast and a special dividend — are governed by the same registry and the same operators: the forecast is a composite of cash and proportional components that the split scales component-wise, and the special dividend carries a cum/ex operator like the ordinary one. Their fully worked arithmetic is deferred to the Worked-Examples companion (register lines E71 and E72); the mechanism here is complete without them.

8.8 Failure regimes at the boundary

The boundary has four failure modes, and each has a determined, replayable disposition — a function of log state alone, so replay reproduces it bit-identically. None improvises; each fails closed or records a flag, never a silent pick.

	Boundary condition	Determined, replayable disposition
W1	missing datum	the operator applies to the last recorded original, carrying a staleness flag; with no original, the price is undefined, not zero, and valuation over the unit defers (Chapter 7)
W2	late datum	recorded at its observation-time coordinate; values derived over [observed, arrival) are recomputed — “as known at t ” and “with corrections through t' ” are both queryable
W3	contradictory sources	sources diverging beyond the declared threshold yield a flagged “aggregation failed” observation, never a silent pick; sources and threshold are recorded
W4	unmappable event	a pairing with no declared operator, or a datum of an unregistered kind, is refused at the door (Principle 8.1)

W1 is the working state of §9’s “until fresh post-event data are recorded”: between the event and the first fresh datum, every read is the original under its operator, flagged stale, and the flag is a fact of the record, not a property of a cache. W2 makes the boundary bitemporal: because an observation carries its own observation time, a datum arriving late does not rewrite history — it is placed at its own time, and the derived folds over the affected interval are recomputed, both the original and the corrected view surviving (§2, §12); an amended or withdrawn corporate-action notice is this same case — a superseding datum recorded at its own observation time, never an edit of the original notice, so the pre-amendment frame and the corrected one both survive. W3 is the multi-source case: a single source is a single point of failure, so where a datum materially affects valuation it is drawn from more than one attested source, aggregated by a declared rule, and a divergence beyond the declared threshold produces a flagged output rather than a silently chosen one. The divergence threshold is a governance parameter, recorded and calibrated, not a silent constant: under correlated stress — every venue diverging at once — it defers valuation book-wide rather than choose a number, an intended fail-closed whose calibration is a governance matter, not a hidden behaviour. W4 is adjustment-schedule totality enforced: the undeclared pairing cannot cross.

8.9 The boundary-convention slot

One class of boundary fact is neither a datum nor an operator: a *convention* that some downstream projection needs and that must be stated once, as data, so that two readers of one log compute one answer. §9 places such conventions on the record beside the operators; this section declares the **slot** they occupy, and Chapter 12 fills and consumes one.

A convention in the slot is a total function of log state. The convention Chapter 12 consumes — the attribution of commingled received value across a reinvestment pool — has three components, and the slot’s shape is exactly these three:

```
data Convention = Convention
  { pool      :: Predicate (Wallet, Move)  -- the pool over which value is
    attributed
    , basis    :: PeriodBasis               -- when the pro-rata ratio is struck
    , rounding :: RoundingRule }           -- remainders booked explicitly (
    Chapter 3)
attribute :: Convention -> LogPrefix -> Partition Amount -- consumed in Chapter
12
```

The slot declaration is itself a recorded event, so it versions with the log prefix: a report computed as of a past date uses the convention then in force, and a changed convention is a new recorded version, never an edit. Totality is the requirement that makes the slot safe to consume: for every log prefix the convention assigns every attributable quantity, the attributed

parts and the explicit remainder partition the whole, and the rounding rule leaves no quantity silently dropped (§4’s remainder discipline). A convention missing any component is refused, because two readers would then compute different figures from one log — internal reconciliation failure reintroduced through a report. Chapter 12 states the totality property and proves the figures it builds are projections of coordinates, units, and this declared convention alone; this chapter’s duty is discharged by declaring the slot and its totality obligation.

8.10 What an auditor checks

A candidate implementation satisfies this chapter when six things hold, each mechanically checkable over generated data and histories: every datum on the record carries provenance, and no valuation reads a datum that does not (Section 8.2); every datum’s kind is registered with component dimensions before it crosses, and an unregistered kind is refused (Section 8.3); an operator is declared for every (registered kind, event kind) pair, and an undeclared pairing is refused, not defaulted to identity (Principle 8.1); every original is preserved and every adjusted value is recomputed at read from the original under the declared operators (Sections 8.5–8.7); the invariance witness holds across every generated corporate action, and a stale-frame or wrong-side-of-ex read is refused (Section 8.6); and each of W1–W4 produces its determined disposition on replay (Section 8.8). These restate as minima in Chapter 16. The boundary is then closed: what crosses it is recorded, framed, and replayable, and the conservation the rest of the specification proves rests on solid ground.

9 Collateral, Margin, and Lending

This chapter discharges the amended clauses of constitution §4 and §8, and §6 for the agreement units it introduces.

9.1 One representation, keyed by declared regime

One representation serves every collateral, margin, and lending arrangement the ledger records. Every (wallet, unit) balance is the signed coordinate vector (owned, lent, posted) of Chapter 3; the outright holding is the degenerate case with all mass on owned; and any unit — cash, a bond, a structured note, a listed option — may sit on the non-owned coordinates. Which coordinate an inflow writes is decided by the legal regime declared once on the governing agreement unit, never by the asset class of what moves. Title transfer writes owned, plus a claim and an obligation that are themselves units, and touches no marker coordinate; a security interest writes marker mass on the posted plane and never touches owned; settled-to-market margin writes owned with no obligation at all. The rest of this chapter is that key, applied.

The agreement is a unit — constitution §6: every right and obligation the legal contract creates is a unit. Its declared terms carry the regime (settlement, financing, or custody); the eligibility schedule and the per-line valuation percentages; the thresholds, deadlines, and close-out mechanics of its margining; the trapping conditions its distributions carry; and its watches — the valuation watch, the income watches — declared at registration like any other unit’s. The agreement unit itself is non-valued: the claims and obligations it creates carry the value, and pricing the governing shell beside them would count the same economic reality twice (§4).

A claim or an obligation is a valued unit held positive by the obligee and negative by the obligor: one unit, one fact, both sides, and conservation makes disagreement between them unrepresentable. An obligation carries, as declared terms, a deadline, a discharge predicate, and a fallback (§14.1); by its deadline it is discharged, compensated, or declared defaulted — a duty that fails to fire is a visible, overdue item, never silence — and, like every unit, it retires only from the zero vector.

Eligibility and haircuts are declared data, checked at the door — the Transaction Executor’s admission check — and never typed. Collateral is a role a balance plays under an agreement, not a kind of thing: the type system knows planes and agreement references; what may be posted, and at what valuation percentage, is a schedule in the agreement’s terms. Admitting a new eligible asset class is a terms version, not a code change; an ineligible posting is refused at the door. The declared eligibility predicate may itself read recorded ledger state: a concentration limit makes eligibility portfolio-dependent, admitting a posting only while the resulting holding stays within the declared bound — read at the door exactly as the coverage check reads state, and never typed. What the type system does enforce is stronger than any schedule: a move writes exactly one coordinate of one unit, positive quantity, the same coordinate debited at the source and credited at the destination, and a move whose coordinate is not owned carries a mandatory reference to its governing agreement. Orphan collateral mass is not refused; it is unwritable.

```

newtype AgreementRef = AgreementRef UnitId -- the governing agreement, itself a
    unit
data Plane = Lent | Posted                  -- the marker planes

data Placement                             -- where a move writes
  = OnOwned                               -- ownership moves; no agreement
    needed
  | OnMarker Plane AgreementRef            -- marker mass: unwritable without its
    agreement

data Regime = Settlement | Financing | Custody -- declared once, on the agreement
    unit

```

Everything computable from the coordinates — available quantity, encumbered quantity, mass in possession, mass re-used — is a projection, computed when needed and never stored (§4). Chapter 12 builds the reporting projections on exactly this surface.

9.2 The two guards

Two guards govern the marker planes, and they are never collapsed into one word. Both are *used* throughout this chapter; their formal statements and proof obligations are catalogued in Chapter 14.

Coverage is an invariant. No wallet delivers what it neither owns nor holds. Checked at the door on every admission, per wallet and unit, net over agreements:

$$\sum_G \text{posted}_G(w, u) \leq \max(\text{owned}(w, u), 0),$$

and the same bound with lent in place of posted. The sum runs over the signed balances, so mass received offsets mass re-posted: a wallet that received 100 under one agreement and re-posted 60 under another nets to -40 , within the bound — re-use of received collateral is thereby bounded by possession. The max makes the bound bind only the posting direction: a short wallet with nothing posted is admissible — the issuer wallet that conservation itself creates is one — while a poster with owned $-5,000$ attempting to post 5,000 is refused at the door. Coverage is never lawfully false, because a transaction is atomic and possession is instantaneous within it; that is what makes it an invariant.

Sufficiency is an obligation. Collateral value at or above exposure is lawfully false intraday: a market move, a knock, a call in flight all leave it false with no party in breach. It is therefore an obligation, never an invariant: a deadline, a discharge predicate (the delivery amount met), and close-out as the declared compensation. A book below its sufficiency line is not a broken state; it is an open, deadline-bearing item on the record, and the deadline covers it.

9.3 Title transfer: owned re-books; the claim is a unit

Owned must be the coordinate the move machinery can act on. The taker of title-transfer collateral may lawfully sell it outright; a design that keeps the asset on the poster's owned makes that lawful sale a special constructor and makes the manufactured payment — the income the taker owes back to the poster — unrepresentable. So the poster's owned re-books to the taker, and what the poster keeps is what the poster legally has: in the same transaction the poster gains a **claim-for-equivalent** unit, and the taker's negative balance in that unit is the redelivery (or repurchase) obligation. Three rules complete the shape; each is a correctness requirement.

Pricing. The claim is priced through the ordinary pricing layer, parameterised — as valuation under §8 already is — by state: here the taker's. While the taker performs, the claim tracks the referenced asset (the accrued redelivery value) and bears the taker's credit; on the taker's default it decouples from the referenced asset and becomes a recovery claim on the taker's estate. It is not “priced identically to the asset”: that rule would mark the poster long at par on the one day the mark exists for.

Identity. Claims are per-agreement units — per netting set. Close-out nets within a master agreement and never across masters, and because owned moves carry no agreement reference, the claim's netting-set identity cannot live on a move: it lives in the unit's own declared terms, which name both the referenced asset and the governing agreement. The unit registry grows by one claim per financing transaction. Accepted: fungible claims would break netting-set attribution at close-out, which is the one computation the claims exist to get right — and the counterparty-state pricing above is then natural, because the claim is a per-counterparty object.

Timing. Owned re-books at instruction — trade-date booking — and the instruction-to-settlement gap is carried explicitly, never elided. Where a record date falls inside the gap, the paying agent pays the record holder — still the deliverer — while the ledger's owned shows the buyer. The reconciling leg is a **market-claim obligation unit** created over the gap: first-class, recorded, deadline-bearing — an instrument, not a break. Settlement state is thereby load-bearing for entitlement routing: a correctness requirement of this design, not a reporting axis. Chapter 11 carries the settlement-side machinery; the mechanism is stated here because the re-booking rule creates the gap it must close. Re-booking at instruction also lets a wallet deliver or post a bought-not-yet-settled position; should that purchase later fail, the shortfall is not an impossibility but a fails cascade the model carries — order-forced at the door (an owned reduction that would drop owned below the mass already posted is refused), then the unmet return as a recorded obligation, then, on default, the supervised write-off path of this chapter's close — never a silent breach. That fails obligation is, like every obligation, a valued unit, minted — once the reversal is refused — by the settlement-fail event's contract, symmetric to the market-claim unit the record-date firing mints (Chapter 11; the projection writes nothing). With its paired settlement — delivery-versus-payment — cash leg it offsets the bought-not-yet-settled position at fair value, so net owned value — and NAV — is preserved across a failed purchase, on the same declared-rate-equals-fair-rate basis as the §8 case-2 return obligation below.

Cash under title transfer is the same shape, by §4's own admission test: a quantity earns a coordinate only when a distinct real-world action can change that coordinate independently of ownership — and taking cash under title transfer *is* the change of ownership, so it earns no marker coordinate. The receiver owns the cash, reinvests it, bears the result, and owes an equivalent-return debt: §8's second case, the obligation itself a unit. A custody marker over commingled cash would assert a per-dollar fact no bank statement can confirm — the nostro observes only the sum — and its intraday divergence from the statement is precisely the internal reconciliation failure the ledger exists to abolish. The return obligation is valued at the inflow amount — par plus accrued at the declared rate — so net owned value cannot move at receipt: §8's deposit-neutrality sentence holds by construction, not by formula. Balance-

sheet presentation, encumbrance, and re-use figures are projections over claims, regimes, and coordinates — Chapter 12; no second store.

9.4 Pledge: marker mass under the coverage bound

A pledge changes possession independently of ownership, so it writes the marker plane and only the marker plane: one move on the posted plane — poster $+Q$, taker $-Q$, the taker’s negative balance projecting as received as collateral — admissible under the coverage invariant, with owned untouched. That is the legal fact: the pledgor still owns. Because posting never writes owned, $V = \sum \text{owned} \cdot P$ is unchanged by any post-and-return round trip, and §4’s “only the owned coordinate carries economic value” becomes a machine-checkable property: insert a post-and-return pair within any watch-free interval of the referenced agreement and every later valuation is identical. The side condition is real — an insertion spanning the agreement’s valuation watch lawfully changes later firings and cash — and the property, with it, is catalogued in Chapter 14.

Re-use of pledge-form collateral is a posting of mass held on the received ray, within the same coverage bound. Every re-pledge move carries a source-agreement reference beside its governing reference — the source is an observable fact of the custody instruction, so nothing fictitious is stored — and re-use therefore projects as per-(unit, agreement) actuals in Chapter 12.

9.5 Cash margin: four cases, one machine

Cash margin is four cases, keyed by the declared terms of the agreement unit, and the enumeration is total: a margin flow whose agreement declares no regime has no admissible booking and is refused at the door — the system stops rather than improvises (§2).

The agreement declares	The inflow is	The transaction writes
Settled-to-market (STM) variation margin	settlement (§8 case 1)	owned; no obligation — the day’s exposure is extinguished
Collateralised-to-market (CTM) or title-transfer credit support	financing (§8 case 2)	owned, plus a return-obligation unit accruing the declared rate
Security interest without right of use (segregated)	custody (§8 case 3)	posted-plane marker mass; owned untouched
Security interest with right of use	financing upon exercise of the right	owned, plus the return-obligation unit, upon exercise

The custody case raises no fungibility objection, because the segregated account reconciles one-for-one against the marker. For cash under a right of use, receipt into the receiver’s own accounts is the exercise: commingled right-of-use cash books as financing, never as custody — the fourth row collapses into the second at the moment commingling makes the custody claim unverifiable.

Episode (future): the settled-to-market margin day. FUT-IDX is a cash-settled listed future on index IDX, multiplier 10, settled-to-market by the declared terms of the central counterparty (CCP) agreement unit; wallet W-ALPHA is long one contract, and day 1 settled at 1,000.00. *Trigger.* The day-2 official settlement print, 990.00, is recorded; the Event Monitor emits the settlement event. *The contract.* The future’s contract fires, reading the last applied level, 1,000.00, from PositionState (Chapter 6). *The transactions.* One transaction:

- (1) move owned(USD) $(1,000.00 - 990.00) \times 10 = \text{USD } 100$, W-ALPHA $\rightarrow$ W-CCP;
- (2) PositionState update: margin applied, settlement level 990.00.

No marker coordinate, no obligation unit: the payment settles the day’s exposure, and on unwind nothing is returned, which is correct. W-ALPHA’s net asset value falls by exactly 100, the day’s loss. *The door*. Conservation per (unit, coordinate) on the paired-leg move; an owned move, so no agreement reference is required; idempotence under the cause-derived identifier — a duplicate settlement event proposes nothing new. *The contrast (CTM)*. The episode is settled-to-market: the day’s exposure is extinguished and nothing is owed back. Had the CCP agreement declared CTM, the identical cash move would be paired with a return-obligation unit — the returnable margin liability — valued at 100 and accruing the declared rate. That single unit is the whole difference: STM and CTM share one machine and diverge by one declared term and one returnable-liability obligation unit, not by a different case. *The regime is a declared term of the agreement unit; the machine is the same*.

The regime is one declared bit carrying the largest balance-sheet consequence in this chapter, and its misdeclaration is a class of error the recomputation check of §13 cannot catch: recomputation reproduces the declared bit and faithfully reproduces the wrong answer. A margin agreement misdeclared as STM omits the returnable-margin liability and its interest accrual; the converse fabricates them. Detection is boundary reconciliation against the counterparty’s or the clearing house’s margin statement — nothing internal detects it, and the reconciliation is therefore a named duty of the boundary, not a redundancy. The repair is defined now, not improvised at the first discrepancy: a ProductTerms amendment — a new terms version declaring the true regime — *plus* compensating transactions rebooking the obligation units and accruals of the affected interval. Both, not one: the amendment without the rebooking leaves the interval wrong; the rebooking without the amendment repairs history and re-breaks tomorrow.

The regime bit is the worked instance of a rule that governs every declared term of the agreement. The eligibility schedule, the valuation percentages, the thresholds and deadlines, and the trapping predicates are all **boundary data**, transcribed once from the executed agreement, which is their provenance. A transcription error in any of them is not an inconsistency the ledger can catch by recomputation — recomputation reproduces the transcribed term and faithfully reproduces the wrong answer — but a perimeter-reconciliation matter, the same class as the §13-invisible regime bit above and the market-data trust assumption of Chapter 8. It is detected by boundary reconciliation against the counterparty’s own record of the agreement, and repaired by the same two-step path: a terms amendment declaring the corrected term *plus* compensating transactions rebooking the affected interval, never a silent edit. The regime bit carries the largest consequence and earns the worked treatment; the duty is general over every declared term.

9.6 Line valuation

Collateral value under an agreement is the sum, over the lines the agreement holds, of market value times the declared valuation percentage of that line. The derivation is §6 faithfulness: the ledger values what the agreement declares, and the agreements declare lines — an eligibility schedule with a valuation percentage per line — not floored packages. An agreement whose declared terms define package-level valuation prices the package through the ordinary pricing layer, admissible as declared data like any other term; and a portfolio floor, where computed, is a projection — never a stored valuation, and never a haircut the agreement does not grant.

9.7 Entitlements: determination and payment

Entitlement *determination* always reads the owned plane. The instrument’s contract is possession-blind and regime-blind: who is owed a dividend, a coupon, a payout is a question about ownership, and ownership lives on exactly one coordinate. Entitlement *payment* routes through a **conditional obligation unit** created by the determination, whose discharge

predicate carries the governing agreement’s declared trapping terms — the sufficiency and default conditions under which the agreement withholds a distribution. Where no agreement encumbers the owner’s mass, the predicate is vacuously true and the obligation discharges at once: in the ordinary state the *condition* vanishes, never the *leg*. An unconditional issuer-to-owner pass-through is forbidden: the agreements trap distributions exactly when a delivery amount or a default stands, and §6 binds the ledger to the terms as declared. The predicate traps or releases the gross determined entitlement; where an agreement instead declares an *amount* transformation — a manufactured dividend net of withholding, or a contractually reduced manufactured rate — that amount is a declared term of the agreement, read like any other declared datum, not a new mechanism.

Episode (dividend): the unencumbered case. W-ALPHA holds 10,000 ACME; the declared dividend is 1.50 per share; the record-date firing has already determined and recorded the entitlement, USD 15,000, in PositionState (Chapters 5 and 6). Nothing encumbers W-ALPHA’s ACME. *Trigger.* The payment-date watch fires; the Event Monitor emits the event. *The contract.* The stock’s contract, on its third firing, pays against the recorded entitlements. *The transactions.* One transaction:

- (1) the conditional payment obligation is created against the recorded entitlement — ACME’s issuer wallet owes W-ALPHA USD 15,000; its predicate carries the trapping terms of every agreement encumbering the holder’s mass, and there are none, so it holds vacuously;
- (2) move owned(USD) USD 15,000, ACME issuer wallet → W-ALPHA; the obligation is discharged and retires at the zero vector.

The door. Conservation on both paired-leg moves; idempotence — a duplicate payment-date event finds the entitlement discharged and proposes nothing; the obligation retires only from the zero vector. *In the ordinary state the condition vanishes, never the leg.*

9.8 The knock while pledged

The centrepiece assembles every rule above on one instrument. W-ALPHA owns one one-touch option OT-1, written by W-BANK, paying USD 1,000,000 on touch, marked USD 400,000 before the knock. It is pledged under security-interest agreement G to counterparty wallet B at a declared valuation percentage of 50%: W-ALPHA posted +1, B posted −1; collateral value to B USD 200,000 against an exposure of USD 150,000. The declared barrier is 80.00 on stock OMEGA; OMEGA prints an official close of 79.00.

1. The low print is recorded; the Event Monitor emits the touch. The knock is now on the record; no ledger state has changed — the Monitor emits events and cannot write state.
2. OT-1’s contract fires on the event, and the Transaction Executor admits its proposal: UnitStatus → *triggered*; the entitlement is *determined* from the owned plane — W-ALPHA; and the conditional payment obligation is created — W-BANK owes W-ALPHA USD 1,000,000, discharge predicate carrying G’s declared trapping terms. Mass on the posted plane is unchanged. A duplicate emission finds the unit triggered and proposes nothing.
3. The pricing layer, state-parameterised, prices triggered OT-1 at 0. Value has collapsed; mass is intact — coordinates carry mass, prices carry value.
4. G’s contract, at its declared valuation watch, values B’s received projection: 0. The delivery amount is USD 150,000; it fires a margin-call obligation with a deadline.
5. The payment obligation discharges under its predicate — release only to the extent no delivery amount stands. One transaction, two moves: owned(USD) USD 1,000,000 W-BANK → W-ALPHA, and posted(USD) USD 150,000 W-ALPHA → B under G, delivered as credit support and discharging the call; USD 850,000 stays free with W-ALPHA. Coverage holds within the transaction: owned cash 1,000,000 ≥ posted 150,000. Had G declared no trapping terms, the predicate would be vacuous and the full payout would

release; an agreement declaring proceeds attachment would have G's contract post the cash instead — declared terms decide.

6. The pledge closes: the marker return clears the posted plane of OT-1 (B/W-ALPHA $\pm 1 \rightarrow 0$); then the extinguishment move owned(OT-1) W-ALPHA $\rightarrow$ W-BANK takes every wallet's vector for OT-1 to zero — **the ordering is forced by the coverage invariant**, which refuses extinguishing owned while posted mass remains; and OT-1 retires at the zero vector.

The door, across the six steps: idempotence at step 2; conservation per (unit, coordinate) on every move; coverage on step 5's transaction and again as the ordering constraint of step 6; retirement only from the zero vector. The residuals are named, not hidden. Between steps 2 and 5 the book is lawfully under-collateralised: sufficiency is an obligation, and the deadline covers it. The knock-revalue-call-discharge ordering is temporal, outside the move algebra, and carries the safety and liveness model of Chapter 15. And a worthless unit that no voluntary machinery will bury exits by the supervised write-off path below. *One move-and-coordinate algebra, no special case: the agreement's declared terms — never the machinery — decide where the payout lands.*

9.9 The lent plane: securities lending

The title-transfer derivation applies to the lending axis verbatim. Owned is the coordinate the move machinery can act on, and the borrower's purpose is to sell: a design that keeps the security on the lender's owned and marks the loan as lent-plane mass makes the borrower's short sale either unrepresentable — the borrower has no owned balance to sell — or coverage-violating — a lent-out marker standing against an owned balance the loan has emptied. A securities loan that passes title therefore books as financing: owned re-books at instruction; the lender holds the per-agreement claim-for-equivalent; the borrower's negative balance in it is the redelivery obligation; income is manufactured through the conditional obligation of the entitlement rule. The lent plane itself remains what the posted plane is: the marker plane for arrangements under which possession moves without ownership passing, written under the same coverage bound. One derivation, two planes — the regime key applied to lending, not a second rule.

Episode (securities loan): open, dividend, recall, return. Stock SIGMA trades at 40.00; wallet W-LEND holds 1,000 shares and lends them to W-BORR under title-transfer lending agreement L, whose declared terms carry the financing regime, an income watch, and the redelivery window. The collateral leg of the loan follows the regime key of this chapter unchanged and is not repeated here.

Trigger 1 — open. The executed loan instruction is recorded; L's contract fires. One transaction:

- (1) move owned(SIGMA) 1,000, W-LEND $\rightarrow$ W-BORR — re-booked at instruction;
- (2) issuance of the per-agreement claim-for-equivalent CLM-L: move owned(CLM-L) 1,000, W-BORR $\rightarrow$ W-LEND; the borrower's $-1,000$ is the redelivery obligation, and CLM-L's declared terms name SIGMA and L.

W-BORR may now sell outright — the purpose of the borrowing. CLM-L is priced by the borrower's state: it tracks SIGMA while W-BORR performs (USD 40,000 at inception) and decouples to a recovery claim on default.

Trigger 2 — a dividend mid-loan. SIGMA declares 2.00 per share. Determination reads the owned plane: W-BORR, holding 1,000. On the payment date, move owned(USD) USD 2,000, SIGMA issuer wallet $\rightarrow$ W-BORR. The same recorded event fires L's contract on its income watch: the manufactured payment is the conditional obligation — W-BORR owes W-LEND USD 2,000, predicate carrying L's declared conditions (no default, no undischarged margin deficiency). In the ordinary state it discharges at once: move owned(USD) USD 2,000, W-BORR $\rightarrow$ W-LEND. The lender's cash offsets the dividend drop in CLM-L's price: a stockholder's

profit and loss, exactly, while the borrower performs.

Trigger 3 — recall. W-LEND’s recall notice is recorded at the boundary; L’s contract fires, and the redelivery obligation acquires its deadline — the declared redelivery window — with redelivery of 1,000 SIGMA as its discharge predicate and close-out compensation as its declared fallback. By §14.1 it is discharged, compensated, or declared defaulted by the deadline; an unmet recall is a visible, overdue item, never silence.

Trigger 4 — return. The redelivery is recorded. One transaction:

- (1) move owned(SIGMA) 1,000, W-BORR → W-LEND;
- (2) move owned(CLM-L) 1,000, W-LEND → W-BORR, taking every wallet’s vector for CLM-L to zero; CLM-L retires at the zero vector, and the redelivery obligation is discharged within its deadline.

The door, across the four triggers: conservation per (unit, coordinate) on every paired-leg move; the claim retires only from the zero vector; the deadline-bearing obligation is visible on the record until discharged. *A securities loan is title-transfer financing on the lending axis: the same re-booking, the same claim, the same conditional leg — no new machinery.*

The operational lifecycle of lending — locates, buy-ins, and the securities-financing reporting and settlement-discipline regimes that bind it — is the subject of the *SBL Operations Companion*; the ledger shape is complete here.

9.10 Retirement, and the supervised write-off

A lifecycle event extinguishes value, never mass (§4). A knock, a default, an expiry changes UnitStatus and price — never a coordinate balance — and a unit leaves the ledger only from the zero vector, the agreement’s machinery clearing the marker planes first, in the order the coverage invariant forces.

Zero-vector retirement has no voluntary exit for a defaulted issuer’s marker mass: nobody returns a worthless unit, and dead units would otherwise accumulate — polluting the encumbrance projections and drowning the obligation-liveness queue in items that can never discharge. The **supervised write-off path** is the exit: an issuer-default lifecycle event permits clearing the marker planes by compensating moves against explicit, simultaneous loss recognition — recorded, authorised, and open, without the voluntary agreement machinery, and never silent. The centrepiece’s step 6 needs the voluntary machinery; a book that cannot run it runs this one.

10 Virtual Ledgers, Strategies, and the Total Return Swap

This chapter discharges §10 of the constitution.

Nothing in this architecture is specific to real holdings. The machine of Chapters 2 through 7 — units, wallets, balances, an immutable log, and the three machines that map events into transactions and fold transactions into state — makes no reference to whether the positions it records will ever settle. A *virtual ledger* is a complete instance of that same machine whose holdings are notional: it settles nothing, and none of its wallets ever reaches a settlement instruction or the balance sheet. Everything else is identical. It has its own units and wallets, its own append-only hash-chained log, its own Event Monitor, Events Executor, and Transaction Executor, and it is subject to the same six commitments as a true ledger — above all to full reproducibility, so that any party holding the record recomputes its every number to the minor unit, and to time travel, so that every past state is reconstructible from that record alone.

The word *virtual* carries exactly one meaning throughout this specification, and no other: on the far side of the settlement boundary. A virtual ledger is not an approximation, a draft, or a lower grade of record. It is held to the identical standard of correctness; it differs from a true ledger in one respect only — its balances are claims about a hypothetical world, so no move it

records is ever projected into a payment. This is the same boundary the settlement interface draws for a true ledger, read from the other side: the true ledger says what settles, the virtual ledger says what nothing settles.

10.1 The strategy as a smart contract

The motivating case is a quantitative investment strategy: a rulebook that, reading market observations, decides a hypothetical portfolio and rebalances it on a schedule. The architecture already has the parts to record this without a new primitive.

A deterministic strategy is a smart contract, and its rulebook is the declared terms of a *strategy unit* — a non-valued unit (Chapter 3), whose price is undefined, not zero, because pricing the governance shell beside the portfolio it governs would count one economic reality twice. The hypothetical portfolio the rulebook manages is an ordinary wallet in a virtual ledger. Each rebalancing is a recorded transaction: the rebalancing date is a declared watch on the strategy unit; the Event Monitor emits the event; the strategy contract fires, reads the stamped market observations and the portfolio as the virtual ledger then stands, and proposes the moves the rules require; the Transaction Executor admits them onto the virtual log. The contract never waits and never remembers — whatever it must carry from one rebalancing to the next it writes into the three homes, exactly as every other contract does.

The strategy's *level* is that wallet's Net Asset Value: the same NAV projection of Chapter 7, the same fold of the same log against a price vector, computed on demand and never stored. Because the strategy ledger is replayable and the rebalancing contract is a pure function of recorded rules and recorded observations, any party holding the recorded rulebook and the recorded observations recomputes the level exactly — which is the whole content of the claim that the level is well defined. There is no privileged copy of it inside the strategy provider.

An index restatement follows the auditability commitment without exception. When the level of a published index is corrected — a late-arriving observation, a provider's amended fixing — the virtual ledger does not rewrite its log. It records a compensating entry that names the transaction it supersedes, so that both the original computation and its correction survive on the record. A restatement is an entry, never a rewrite; the historical level as first published and the historical level as corrected are both reconstructible, and which one a consumer read on which date is itself a fact of the record.

10.2 Two disciplines

Two disciplines make the arrangement safe, and both are checkable.

The bridge between ledgers is an observation, never a move. A move transfers a positive quantity within one ledger; conservation is stated per ledger, per unit, per coordinate, and it is closed by the virtual wallets that represent every outside party (Chapter 3). No move therefore spans two ledgers, and none can: a move from a wallet of $V-1$ to a wallet of the true ledger has no paired leg that conserves in either, so the Transaction Executor of neither can admit it. The only thing that ever crosses is a number — a stamped level, admitted into the receiving ledger through the observation door of Chapter 8, with its provenance recorded like any other observation. The two ledgers touch at exactly one point, and at that point mass does not flow; a fact is copied.

The wall between models and the ledger holds level by level. A ledger records the outputs of its models and never runs one (Chapter 16); the true ledger does not execute the strategy, it consumes the level as an observation. This is not a property of the top ledger alone. Virtual ledgers may nest — a strategy over strategies — and at each level the same wall stands: each ledger relates to the one it references only through stamped observations admitted at its boundary, never by reaching inside to run the other's machinery. Because the wall is re-erected

at every level, no amount of nesting lets a model's execution leak into an accounting record; each ledger remains a pure fold of its own log.

A short type sketch makes both disciplines mechanical:

```
-- a virtual ledger has the very same type as a true one;
-- only the settlement projection distinguishes them.
type VirtualLedger = Ledger

-- the sole thing that crosses the boundary is a stamped level:
observeLevel :: VirtualLedger -> Wallet -> Observation Money

-- a move lives inside one ledger and has no cross-ledger form;
-- none can be given a conserving type, so none can be written.
```

10.3 Real execution: orders, benchmark, and slippage

When the strategy's trades are meant to be real, the strategy's output toward the world is an *order* — an instruction, not a fact. An order records an intention to trade; it changes no balance, because nothing has happened yet. What happens is external: the market fills the order, wholly, partly, or not at all, at a price the strategy did not set. Those fills return as external events, captured at the boundary, mapped by the responsible contracts, and folded into a real wallet of the true ledger — the ordinary path of any executed trade.

The virtual ledger now becomes the benchmark. It holds what the rules say the portfolio should be; the real wallet holds what execution achieved. Both are NAV projections, each a fold of its own log against the same prices. Their difference is the slippage:

$$\text{slippage} = \text{NAV}(\text{benchmark wallet in } V) - \text{NAV}(\text{real wallet in the true ledger}).$$

It is the exact difference of two folds, computed from two records, never an estimate — every cost that separates the idealised portfolio from the realised one is captured here in full, because both sides are recorded to the minor unit and neither is a model output.

10.4 The total return swap

A total return swap (TRS) is then one definition and no new machinery: its reference leg is the NAV of a designated wallet or virtual ledger. The swap itself is an ordinary unit of the *true* ledger. Its two counterparties hold wallets there; its smart contract, at each reset, reads the stamped NAV observation and emits the reset and financing moves. The reference NAV is not computed by the TRS — it enters through the door of Chapter 8, exactly as the strategy level does, and the wall between the two ledgers is the discipline above.

Episode TRS-1. TRS-1 has notional USD 10,000,000. Its reference leg is the NAV of wallet W-QIS in virtual ledger V-1, managed by the declared rulebook of the non-valued strategy unit S-QIS. Its financing leg is fixed at 4% per annum, reset quarterly ($\Delta t = 0.25$). At inception the reference NAV observation is USD 10,000,000, and the reference level is set to that figure.

The trigger. The first quarterly reset date arrives. V-1 has been running: S-QIS's rebalancings are recorded transactions on its log, and its level is the replayable NAV of W-QIS. On the reset date that level is stamped — USD 10,300,000 — and admitted into the true ledger as an observation with provenance (Chapter 8). The Event Monitor emits the reset event for TRS-1.

The contract that catches it. The Events Executor fires TRS-1's smart contract, handing it the reset event and the current projection of the true ledger. The contract reads the stamped reference NAV and the reference level carried since inception, and computes one net move.

The transaction it produces.

#	Component	Quantity (USD)
1	Total-return leg: stamped NAV 10,300,000— reference 10,000,000	300,000
2	Financing leg: $10,000,000 \times 4\% \times 0.25$	100,000
3	Net move — owned(USD) , payer → receiver	200,000
4	Reference level reset (UnitStatus): 10,000,000 → 10,300,000	—

The total-return leg is the appreciation of the reference over the period, USD 300,000; the financing leg accrues on the period’s opening reference, USD 100,000; their difference nets to a single move of USD 200,000 of **owned(USD)** from the payer’s wallet to the receiver’s wallet. In the same transaction the contract writes the new reference level, USD 10,300,000, into the swap’s UnitStatus, arming the next period against it. One reset, one net move, one state change.

What the door checks. The Transaction Executor admits the transaction on its structure alone. The USD 200,000 move is a paired-leg **owned(USD)** transfer, so conservation holds by construction. The reference-level write has a single legal writer — the swap’s UnitStatus — and lands in its one home. The proposal carries a cause-derived identifier keyed to the reset observation, so a retried or duplicated reset commits once and no second time. The stamped NAV that the contract read is itself on the record, with its provenance, so the whole reset is reproducible: re-running the map on the recorded observation reproduces the identical USD 200,000.

The design point. The reset moves cash in the true ledger, and the strategy that produced the number never touched it: the only thing that crossed the settlement boundary was a stamped observation, and a move never can.

11 The Settlement Interface

This chapter discharges the constitution’s Scope commitment to the settlement-projection layer; it cites §3, which makes every view a projection of the record, and §4, which fixes what the record is.

11.1 Settlement as a projection

The settlement interface is where the closed ledger meets the open infrastructure of settlement. It is a projection like any other (§3): a function of the committed log that reads the moves the ledger has admitted and emits *settlement instructions*. The division of labour is exact — the ledger records *what* settles; the external infrastructure decides *how*, and how is out of scope (settlement finality, payment-system access, and central-securities-depository connectivity are external authorities the ledger reconciles against, never functions it performs). An instruction is the boundary object: it names the transfer to be performed and carries none of the machinery that performs it. Settlement in size may confirm in part: the ledger records the settled quantity each confirmation carries, as a new external fact, while how a delivery is partialled, tranced, or auto-split under settlement discipline belongs to the settlement layer — the *how*, out of scope.

What the interface never does is write. It is not the Transaction Executor and holds none of its privilege; it proposes nothing to the door and changes no balance. The single-writer discipline (§5) is not bent here — the interface reads and emits, and only the Transaction Executor writes. Every fact the interface consumes is already on the record, and every fact it produces is an instruction, never a move. Four properties follow, and each is load-bearing.

- *Pure.* It reads the recorded log alone — no clock, no call to the outside world — so any party recomputes the same instructions from the record (§2, §3).
- *Total.* It is defined on every committed transaction. A transaction that carries no boundary-crossing move — a moveless corporate-action component, a holding in a virtual ledger (§10) — has the empty instruction list as its image, by construction, not by a special rule.

- *Deterministic.* The same log yields the same instructions, always.
- *Idempotent.* Because it is a function of the recorded log and the recorded settlement state, re-running it produces the same instructions, and an instruction whose transfer the record already shows as settled is recognised and not performed twice.

```
settle :: Ledger -> [Instruction]    -- pure, total, deterministic, idempotent;
                                     writes nothing

data Instruction                    -- the boundary object: what settles, never
  how
  = Cash      WalletId WalletId UnitId MinorUnits  -- from, to, currency, amount
  | Delivery  WalletId WalletId UnitId Quantity    -- from, to, security, quantity
```

One consequence is stated once and relied on throughout: *settlement failure is a recorded event, never a reversal.* A settlement that does not complete returns as an external event, recorded as a new fact; the instruction's underlying move is not undone. History is never rewritten (§2): a trade ultimately unwound is unwound by a compensating transaction, through the same door, never by deleting the booking that instructed it.

11.2 Four episodes

Dividend: cash projects to one instruction. W-ALPHA holds 10,000 ACME; the declared dividend is 1.50 per share, and nothing encumbers the holding. The payment-date firing has already produced its transaction (Chapter 9): the conditional payment obligation discharges vacuously, and one move crosses the boundary — owned(USD) USD 15,000, ACME's issuer wallet → W-ALPHA. *The projection* reads that committed move. *The instruction:*

(1) Cash USD 15,000, ACME issuer → W-ALPHA.

The moveless component of the same transaction — the obligation's creation and vacuous discharge — crosses nothing and projects to the empty list. *The door* is not the interface's: the move already passed the Transaction Executor, and the projection only reads what was admitted. *The ledger determined and recorded the entitlement; the interface instructs only the cash that leaves the ledger's edge.*

Future: each day's margin, and the expiry unwind. FUT-IDX is settled-to-market, multiplier 10; W-ALPHA is long one contract bought at 995.00. On each settlement print the future's contract fires and emits the day's variation-margin move (Chapter 9); the interface projects each to one instruction, and the expiry's final settlement to a settlement instruction that also closes the contract.

Recorded print	Variation margin	Committed move	Instruction
day 1 1,000.00	+50	owned(USD) 50, W-CCP → W-ALPHA	Cash 50 to W-ALPHA
day 2 990.00	−100	owned(USD) 100, W-ALPHA → W-CCP	Cash 100 to W-CCP
day 3 1,005.00	+150	owned(USD) 150, W-CCP → W-ALPHA	Settlement 150 to W-ALPHA

At expiry the position unwinds: the final settlement of USD 150 projects to the settlement instruction above, while the unit-state change that retires the contract carries no move and projects to nothing. *The door* again belongs to the Transaction Executor; the interface reads three admitted moves and emits three instructions. Cumulative variation margin is $50 - 100 + 150 = 100$, the total profit and loss (Chapter 7) — but the interface never sees the profit and loss, only the cash that crosses each day. *The ledger nets the day to a single move; the interface settles exactly that move, and no valuation reaches the boundary.*

Split: the moveless corporate action projects to nothing. ACME splits two-for-one, effective 2026-06-01. One transaction carries the re-denomination and the new terms (Chapters 3 and 8):

- (1) move owned(ACME) 10,000, ACME issuer $\rightarrow$ W-ALPHA — W-ALPHA's 10,000 shares become 20,000;
- (2) a new ProductTerms version: price frame 100.00 $\rightarrow$ 50.00, with the market data operators declared (spot scales by $\frac{1}{2}$; the cash dividend figure scales 1.50 $\rightarrow$ 0.75; proportional figures stand).

The projection. Component (2) carries no move; its image is the empty list by the projection's totality. Component (1) is the ledger mirroring a re-denomination the external register performs of its own accord as the corporate action; the interface issues no delivery or payment instruction for it, because a split settles nothing between counterparties. *The door* admitted a well-formed transaction whose consistency of reference is checked in Chapter 14 (20,000 against the fresh 50.00, never the stale 100.00); the interface adds no check and emits nothing. *What crosses the boundary is a transfer; a corporate action that re-denominates terms crosses nothing, so it projects to nothing — by construction.*

Variance swap: the terminal settlement projects to one instruction. VS-1 is a one-year OTC variance swap on IDX, 252 fixings, strike 400 variance points at USD 1,000 per point. Through its life each fixing is an observation the ledger records, not a transfer (Chapters 8 and 7); none crosses the boundary, and each projects to the empty list. At expiry the realised variance is 441 points and the contract proposes one settlement move — owned(USD) $1,000 \times (441 - 400) =$ USD 41,000, from the strike payer to the realised-variance receiver. *The projection* reads that one committed move. *The instruction:*

- (1) Cash USD 41,000, strike payer $\rightarrow$ realised-variance receiver.

A whole year of fixings accrues one statistic and settles once; only the terminal cash crosses the edge, and the interface instructs exactly it.

11.3 Settlement state and the market claim

One recorded fact upstream of the projection is load-bearing for its correctness: the settlement state of a trade. A determination turns on it — who receives a distribution when a trade is instructed but unsettled across a record date — and the fact must be read, not assumed away.

Owned re-books at instruction. Title moves on the instruction event, on the record (§4), not on settlement confirmation — the trade-date booking of Chapter 9. The instant a trade is instructed the owned plane shows the buyer; the settlement instruction the interface projects is the promise that the external register will catch up. Between the two lies the instruction-to-settlement gap, and the gap is carried explicitly — never elided, never back-filled at read time.

A record date inside the gap. Economic entitlement to a distribution is fixed by the *ex-date*, not by the record date alone. A trade executed before the ex-date is *cum* — the price it paid still includes the coming distribution — and one executed on or after the ex-date is *ex*, its price already excluding it; the ex-date operator that separates the two frames is the one Chapter 8 declares, and the cum/ex invariance witness that forbids reading a price on the wrong side of it is catalogued in Chapter 14. The routing predicate reads the trade's execution date against that recorded ex-date; the record date only fixes the registration snapshot the paying agent pays against. Two cases follow, and only one raises a claim.

The cum case. A *cum* sale still unsettled at the record date leaves the owned plane showing the buyer — who is cum-entitled — while the paying agent credits the *record holder* at the external register, still the deliverer, the seller. The two disagree by construction, and the disagreement is not a break to be reconciled at report time: it is a recorded obligation. The **market claim** is that obligation — a unit (§6) whose obligee holds it positive and whose obligor holds it negative, the cum buyer's claim-for-equivalent on one side and the deliverer's duty to remit on the other, created over the gap by the record-date firing and discharged when the deliverer receives the distribution.

The ex case. An *ex* sale unsettled at the record date raises no buyer claim at all: the buyer bought *ex*, is not entitled to this distribution, and the registered holder — the deliverer — keeps it, matching the *ex*-buyer’s economic expectation exactly. The owned plane shows the buyer here too, so owned read alone would misroute; entitlement keys on the *ex*-date, and the record-date contract stays silent.

Here is the seam that keeps the projection pure: the market claim is created not by the projection, which writes nothing, but by the contract that fires at the record date. That contract reads the owned plane, the trade’s *cum/ex* status against the recorded *ex*-date, and the recorded settlement state, and proposes the reconciling leg — where the *cum* case calls for one — as one component of the determination transaction, which the Transaction Executor admits like any other. The projection then reads the resulting committed moves; it originates none of them.

```
data SettlementState = Instructed | Settled | Failed -- a recorded fact, read by
routing

-- The market claim is a unit (S6): the obligee holds it +, the obligor -.
-- Conservation makes the two signed sides one fact.
data MarketClaim = MarketClaim
{ distribution  :: UnitId          -- what the deliverer receives and owes onward
, parAmount    :: MinorUnits      -- valued at par; the deliverer's owned value
  nets to zero
, dischargePred :: Predicate       -- satisfied by the deliverer's receipt of the
  distribution
}
```

Proposition 11.1 (Entitlement routing). *The recipient of a distribution is a total function of the recorded owned plane, the trade’s *cum/ex* status against the recorded *ex*-date, and the recorded settlement state at the record date. Determination reads owned; a market claim arises in exactly one case — a *cum* trade, executed before the *ex*-date, still unsettled across the record date — where the paying agent credits the deliverer and a market-claim unit carries the reconciling leg from the deliverer to the *cum*-buyer. An *ex* trade, executed on or after the *ex*-date, unsettled at the record date raises no buyer claim: the registered holder keeps the distribution. No distribution reaches any wallet except by a recorded move against a determined entitlement or the discharge of a market-claim unit.*

Worked micro-example (dividend). Stock ACME pays a cash dividend of 1.50 per share; the record date is 2026-05-15. Wallet W-ALPHA holds 10,000 ACME and sells them to a buyer, wallet W-BETA: the sale is instructed on 2026-05-14 for settlement on 2026-05-16, so the record date falls in the gap. The 2026-05-14 execution precedes the *ex*-date associated with the 2026-05-15 record date, so the sale is *cum* — which is why the distribution routes to the buyer; an otherwise identical trade executed on or after the *ex*-date would be *ex* and raise no market claim. (The cash consideration settles symmetrically and does not bear on the entitlement.)

2026-05-14 — the sale; owned re-books at instruction. One transaction:

- (1) move owned(ACME) 10,000, W-ALPHA → W-BETA — re-booked at instruction; the settlement instruction, due 2026-05-16, is recorded as unsettled.

The owned plane now shows W-BETA holding 10,000 ACME and W-ALPHA holding none.

2026-05-15, record date — determination and the market claim. The record-date watch fires; the stock’s contract determines the entitlement and, reading the trade’s *cum* status against the *ex*-date and the recorded settlement state, proposes the reconciling leg. One transaction:

- (1) determination reads the owned plane — W-BETA owns 10,000 — and records the entitlement, $10,000 \times 1.50 = \text{USD } 15,000$, to W-BETA in PositionState;
- (2) the sale is *cum* and unsettled across the record date, so a market claim arises: the paying agent will credit the record holder, the deliverer W-ALPHA. Issue the market-claim unit

MC-1, valued at par: move owned(MC-1) 1,500,000 (minor units, = USD 15,000), W-ALPHA → W-BETA — W-ALPHA holds −1,500,000 (the duty to remit once received), W-BETA holds +1,500,000 (the claim-for-equivalent). MC-1’s discharge predicate is the deliverer’s receipt; the obligation is contingent on it and is matched by the deliverer’s record-holder receivable, so the conduit’s owned value is neutral. While W-ALPHA performs the claim is valued at par, and it decouples to a recovery claim on W-ALPHA’s default.

This transaction is admitted through the door; MC-1 exists because the record-date firing wrote it, not because the projection did.

The sale settles on 2026-05-16, recorded as an external confirmation; this updates the register but not the record-date snapshot that governs the distribution — the paying agent still credits the record holder as of 2026-05-15.

2026-05-22, payment date — the distribution and the discharge. The payment-date watch fires. One transaction, two moves:

- (1) move owned(USD) USD 15,000, ACME issuer wallet → W-ALPHA — the paying agent credits the record holder;
- (2) move owned(USD) USD 15,000, W-ALPHA → W-BETA — the market claim discharges under its predicate, and MC-1 retires at the zero vector.

W-ALPHA nets to zero across the two moves — a conduit, not an owner; W-BETA ends with the USD 15,000 it economically owns. *The door*: conservation per (unit, coordinate) on every paired-leg move; MC-1 retires only from the zero vector; idempotence — a duplicate payment-date event finds MC-1 discharged and proposes nothing. Each of these two cash moves is what the interface then projects to a cash instruction; the market claim itself never crosses the boundary — it is the internal record that keeps the two boundary-crossing payments consistent.

Settlement failure is a recorded event, never a reversal. Should the sale fail to settle on its due date, the failure returns as an external event and is recorded as a new fact; the instruction’s owned re-booking is not undone and the market claim is not withdrawn. The market claim, an obligation unit, carries a deadline, a discharge predicate, and a fallback (§14.1); by its deadline it is discharged, compensated, or declared defaulted — a failed distribution is a visible, overdue item, never a silent balance edit.

Load-bearing, not a reporting axis. Settlement state is therefore a correctness input, not a presentation dimension. A design that read owned alone would instruct the venue to pay the buyer directly — an instruction it cannot honour, since the buyer is not yet the record holder — and the cash would arrive at the deliverer with nowhere on the record to go, resurfacing as exactly the reconciliation break the ledger exists to abolish. Reading settlement state, and recording the market claim as a first-class unit, makes the routing both faithful to what the venue will do and reproducible by any party from the record alone (§2, §3). The construction is symmetric: where an owned re-booking runs the other way across the record date, the same market-claim unit carries the mirror obligation, its signs reversed — one mechanism, total over the gap.

The ledger says what settles and, across the gap, who is owed; the interface projects that settled activity into instructions, writing nothing and inventing nothing.

12 Presentation and Reporting Projections

This chapter discharges §1 and §3 of the constitution — a report is a projection of the one record, computed on demand — and the auditability and reproducibility commitments of §2.

12.1 The projection principle

Every reporting figure is a projection: a pure, total, deterministic function of the record and of the declared terms the record carries, and of nothing else. There is no reporting store and no parallel reporting pipeline; a report is computed when asked for and discarded when read.

The principle is forced, not chosen. §1 removes reconciliation by recording each fact exactly once; a reporting database would be a second store of the same facts, maintained by its own logic on its own schedule, and two stores of one fact eventually disagree. The divergence would surface in the documents third parties read — the balance sheet, the disclosure, the supervisory return — which is the worst possible place for the failure mode the architecture exists to abolish. So no reporting figure is ever kept: what can be computed from the coordinates, the units, and the declared terms is computed from them, every time.

Three commitments of §2 are discharged at once, by construction rather than by apparatus. *Auditability*: a projection is a fold over the immutable log (Chapter 2), so every figure decomposes mechanically into the transactions that entered the fold — drill-down is the fold’s own trace. *Reproducibility*: the function is pure and its inputs are recorded, so any party holding the record recomputes the same figure to the last minor unit; where a figure involves value, the prices come from the pricing layer, itself reproducible from recorded inputs (Chapter 16). *Time travel*: a report as of any past date is the same function applied to the log up to that date, and a restated report is a new projection of a longer log containing recorded corrections — never an edited document.

One input class stands outside the record: reference data — identifiers, classifications, counterparty attributes — owned by external authorities, per the constitution’s scope. It is supplied at the boundary by the authority that owns it and joined to the projection as the report is built. The join attaches labels and groupings; it can never alter a quantity. Gating admission and altering a quantity are different acts, and neither contradicts the other: an eligibility predicate may read declared or reference data to refuse a transaction at the door (Chapter 9), but a refusal admits or rejects, it never rewrites a booked number — and the report-time join does not even gate, it only labels. A figure carrying a wrong identifier is a labelling defect at the boundary, repaired by re-joining fresh reference data to the unchanged projection — never a ledger defect. The same division of labour governs format: the ledger computes what the figures are; a submission gateway, outside the boundary, arranges them into whatever shape a supervisory template requires. Per-regime field inventories therefore live outside this specification and are recorded as exclusions in the Exclusions Register; where a template requires a figure, the figure is derived here from the record and shown to satisfy the template — the template is never the derivation.

```
-- One arrow, no state of its own, total in every argument.  
-- Declared terms and conventions travel inside the prefix.  
report :: LogPrefix -> BoundaryRef -> Report
```

12.2 The presentation projection

Chapter 9’s regime key decides what a collateral inflow writes; the presentation projection recovers from those writes the figures that a balance sheet or a supervisory template asks of a firm that finances, pledges, and re-uses assets. It reads coordinates, units, and declared terms alone. Four figures cover the surface.

Transferred assets and associated liabilities. Under title-transfer financing the poster’s asset re-books to the taker; the poster holds a claim-for-equivalent unit and the taker a repurchase or redelivery obligation unit, both per governing agreement (Chapter 9). The projection maps each claim held under a financing agreement to a transferred asset at fair value — the claim priced by the pricing layer, parameterised by the taker’s state (Chapter 7) — and maps the

paired obligation to its associated liability. The presentation is complete within the ledger’s fair-value scope.

Encumbrance, poster side. Under pledge the asset never leaves the poster’s owned coordinate; its encumbered mass is the posted-as-collateral ray, per agreement. Under title-transfer financing the encumbered position is the claim held under the financing agreement. Unencumbered mass is owned less posted, and it is non-negative because the coverage invariant (Chapter 14) bounds posting by ownership — the report cannot show negative free assets because the door refused the state that would produce one.

Re-use, receiver side. Mass held on the received-as-collateral ray is available for re-use. Actual re-use is read from re-pledge moves, each of which carries a source-agreement reference beside its governing reference (Chapter 9); the source is an observable fact of the custody instruction, so nothing fictitious is stored, and received, re-used, and available project as per-(unit, agreement) actuals.

Fungible received mass. Title-transfer received cash and securities land on owned, commingled with proprietary holdings of the same unit. The record cannot say which dollar was re-invested, because no bank statement can say it either: a per-dollar “actual” over commingled mass would be a stored fiction. Figures over this mass — how much of the received value was re-used, and where — are attributions, computed by the declared convention of the next subsection.

Reported figure	Read from — and from nothing else
Transferred asset / associated liability	claims and obligation units under financing agreements
Encumbrance under pledge	posted-as-collateral ray of owned units, per agreement
Encumbrance under financing	claims held under financing agreements
Re-use actuals (pledge form)	source-agreement references on re-pledge moves
Re-use of received fungibles	owned mass, obligation units, the declared convention

12.3 The attribution convention is total

The convention is data, not code: it is declared once, in the declared-data slot of Chapter 8, and consumed here — behaviour carried as declared data, per §2. It has exactly three components:

1. the *pool predicate* — a declared wallet and move predicate defining the pool over which received value is attributed (for instance, the wallets and moves constituting the reinvestment book); it is total over recorded log state, ranging only over wallets and moves already on the log, so an input not on the record is not admissible and there is no hidden manual-tagging step outside the projection;
2. the *period basis* — the timestamp or period on which the pro-rata ratio is struck;
3. the *rounding rule* — with remainders booked explicitly, per §4’s remainder discipline; nothing is silently dropped.

Principle 12.1 (Attribution totality). *The declared convention is a total function of log state: for every log prefix it assigns every attributable quantity; the attributed parts and the explicit remainder partition the whole; and two readers of one log and one declared convention compute identical figures to the last minor unit.*

A convention missing any of the three components is a defect: two readers of one log would compute different figures, which is internal reconciliation failure reintroduced through a report. The projection therefore fails closed — it refuses to produce the figure rather than improvise a ratio (§2, Correctness).

```
data Convention = Convention
{ pool      :: Predicate (Wallet, Move) -- who is in the pool
```

```

, basis      :: PeriodBasis          -- when the ratio is struck
, rounding   :: RoundingRule }      -- remainder explicit
attribute :: Convention -> LogPrefix -> Partition Amount

```

12.4 The reconciled surface

An attributed figure is single-sided: it is the firm’s own aggregate, and no counterparty computes a matching number to break against. For such figures the correctness standard is Principle 12.1 — totality and determinism — not agreement with a second party, because no second party exists.

One figure on this surface is genuinely two-sided: the collateralisation of net exposure under a governing agreement. It is the specification’s single reconciled surface, present by design and named beside the projections, not a concession extracted from them after the fact. The counterparty computes the same figure from the same agreement, and the two are compared at the boundary. It is therefore sourced from the governing agreement unit’s declared terms — valuation percentages, thresholds, netting — consistently with the counterparty, and it is named beside the single-sided aggregates in every presentation, so that a reader never mistakes an attribution for a reconciled fact. Where the two sides disagree, the disagreement is a boundary reconciliation item against an external authority — the class the constitution’s scope leaves at the perimeter — never an internal break: inside the boundary there is one log and one declared convention.

12.5 Episode: the pledged one-touch in the encumbrance projection

OT-1, the one-touch of Chapter 2, pays USD 1,000,000 on a touch of OMEGA at or below 80.00; it is held by W-ALPHA, marked 400,000 pre-knock, and pledged security-interest to counterparty B under agreement G at a declared valuation percentage of 50%: W-ALPHA posted +1, B posted −1 (Chapter 9). Before the knock, the encumbrance projection reads, with no store of its own: under G, mass posted 1; price 400,000; collateral value $400,000 \times 50\% =$ USD 200,000 against an exposure of USD 150,000 — sufficiency holds.

The trigger. OMEGA’s official close 79.00 is recorded, at or below the declared barrier 80.00; the Event Monitor emits the touch.

The contract. OT-1’s contract fires on the event with the current projection of the ledger and proposes one moveless transaction.

The transaction.

1 UnitStatus of OT-1: *live* → *triggered*

2 conditional payment obligation created: writer owes W-ALPHA USD 1,000,000, predicate G’s trapping te

No moves; no coordinate changes anywhere.

The door. Conservation holds trivially — there are no moves; writer discipline — the unit’s contract is the sole writer of its UnitStatus; a duplicate emission finds the unit already *triggered* and proposes nothing.

The read after. The same projection, one transaction later: mass posted under G is still 1 — a lifecycle event extinguishes value, never mass (§4) — but the pricing layer, parameterised by unit state, now prices the triggered OT-1 at 0, so collateral value reads $0 \times 50\% =$ USD 0 against the unchanged exposure of USD 150,000. The shortfall does not live in the report: it surfaces as G’s margin-call obligation at G’s declared valuation watch, with a deadline (Chapter 9), and the report shows both the collapsed line and the open obligation from the record alone.

The projection reads mass from the coordinates and value from the pricing layer, so the collapse is on the report the instant it is on the record — and there was no report-side store to go stale.

12.6 Verification and the audit gate

Per the Testability commitment, the projections of this chapter enter the executable-check regime of Chapter 15 as properties over generated logs, agreements, and conventions:

1. *Presentation completeness* — every claim held under a financing agreement appears exactly once as a transferred asset, and its paired obligation unit exactly once as the associated liability.
2. *Encumbrance bound* — per (wallet, unit), reported encumbered mass never exceeds $\max(\text{owned}, 0)$: the coverage invariant of Chapter 14, read back from the report.
3. *Attribution partition* — attributed parts plus the explicit remainder equal the whole, for every generated log and declared convention.
4. *Re-use bound* — per source agreement, re-pledged mass never exceeds the mass received under it.
5. *Mass-value separation* — a lifecycle event inserted into a generated history changes reported values, never reported mass, exactly as the episode above exhibits.
6. *Recomputation* — every published figure equals the figure recomputed from the log prefix and the boundary join: the reporting analogue of §13's economic verification.

Two readers of one log compute identical reports; the properties above make that sentence executable rather than asserted. One gate remains before the specification freezes: the auditor bench signs the faithfulness of the presentation projection — that the mapped figures state what a fair-value presentation asks — against these property tests. The sign-off is recorded here as a Phase 4 gate of the drafting programme. It confirms that the derivation is faithfully presented; the derivation itself rests on the constitution alone.

13 Alignment with the Common Domain Model

This chapter discharges constitution §6 — every right and obligation the legal contract creates is a machine-readable unit — and the §2 Clarity commitment that behaviour is carried as declared data. It discharges them by a single demonstration: the industry's Common Domain Model (FINOS CDM, version 6.0.0) maps onto the ledger's declared-data vocabulary. The direction of authority is the constitution's and does not turn. The ledger's objects are derived from first principles; the CDM is exhibited *against* them, never consulted as their source. Where the two models disagree, the disagreement is a finding about the mapping — recorded, and carried by a lineage convention — and never a reason to alter a ledger primitive.

13.1 Unit maps to product; a bespoke instrument maps to the trade

A ledger unit's declared terms map to a CDM product. The counterparty-agnostic template — the CDM `NonTransferableProduct`, whose `EconomicTerms` carry the `Payout` legs, the effective and termination dates, and the notional — is the image of the unit's product-graph terms (Chapter 3). For a listed, fungible instrument this template is the whole image: the same terms serve every holder.

For a bespoke bilateral instrument the faithful image is not the template but the CDM `Trade`. The reason is economic individuation. In CDM the collateral terms live on `Trade.collateral`, whose type `Collateral` carries the `CollateralProvisions`; so two executions sharing one `NonTransferableProduct` but differing in their collateral terms are distinct `Trades`. The ledger reaches the same conclusion from the other direction and for the same reason: collateral eligibility and the governing regime are declared terms of a distinct agreement unit (Chapter 9), and a unit whose economics are inseparable from those terms is a distinct unit. Unit identity therefore corresponds to the CDM `Trade` layer, not to the `NonTransferableProduct` template — collateral terms individuate a trade economically in both models. The individuation is not the identifier: a CDM trade's identity is carried by its `TradeIdentifier`, and collateral terms

individuate the economics that identifier then names. The one structural difference is placement: CDM co-locates the collateral terms — `Trade.collateral`, a `Collateral` carrying the `CollateralProvisions` — inside the `Trade`, whereas the ledger factors the same terms into a separate agreement unit and relates the two by projection. That is a finding about the mapping, carried by the agreement-reference lineage of Chapter 9; it changes no primitive.

13.2 Business event maps to transaction

The CDM `BusinessEvent` — a before-state, a set of `PrimitiveInstructions`, and the resulting `TradeState` — maps to the ledger’s recorded *transaction*, the atomic bundle that advances the fold. The correspondence is structural on both sides. CDM composes a `BusinessEvent` from primitive instructions (quantity change, termination, transfer, reset, observation, and their siblings); the ledger composes a transaction from its four kinds of change (moves, unit-state changes, position-state changes, terms changes). The map from ledger transactions to CDM business events has the shape the constitution already requires of the ledger’s own arrows, and it inherits their properties:

- **total** — every recorded transaction has a CDM image, because every transaction is one of the four declared kinds of change and each maps to a primitive instruction or a qualified composition of them;
- **deterministic** — the image is a function of the record alone;
- **composing** — a transaction’s four change-kinds map component-wise to the event’s primitive instructions, so composition of changes is composition of instructions;
- **conserving** — the ledger’s transaction conserves value per (unit, coordinate) by construction; the CDM event does not carry this law, and the ledger supplies it;
- **idempotent under the cause-derived key** — a re-presented cause commits once, under the identifier of constitution §5; CDM has no native idempotence key, and the ledger’s key is the lineage carried across the map.

The division of labour is exact and worth naming: *the ledger keeps the economic substance, the CDM carries the business meaning*. The conserved moves and the state written into the three homes are the substance, guaranteed by the Transaction Executor; the CDM qualification — that this event *is* a partial termination, a reset, a variance fixing — is the meaning, layered on top of substance the ledger has already made unforgeable.

Variance swap. Unit VS-1 (one-year OTC variance swap on IDX, 252 fixings, strike 400 variance points, 1,000 USD per point) maps to a product whose `EconomicTerms` carry one `PerformancePayout`; its underlier is the index IDX as an `Observable`, and the variance computation sits in the payout’s `returnTerms` as `VarianceReturnTerms` — the field on which the CDM equity-variance qualification depends. Each of the 252 fixings — the recorded IDX official closes — is a CDM `Observation`, an observed value on the record; the ledger’s accrual firing, which writes the accrued sum-of-squared-returns statistic into `UnitStatus` for the next firing to find, maps to a `BusinessEvent` whose `Reset` lands in the trade’s `resetHistory`. Each fixing is thus a recorded observation and each accrual a recorded business event, exactly as the two models agree they should be. Final realised variance 441 points settles $1,000 \times (441 - 400) = \text{USD } 41,000$ to the realised-variance receiver: one CDM transfer primitive, one ledger owned-cash move, one conserved fact.

Total return swap. Unit TRS-1 (notional USD 10,000,000, financing 4% quarterly) maps to a two-legged product: a `PerformancePayout` for the reference leg, whose underlier is the NAV of wallet W-QIS as an `Observable`, and an `InterestRatePayout` for the financing leg. The first reset is a CDM `BusinessEvent` carrying a `Reset` (reset value: the stamped NAV observation USD 10,300,000) and a transfer; the ledger records it as one transaction with a

single net move of USD 200,000, payer to receiver. The alignment makes a reporting point the netting would otherwise erase: the report derives from the *retained business event* — total-return leg USD 300,000 against financing leg USD 100,000, both recoverable by recomputation from the recorded reset and NAV observation — never from the USD 200,000 net move alone. Substance is the net move on the record; meaning is the gross legs the business event retains.

13.3 Eligibility as declared data restates the CDM collateral model

The ratified collateral ruling makes eligibility and haircuts declared terms of the collateral-agreement unit, checked at the single door and never encoded in a type. This is the CDM collateral model restated in the ledger's primitives. CDM carries eligibility as an `EligibleCollateral` schedule — asset class, currency, and a per-line valuation percentage after the shape of CSA Paragraph 13 — read as data, not as a subtype hierarchy of collateral kinds. The ledger's line-level valuation default (Chapter 9) is the same choice: collateral value is the sum of per-line market value times the declared percentage, package-level valuation admissible only where an agreement declares it. That the two models, derived independently, land on *eligibility as declared data valued line by line* is the alignment this section exhibits; it is shown, not borrowed.

13.4 Closed enumerations and the product graph

Two further alignments serve later chapters. First, the generator universe of Chapter 15 rests only on the enumerations that are genuinely closed. The CDM `Payout` is not one of them: its components *compose* rather than *exclude*. A product's `EconomicTerms` co-populate one payout component per leg — `EconomicTerms.payout` has cardinality `(1..*)`, so an interest-rate swap carries two interest-rate legs and a total return swap an interest-rate leg alongside a performance leg — and a product is therefore a composition of payout components, not a choice of one from a fixed arity. The closed enumerations that do seed the generator universe are the day-count fraction, the option type (call or put), and the primitive-instruction set, together with the ledger's own closed sets: the regimes {title-transfer, pledge, STM/CTM}, the coordinate planes {owned, lent, posted}, the trigger and event kinds, and the product-graph node and edge sets closed at registration (Chapter 3). These enumerations seed the generator universe of Chapter 15, so that the property tests range over the industry's own catalogue of product and event shapes together with the ledger's, not an invented one.

Second, the product graph of Chapter 3 — nodes the closed lifecycle states, edges the guarded transitions that emit transaction templates — is the natural translation target for CDM lifecycle representation. A CDM `BusinessEvent` is an edge traversal: the before-`TradeState` is the source node, the after-`TradeState` the target, the primitive instructions the edge's action. The CDM lifecycle and the ledger graph are the same object described twice; the graph is where a CDM event round-trips without loss.

13.5 Gaps

Where CDM 6.0.0 has no native form for a situation the ledger must represent, the mapping states the gap and names the lineage convention the ledger carries instead. None is a defect in the ledger; each is a place the industry model has not yet reached, recorded so that a later CDM version can be met without surprise.

Situation	CDM 6.0.0 native form	Lineage convention carried
Securities-lending recall	None — SL lifecycle events are not primitive instructions	Lent-plane transaction keyed to the loan agreement unit (Chapter 9): open → recall → return
Locate-style pre-trade confirmation	None — CDM is a post-trade model	The locate is a declared obligation on the agreement unit, with deadline and discharge predicate; the pre-trade act itself is out of ledger scope
Collateral re-use / rehypothecation event	Collateral is modelled; re-use is not a first-class event	Posting of mass on the received ray with a mandatory source-agreement reference, from which per-agreement re-use projects
The coordinated cascade (one cause, many units)	None — events are per-trade	The cause-derived key of constitution §5 ties the cascade's transactions together and makes them jointly idempotent

No gap adds a mechanism: the recall rides the lent plane, the locate rides the obligation-liveness discipline, re-use rides the source-agreement reference, and the cascade rides the cause-derived identifier. The mapping is total on the ledger's side precisely because the residue is absorbed by primitives the design already carries.

14 The Invariant Catalogue

This chapter discharges §11 and §12 of the constitution.

The chapters before this one *use* invariants; this chapter states each one once, as an executable property, and discloses the inner ground for it — the reason it holds, not merely the fact that it does. The catalogue divides in two. A **law** holds by construction: no check runs, because the property is a theorem about the fold, and a state violating it is unrepresentable. A **guard** is enforced at the door: the Transaction Executor computes it on every admission, and a violation is refused, never written. Each property below is stated in the form a test consumes; the executable-check regime of Chapter 15 (Testability and the Executable-Check Regime) exercises all of them over generated products, events, and histories, together with the product-graph consistency invariant of Chapter 3 — restated there, not here.

14.1 Conservation per (unit, coordinate)

The central law is a conservation law, and it is stated per coordinate, not per unit. For a wallet w , a unit u , and a coordinate $c \in \{\text{owned, lent, posted}\}$, write $\text{bal}_c(w, u)$ for the balance obtained by folding the log (Chapter 2).

Theorem 14.1 (Conservation). *For every unit u , every coordinate c , and every position in the log,*

$$\sum_w \text{bal}_c(w, u) = 0,$$

the sum running over all wallets — virtual counterparty and issuer wallets included, because the system is closed (Chapter 3).

Issuance is the first paired-leg owned move out of the issuing wallet; retirement is the log position after which every wallet's vector for u is the zero vector. The per-unit summed law — $\sum_w \sum_c \text{bal}_c(w, u) = 0$ — is a corollary, never the primitive: the per-coordinate form is primitive because it alone forbids a cross-coordinate rebooking that fabricates custody on the posted plane while conserving the gross sum.

Proof. Induction on the log. *Base:* the empty log gives every (u, c) plane a sum of zero. *Step:* the only operation that writes a balance is an atomic move, which debits one wallet and credits another in the same coordinate of the same unit by the same positive quantity q , adding $(-q) + (+q) = 0$ to the sum; the other three kinds of change a transaction bundles — unit-state, position-state, terms — touch no balance. The sum is therefore invariant across every admitted transaction, and remains 0. $\square$

```
conserved :: Log -> Unit -> Coord -> Bool
conserved l u c = sum [ balance c w u (fold l) | w <- wallets l ] == 0
-- a law, not a guard: the fold cannot reach a prefix where this is False.
```

The word *conservation* is exact: the vanishing sum is the quantity a symmetry of the log preserves. Reorder any two independent transactions — ones that share no (wallet, unit, coordinate) — and every coordinate sum is unchanged, because each is a sum of paired-leg contributions that commute. Permutation invariance of independent moves and the vanishing coordinate sum are one fact seen twice; the first is itself a checkable (metamorphic) property.

Episode — the future (T1): variation margin is conservation, not a check. *Trigger.* FUT-IDX (multiplier 10, settled-to-market) prints official settlements 1,000, 990, 1,005 on its three days; W-ALPHA is long one contract, bought at 995 on day 0, against the central counterparty wallet W-CCP. *Contract.* Each print fires the future’s contract, which proposes one owned(USD) move between the two wallets (Chapter 9). *Transactions.* Three, one per print:

- (1) day 1, 995 $\rightarrow$ 1,000: move owned(USD) USD 50, W-CCP $\rightarrow$ W-ALPHA;
- (2) day 2, 1,000 $\rightarrow$ 990: move owned(USD) USD 100, W-ALPHA $\rightarrow$ W-CCP;
- (3) day 3, 990 $\rightarrow$ 1,005 (final): move owned(USD) USD 150, W-CCP $\rightarrow$ W-ALPHA; the position unwinds.

The door. Each move is paired-leg owned(USD): whatever W-ALPHA gains W-CCP loses, so $\text{bal}_{\text{owned}}(\text{W-ALPHA}, \text{USD}) + \text{bal}_{\text{owned}}(\text{W-CCP}, \text{USD})$ contributed by these moves is zero at every print. The cumulative variation margin to W-ALPHA is $50 - 100 + 150 = 100 = (1,005 - 995) \times 10$, exactly the position’s profit and loss (Chapter 7). *Design point.* *Variation margin is zero-sum because it is conservation per (unit, coordinate); no reconciliation enforces the zero-sum — the paired-leg move makes any other outcome unrepresentable — and the cumulative 100 W-ALPHA gains is exactly what W-CCP pays, and equals the profit and loss.*

A second telling, in categorical language; nothing later depends on it.

Fix (u, c) . The map sending a log to the wallet-indexed integer vector of balances is a homomorphism from the free monoid of moves under concatenation to the group of such vectors under addition: the empty log maps to the zero vector, concatenation to addition. Each move maps to a vector carrying $+q$ and $-q$ in two entries and zero elsewhere — an element of the subgroup whose entries sum to zero — and a homomorphism into that subgroup keeps every image inside it. Conservation is the statement that the image lies in the kernel of the sum-over-wallets map. Nothing below depends on this telling.

14.2 The log’s structural invariants

Four further properties are laws of the log itself, guaranteed by the single-writer discipline of the Transaction Executor (Chapter 4). Each is stated once, in the form a test consumes.

```
atomic      :: Tx  -> Ledger -> Bool  -- apply writes the whole bundle or
nothing
appendOnly  :: Log -> Log   -> Bool  -- a later log extends an earlier one; no
edit
verifyChain :: Log -> Bool              -- each entry carries its predecessor’s
hash
idempotent  :: Tx  -> Ledger -> Bool  -- re-applying a seen cause is a no-op
```

- **Atomicity.** A transaction applies in full or not at all: the Transaction Executor computes the whole four-kind bundle against the current ledger and either appends all of it or writes nothing. A refusal is a returned value, never a half-applied state — no log position exists at which some but not all of a transaction’s changes are present. The split shows it: no instant exists at which 20,000 shares stand beside pre-split terms (Chapter 3).
- **Append-only and hash-chained.** An admitted transaction is never edited or removed; a mistake is repaired by a compensating transaction that names what it supersedes. Each entry carries the hash of its predecessor, so any alteration of history breaks the chain and is detectable (Chapter 3).
- **Determinism of every arrow.** Each of the three arrows — watch, contract, apply — is a pure function of its recorded inputs (Chapter 2): the same record and clock emit the same events, the same event and ledger propose the same transaction, the same transaction and ledger reach the same ledger. Determinism is the ground on which replay and economic recomputation (Chapter 15) stand.
- **Idempotence.** Every proposed transaction carries an identifier derived from its cause (Chapters 4 and 2); a retried or late-arriving proposal for an already-processed cause is recognised and committed once. Applying a transaction whose identifier is already on the log changes nothing.

14.3 Consistency of reference

A quantity and a price name value only when both refer to the same state of the instrument. Every recorded datum carries the basis frame it was observed in (Chapter 8); the guard refuses a read that would combine a quantity with a price drawn from a different frame.

Invariant 14.2 (Consistency of reference). *A quantity Q and a price P may be combined only when Q and P are read in the same corporate-action frame. Across an event carrying a quantity change (a move) and a value-frame change (an operator), total value is preserved: $Q_{pre} \times P_{pre} = Q_{post} \times P_{post}$. This binding is the invariance witness, and a product taken across mismatched frames is refused as a phantom.*

Episode — the split (T4): the phantom the witness refuses. *Trigger.* ACME’s 2-for-1 split is effective; W-ALPHA’s 10,000 shares become 20,000 by a paired-leg move, and the recorded price frame reads 100.00 as 50.00 under the split’s operator (Chapter 8). *The witness.* Before: $10,000 \times 100.00 = \text{USD } 1,000,000$. After, in the post-split frame: $20,000 \times 50.00 = \text{USD } 1,000,000$. The phantom is $20,000 \times 100.00 = \text{USD } 2,000,000$ — the new count against the stale frame. *The door.* The 20,000 is stamped in the post-split basis and the 100.00 in the pre-split basis; the frames do not match, so the product is refused. The count changes by a move, the frame by an operator, and the two are bound so their product is invariant. *Design point.* $20,000 \times \text{stale } 100.00$ is a phantom USD 2,000,000 against a true USD 1,000,000; the witness refuses it because the quantity and the price are read in different frames — the same witness Chapter 8 leans on for every corporate action.

The same witness governs a distribution’s cum/ex boundary, where no quantity changes at all. A cum price includes the coming distribution; an ex price excludes it; the two are one datum kind read in states separated by the recorded ex-date event, whose declared operator subtracts the distribution’s value. A cum price consumed as ex, or an ex price consumed as cum, is a phantom of the split’s kind — the right number in the wrong frame — and the same witness refuses it, because the frame is fixed by the recorded ex-date state, not the wall clock. Chapter 8 cross-references this invariant for both readings.

14.4 Writer discipline

Invariant 14.3 (One legal writer). *Every non-balance fact has exactly one legal writer — the smart contract of the instrument the fact belongs to — and the Transaction Executor refuses a transaction that writes a fact outside its writer’s authority (Chapter 6, constitution §11).*

One writer per fact, one home per fact (Chapter 6): a fact cannot be written into disagreement with itself, because no second writer and no second home exist to hold a rival copy. This is what keeps the three homes — ProductTerms, UnitStatus, PositionState — free of the divergence a second store would breed. Each is a *materialised projection*: a cached, one-writer, rebuildable read of the log, provably equal to the fold it caches, and *not* a second store of any derived quantity. No figure a valuation or a report recomputes is materialised anywhere — NAV, profit and loss, availability, encumbrance are computed on demand (Chapters 7 and 12) — so there is nothing derived to reconcile against the record. Writer discipline over facts, and the refusal to store views, are jointly the condition under which replay reproduces the homes exactly.

14.5 Time travel and replay

Section §12’s guarantees are theorems of the fold.

Theorem 14.4 (Time travel). *Every historical state of the ledger is the fold of a log prefix. For any past position t , the ledger as of t is fold step over the prefix up to t , and every view — a balance, a NAV, a report — is a further fold of that same prefix. Because the fold reads only committed transactions and never re-executes contracts, the historical state is independent of the machinery that produced it and reconstructible at any time, forever.*

Theorem 14.5 (Replay determinism). *Discarding the ledger and the three homes and re-folding a log prefix reproduces them bit for bit. The apply arrow is deterministic (the determinism of every arrow, above), each home is a materialised projection with one legal writer (Invariant 14.3), and a retried or late proposal is idempotent under its cause-derived identifier; so replay adds nothing and drops nothing.*

Proof. The ledger is defined as the fold of the log (Chapter 2); restricting to a prefix folds fewer transactions, and determinism of apply makes the result a function of the prefix alone. Each home is the fold of its own kind of change over the same prefix (Chapter 6), hence reproduced by the same replay. Idempotence guarantees a duplicated cause contributes once. $\square$

These two theorems are the executable ground of the constitution’s Full Reproducibility and Time-Travel commitments. Chapter 15 (Testability and the Executable-Check Regime) exercises them — regenerating a mid-life firing from the recorded fixings and comparing — and this catalogue does not restate the test.

14.6 The coverage invariant and the two-guard contrast

Collateral introduces one guard and one obligation that look alike and are structurally opposite. This is their canonical home; Chapter 9 uses both.

Invariant 14.6 (Possession coverage). *No wallet delivers what it neither owns nor holds. On every admission, per wallet w and unit u , net over agreements G ,*

$$\sum_G \text{posted}_G(w, u) \leq \max(\text{owned}(w, u), 0),$$

the received ray entering with positive sign, and the analogue with lent in place of posted.

The sum over signed balances lets received mass offset re-posted mass: a wallet that received 100 under one agreement and re-posted 60 under another nets to coverage 40, within the bound, so a rehypothecation chain passes and is bounded by possession. The max binds only the posting direction, so a negative-owned wallet with nothing posted — an issuer wallet that conservation itself creates, a plain short — is admissible, while a poster with owned $-5,000$ attempting to post 5,000 is refused at the door. Coverage is never lawfully false, because a transaction is atomic and possession is instantaneous within it: that is what makes it an invariant.

Coverage binds *instructed* ownership, and the qualifier is load-bearing. Owned is booked at instruction — trade-date (Chapter 9) — so a wallet may post or deliver a security it has bought but not yet settled: that state is reachable, admitted at the door because instructed ownership covers it. The invariant forbids only *recording* a state whose posted mass exceeds instructed ownership. A later involuntary reduction of owned — a failed or cancelled purchase that would drop owned below the mass already posted — is therefore not a silent breach but a transaction *refused at the door*: it cannot be recorded while it would violate coverage. The unmet return is carried instead as a recorded, deadline-bearing obligation (Chapter 9), and on default it escalates to the supervised write-off path there. The fails cascade thus lives inside the model — order-forced at the door, then obligation, then supervised write-off — and the invariant is never false in any committed state.

On fungible cash coverage is a per-move check: a cash posting is admitted against owned cash at the instant it is recorded, and because the same owned USD backs every obligation drawn on it, aggregate cash adequacy across all obligations is not coverage at all but the sufficiency obligation — a deadline, not an invariant.

Guard	Statement	Lawfully false?	Kind
Possession	$\sum_G \text{posted}_G \leq \max(\text{owned}, 0)$	never (atomic, instantaneous)	invariant, checked at the door
Value sufficiency	collateral value $\geq$ exposure	intraday, routinely	obligation: deadline, discharge predicate, close-out as compensation

Value sufficiency is the guard coverage is never to be confused with. A market move, a knock, or a call in flight leaves collateral value below exposure with no party in breach; sufficiency is therefore not an invariant but an obligation — a deadline, a discharge predicate (the delivery amount met), and close-out as the declared compensation (Chapter 9, §14.1). A book below its sufficiency line is not a broken state; it is an open, deadline-bearing item on the record.

Episode — the one-touch (T2): coverage on the discharge; value extinguished, mass intact. *Setup.* W-ALPHA owns OT-1 (payout USD 1,000,000, marked USD 400,000), pledged under security-interest agreement G to counterparty wallet B at a 50% valuation percentage: W-ALPHA posted +1, B posted -1 ; collateral value to B USD 200,000 against exposure USD 150,000. OMEGA prints an official close of $79.00 \leq$ the barrier 80.00. *The discharge transaction.* The payout obligation discharges under its trapping predicate in one transaction, two moves: owned(USD) USD 1,000,000 W-BANK $\rightarrow$ W-ALPHA, and posted(USD) USD 150,000 W-ALPHA $\rightarrow$ B under G; USD 850,000 stays free. *The door.* Coverage holds within the transaction: W-ALPHA's owned cash $1,000,000 \geq$ posted 150,000, so the posting is admitted — a posting of collateral W-ALPHA did not possess would be refused. *Value versus mass.* The knock extinguishes value, never mass: pricing the triggered OT-1 at 0 collapses its value, but its coordinate balance is unchanged — coordinates carry mass, prices carry value. Retirement is only from the zero vector: the marker return clears the posted plane of OT-1 first — coverage forces the order, refusing to extinguish owned while posted mass remains — then owned(OT-1) moves W-ALPHA $\rightarrow$ W-BANK, taking every wallet's vector for OT-1 to

zero, and OT-1 retires. *Design point. Coverage is checked at the door on the discharge (owned 1,000,000 $\geq$ posted 150,000) and again as the order of close-out; the lifecycle event extinguishes OT-1's value but not its mass; and between knock and call the intraday shortfall is carried by the sufficiency obligation, not by a broken invariant.*

14.7 The post-and-return metamorphic property

The last property makes “only the owned coordinate carries economic value” (Chapter 3, constitution §4) machine-checkable.

Proposition 14.7 (Post-and-return). *A pledge posting writes the posted plane and never owned, so $V = \sum \text{owned} \cdot P$ is unchanged by any post-and-return round trip. Inserting a post-and-return pair within any watch-free interval of the referenced agreement leaves every later valuation identical.*

The side condition is real and named: an insertion spanning the agreement's valuation watch lawfully changes later firings and cash, so the property owns the move algebra, not the watched case. The watched case is the temporal residual carried by the safety and liveness model of Chapter 15 (Testability and the Executable-Check Regime). As a test the proposition is metamorphic: insert the pair into a generated history and assert that every subsequent NAV is unchanged to the minor unit.

14.8 What the catalogue buys

Every property above is stated once and executable, none defended in prose alone. Together with the product-graph consistency invariant of Chapter 3, they are the surface Chapter 15 tests over one generator universe. Their joint effect is the constitution's claim in §11: the classes of inconsistent state that internal reconciliation exists to detect — unbalanced books, a fact written twice into disagreement, a quantity valued in the wrong frame, collateral posted in excess of the poster's instructed ownership — are not caught here; they are unreachable. What cannot be represented cannot break. The residual economic exposures that remain — a bought-not-yet-settled posting whose purchase later fails, an intraday shortfall of value against exposure — are reachable and real, but the model carries them as recorded, deadline-bearing obligations (Chapters 9 and 11), never as silent breaches of an invariant.

15 Testability and the Executable-Check Regime

This chapter discharges §13 of the constitution — the two layers of correctness — and the Testability commitment of §2. The claim it makes good is the constitution's: the ledger is correct because it is built to be tested. Every invariant catalogued in Chapter 14 is an executable check run over generated products, events, and histories, never a property defended only in prose; and correctness composes, because the steps are pure and their composition is fixed — the map, then the fold — so verifying each part verifies the whole. This chapter does not restate the invariants. It makes them the surface a test regime exercises, states the regime, and turns the specification's three binding conditions into executable gates rather than intentions.

15.1 Two layers of correctness

Correctness rests on two layers, and each is checked by its own means.

Structural correctness holds by construction, at the door. An illegal state is not detected and rejected; it is unrepresentable — no program writes it. A balance changes only through a move (Chapter 3); a marker coordinate is unwritable without its governing agreement reference

(Chapter 9); a knocked note has no un-knock constructor (Chapter 3, Invariant 3.2); conservation is automatic because every move is paired-leg and the system is closed. These properties hold whatever the contract that proposed the transaction did — correct, careless, or malicious — because the Transaction Executor admits on structure alone and is the sole writer (Chapter 4). The test obligation for this layer is negative: no generated event, however adversarial, drives the ledger into a state the type discipline forbids or the door should refuse.

Economic correctness is not guaranteed at admission; it is made checkable by recomputation. A contract may emit a perfectly well-formed transaction that books the wrong economics. The check is to re-run the map on the recorded event and compare the result with what was recorded.

```
-- the economic oracle: re-run the map on the recorded event and compare
prop_recompute :: Recorded (Event, Transaction) -> Property
prop_recompute (e, txRecorded) = contract e (ledgerBefore e) == txRecorded
-- a mismatch is a defect of the CONTRACT, not the ledger; it is repaired by a
-- compensating transaction through the same door -- never by an edit to history.
```

Purity is what makes this oracle exist at all: a contract reads only the event and the record (Chapter 5), so its recomputation is exact and unconditional. A mismatch is a defect of the contract, never of the ledger, and it is repaired the way every change is made — by a compensating transaction through the same door. The regime-bit repair of Chapter 9 (a new terms version *plus* the compensating rebooking of the affected interval) is the worked instance; the valuations of Chapter 7 are recomputable under the same discipline. The two layers divide the labour exactly and neither substitutes for the other: structure is proved once and tested negatively, economics is re-derived per recorded event and tested by comparison, and a structurally valid transaction can still be economically wrong.

15.2 The generator universe and the adversarial scheduler

The enumerations this specification builds on are genuinely closed. On the ledger’s side: the product-graph node and edge sets — the lifecycle states and guarded transitions — fixed at registration (Chapter 3); the collateral regimes {title-transfer, pledge, STM/CTM} and the coordinate planes {owned, lent, posted} (Chapter 9); the component dimensions of the datum-kind registry and the boundary failure regimes W1–W4 (Chapter 8); and the trigger and event kinds of a contract (Chapter 5). On the CDM side the genuinely closed enumerations are the day-count fraction, the option type (call or put), and the primitive-instruction set (Chapter 13). These are closed choices, and they seed the generator universe; a co-populated, cardinality-many composite is not a closed choice and is not one of them. Because the closed enumerations are closed and declared, one generator universe ranges over the whole domain, and its coverage of each enumeration is itself a checkable fact — every inhabitant is reachable by the generators, so a property that never exercises a case is a visible gap, not a silent one.

```
genProduct :: Gen ProductGraph      -- a well-typed graph over the closed node
    /edge set
genEvent   :: ProductGraph -> Gen Event -- on-menu guards, and off-menu
    adversarial events
genHistory :: Gen [Transaction]      -- interleaved admitted walks of many
    products
-- every generator is paired with a shrinker: a violation shrinks to a minimal
    history,
-- and every counterexample is replayable from its seed (a defect is a
    reproduction).
```

Generation is adversarial by design, because the test environment must be harder than production. The Event Monitor and the Events Executor are outside the trust boundary and may duplicate, delay, and reorder firings; integrity owes them nothing (Chapter 4). The scheduler therefore explores every legal interleaving the Transaction Executor’s total order permits — du-

plicated proposals, late firings, events delivered out of arrival order — and injects pathological-but-legal behaviour rather than only the happy path: this is precisely what exercises idempotence and the temporal model below. One discipline guards the generators against vacuity: a property whose precondition is never generated passes without witnessing anything. A generated history that never knocks a pledged note, never re-posts received collateral, never delivers a duplicate is not evidence; the generators are measured by the states they reach, not the cases they cover.

15.3 The test surface: the invariant catalogue and the product graph

The invariants of Chapter 14 are the test surface, in a ramp from the universal to the domain-specific. At the base sit the properties no history may violate: no transaction half-applies (atomicity), no balance is created or destroyed off the paired-leg move (conservation per (unit, coordinate)), the log is append-only and hash-chained, and every arrow is deterministic. Above them sit the structural invariants — consistency of reference, writer discipline — and above those the safety properties of the collateral planes: coverage is an invariant, checked at the door on every admission, while sufficiency is an obligation, lawfully false intraday and bounded by a deadline, and the test regime must assert the first as an invariant and the second only as a liveness obligation. Stating sufficiency as an invariant would be a false theorem the generators refute at once, and insisting on it would forbid lawful states: the two guards are never collapsed, in the tests least of all.

Carried onto this surface are the four properties of the product-graph consistency invariant (Invariant 3.2, defined in Chapter 3), stated here as executable checks over generated products and histories:

```
-- for every unit, at every commit, the three interpreters of its graph agree:
prop_watchesAreOutEdges    :: History -> Property  -- (i) watch list == out-edges(
    current node)
prop_firedMatchesOneEdge   :: History -> Property  -- (ii) fired event -> exactly
    one edge, lands on its target
prop_actionEqualsRecorded  :: History -> Property  -- (iii) emitted tx == edge
    action applied to recorded inputs
prop_traversalExpiresSibs  :: History -> Property  -- (iv) traversal expires
    exactly the declared siblings
```

Property (i) is checked at every log position: the Event Monitor’s live watch set is folded from the record and compared with the out-edge set of the unit’s current node. Property (ii) rejects any firing that matches no edge or more than one, or that lands the unit off the edge’s declared target. Property (iii) is the graph-level face of the economic oracle above: the transaction the contract emitted must equal the edge’s declared action applied to the recorded inputs. Property (iv) walks the sibling set and confirms that a traversal expires those edges and no others. The graph declares watches, state schema, and actions from one object; these four properties are the statement, made executable, that its three interpreters never disagree.

15.4 The three binding conditions as executable gates

Three conditions the specification relies on are stated here as gates a candidate implementation must pass, not as intentions it may assert.

(i) Moves are the sole balance mutator. The constitution makes the move the only operation that changes a balance (Chapter 3); the gate makes that mechanically decidable. Applying a transaction, then applying only its move list, must leave identical balances — so a unit-state, position-state, or terms change that touched a balance is caught as a divergence.

```
prop_movesOnly :: Transaction -> Property
```



```

prop_movesOnly tx = balances (apply tx l0) == balances (apply (onlyMoves tx) l0)
  where l0 = ledgerBefore tx
-- corollary, tested jointly: apply (dropMoves tx) l0 leaves every balance fixed
.

```

(ii) Determinism and replay. The clock is the single input not on the record, and the Event Monitor is the single place it is consulted (Chapter 4); every arrow downstream is a function of the record alone. Two gates follow. *Idempotence*: a duplicated proposal, carrying the same cause-derived identifier, commits once and produces the identical transaction — a duplicate is inert. A late or out-of-order proposal is not a no-op on the tail: the immutable log’s total order for a unit’s fixings is the recorded observation index k , so the proposal is inserted at its observation position k and the order-dependent accumulator tail $A_k, A_{k+1}, \dots$ is re-sequenced and recomputed from k forward. *Replay determinism*: re-running the map on the recorded observations *in observation-index order* reproduces every recorded statistic bit-for-bit, because barrier and fixing observation are deterministic — each datum crosses the boundary once, as a recorded observation, and is never re-read live (Chapter 8); a late insertion recomputes the tail, and the recomputation is bit-exact.

Episode — the future (T1): a duplicate is inert; a late firing is identical. FUT-IDX settles at recorded IDX closes 1,000, 990, 1,005 (multiplier 10); cumulative variation margin is $+50 - 100 + 150 = 100$ (Chapters 5 and 7). *The gate*. The day-2 settlement event is delivered twice, and delivered late, in a scheduler that reorders it against *unrelated* firings. Reordering against unrelated firings leaves FUT-IDX’s own fixing sequence — its observation-index order — untouched, so the contract, a function of the record, emits the identical USD 100 move; the duplicate carries the same cause-derived identifier and commits once at the door. Were day 2 instead reordered *within* FUT-IDX’s own sequence — arriving after day 3 — it would be inserted at its observation position and the variation-margin tail recomputed from there, not applied as a late no-op. *Design point*. *Timeliness depends on the untrusted machines; integrity does not — a duplicate is inert, a firing reordered against unrelated events is identical, and one reordered within its unit’s own sequence is re-sequenced and its tail recomputed; all are properties the scheduler tries and fails to break.*

Episode — the variance swap (T5): regenerate firing 127. VS-1 accrues a sum-of-squared-returns statistic in UnitStatus, an integer at scale 10^{-8} , written by each firing for the next (Chapter 6). At the mid-life checkpoint the accrued statistic is $A_{126} = 1,805,000$, and the previous recorded close is fixing 126 = 1,005. Firing 127 reads A_{126} from the record, forms the return against the recorded closes, and writes A_{127} .

```

accrue :: Close -> Close -> Accrued -> Accrued          -- named, versioned, pure
prop_replay127 :: Property
prop_replay127 = accrue close127 1005 1805000 == recorded A_127
-- the recorded closes are deterministic; A_126 is a recorded integer; so any
  party
-- re-running the same map on the same recorded fixings obtains the identical
  A_127.

-- replay folds the fixings in recorded OBSERVATION-INDEX order (the sequence k),
  never
-- arrival order; a late or out-of-order print is inserted at position k and the
  tail
-- A_k, A_{k+1}, ... is recomputed from k forward:
prop_lateInsertRecomputesTail :: Fixing -> [Fixing] -> Property
prop_lateInsertRecomputesTail late fs =
  foldAccrue (insertAtObsIndex late fs) == foldAccrue (sortByObsIndex (late : fs)
)

```

-- the fold is over an ordered integer sequence, so the recomputed tail is bit-exact.

Replay is a fold in observation-index order, not arrival order: the immutable log's total order for VS-1's fixings is the recorded fixing sequence, so an out-of-order or late print is inserted at its position k and the accrual tail $A_k, A_{k+1}, \dots$ recomputed from k forward, bit-for-bit, while the downstream daily CCP-cash figures at positions after k — the future's variation margin on the same recorded closes — are corrected by compensating transactions, not left as posted. Firing 127 is not trusted to have been right because it ran; it is verified by recomputation, and the recomputation is exact because its every input — the accrued integer, the recorded closes — is on the record. *Economic verification is re-running the map on recorded inputs and comparing; determinism is what makes the comparison bit-exact.*

(iii) The temporal residual: a safety and liveness model. The move algebra owns what is true *within* a transaction — conservation, coverage, atomicity, all checked at the door. It does not own the ordering *across* transactions: which interleavings of independent firings are reachable, and whether an open obligation is eventually discharged, are temporal questions the algebra cannot answer. The one-touch knock while pledged is where the residual bites, and it carries an explicit model.

Episode — the one-touch (T2): the knock-revalue-call-discharge model. W-ALPHA owns OT-1 (payout USD 1,000,000, marked USD 400,000 pre-knock), pledged under security-interest agreement G to counterparty B at a 50% valuation percentage: collateral value USD 200,000 against exposure USD 150,000. OMEGA's official close prints 79.00, at or below the barrier 80.00 (Chapter 9). Four steps then interleave: the Monitor emits the knock; OT-1's contract triggers the unit and mints the conditional payment obligation; the pricing layer, reading the triggered state, marks OT-1 at 0, so G's valuation watch opens a margin call for USD 150,000; the payment obligation discharges USD 1,000,000 (USD 850,000 free, USD 150,000 posted to B), and the pledge closes only after the marker returns — an ordering the coverage invariant forces. These steps are separate admitted transactions, and their interleaving is the temporal residual. The model is a state machine over the reachable states, with two invariants held always and one obligation held eventually:

```
Coverage(s)      == \A (w,u): postedMass(w,u) =< Max(owned(w,u), 0)  -- mass; the
    door's invariant
NoDoubleCount(s) == (status = "triggered") => price(OT1) = 0          -- the payout
    is counted once
Safety   == [] ( Coverage /\ NoDoubleCount )                          -- at every
    committed state
Liveness == (call = "open") ~> (call = "discharged" /\ call = "closed_out")
-- Sufficiency (collateral value >= exposure) is NOT a [] invariant: it is
    lawfully false
-- on the interval [trigger, discharge); only Liveness constrains it. Asserting it
    as an
-- invariant would be a false theorem, and would forbid a lawful state.
```

Safety says no reachable committed state double-counts value or violates coverage. Double-counting is excluded because a triggered OT-1 prices at 0: no committed state shows both the option's positive mark and the USD 1,000,000 cash it resolves into. Coverage is excluded because it is checked at the door on every one of the interleaved transactions, and the marker-before-extinguishment ordering of the final step is coverage refusing to strip owned while posted mass remains. *Liveness* says the margin call opened at the revalue is eventually discharged or closed out — the obligation-liveness requirement of the Event Monitor made a temporal proof obligation (§14.1, Chapter 16), under the fairness that the Events Executor eventually delivers each armed firing. The central discipline is what the model *refuses* to assert: between trigger

and discharge the book is lawfully under-collateralised, so sufficiency appears only under $\rightsquigarrow$, never under $\Box$. Coverage — the mass guard — and sufficiency — the value guard — are the two guards of Chapter 9, and here the temporal model is exactly where their difference is proved: one is a safety invariant, the other a liveness obligation, and a model that confused them would be unsound in the direction that either fabricates breaches or hides them.

15.5 What an auditor checks

A candidate implementation satisfies this chapter when the checks above run green over the generator universe: the structural invariants of Chapter 14 hold as invariants and no adversarial history reaches a forbidden state; the economic oracle reproduces every recorded transaction from its recorded event, and a seeded divergence is repaired only by a compensating transaction; the four product-graph properties confirm that watches, state schema, and actions never disagree; moves are shown to be the sole balance mutator; a duplicate firing is inert and a late firing folds in at its observation index — identical when reordered against unrelated events, its tail recomputed when reordered within its unit’s own sequence; firing 127 recomputes bit-for-bit; and the knock-revalue-call-discharge model holds its two safety invariants always and its liveness obligation eventually, with sufficiency asserted only as the latter. Each is mechanical, each is replayable from a seed, and each fails closed. A property that cannot be checked this way is redesigned until it can be: property-based testing is a construction principle here, not a phase at the end.

16 Minimum Requirements

This chapter discharges §14.1 and §14.2 of the constitution.

It adds nothing. Each requirement below is a consequence already derived and proved in the body — restated once, imperatively, as the minimum any implementation must meet, and traceable by cross-reference to the chapter that established it. The requirements fall in three groups: the Event Monitor and the Events Executor (§14.1, from Chapter 4); the valuation semantics (§14.2, from Chapter 7); and the data boundary (§14.2, from Chapter 8). A candidate implementation is conformant when every requirement holds; §16.4 states how an auditor confirms it.

Principle 16.1 (What the minima secure). *The ledger’s integrity holds by construction and rests on the Transaction Executor alone (Chapter 4). The requirements of this chapter secure the properties integrity does not supply: the liveness and replayability of the untrusted machinery, and the reproducibility and boundary-closure of the pricing layers. Integrity owes these components nothing; timeliness, reproducibility, and the closed-system property are conditional on them.*

16.1 The Event Monitor and the Events Executor

The Event Monitor and the Events Executor sit outside the trust boundary (Chapter 4). No failure either can commit corrupts ledger state; every one must degrade to a visible, overdue item rather than to silence. What they owe is liveness and replayability, and the following seven requirements are its minimum.

M1. Durable watches and timers. A watch registered for a future date or condition must fire when the condition is met, across restarts and failures; timer state must be persisted, never held only in volatile memory (Chapter 4). A watch that could be lost on restart is a silent gap, which §14.1 forbids.

- M2. At-least-once emission.** No emitted event may be lost; the Monitor must re-emit until the event is on the record. A duplicate emission must be harmless, made so by the cause-derived idempotence check at the door (Chapter 4). Losing an event is unrecoverable; emitting one twice costs nothing.
- M3. Acknowledged watches.** Every declared watch must be acknowledged by the Monitor, and the acknowledgement recorded in the unit’s state, so a watch is visibly declared, then armed, then fired or expired. A watch unacknowledged beyond its arming window must stand on the record as an overdue item, never as absence (Chapter 4).
- M4. Triggers carry timing, not data.** A trigger must say only *when*; everything the event means must be read from the record by the contract it fires (Chapter 4). A trigger carrying data would place an unrecorded input before the door and break reproducibility.
- M5. Deterministic, replayable orchestration.** Each machine’s own state must be reconstructible by replaying its append-only history, so a late firing produces the identical transaction and any party can recompute which events should have been emitted, and when (Chapter 4).
- M6. No write privilege.** Neither the Event Monitor nor the Events Executor may hold write access to the ledger. The Transaction Executor is the single writer (Chapter 4); an emission or a firing is a proposal, never a commit.
- M7. Obligation liveness.** Every contractual obligation must be registered with a deadline, a test for its discharge, and a fallback action; by its deadline it must be discharged, compensated, or declared defaulted. A duty that fails to fire must degrade to a visible, overdue item, never to silence. This is the liveness counterpart of the value sufficiency obligation of Chapter 9 and Chapter 14.

16.2 Valuation semantics

Net Asset Value is a projection of the log, folded and priced; valuation adds no store (Chapter 7). Chapter 7 derived six requirements (P1–P6) on the pricing stack and the pricing data layer as consequences of that fact. They are the minima any pricing layer must meet.

- V1. The ledger records pricing outputs and never runs a model.** A pricing model must run outside the ledger; its output must re-enter only as a recorded observation carrying sufficient provenance (Chapter 7, P1). Reproducing a valuation from recorded prices is then unconditional; reproducing the model’s number requires the model itself, whose retention is a governance matter outside scope (Chapter 17).
- V2. Valuation is state-parameterised, and the state comes from the ledger.** The price function must read the unit’s lifecycle state from `UnitStatus` and `PositionState` (Chapter 6), never from a store the pricing layer keeps of its own (Chapter 7, P2). Value that turns on state turns on state the ledger alone authors.
- V3. Valuations are reproducible from recorded inputs.** Every NAV must be recomputable to the minor unit from the composition — a fold of the log — and prices identified on the record (Chapter 7, P3). No valuation may depend on an input that is not on the record.
- V4. Prices and quantities are never mixed across an event.** A price and a quantity may be combined only when both refer to the same state of the instrument — consistency of reference (Chapter 7, P4; catalogued in Chapter 14). The mechanism that enforces this at ingestion is requirement B5.

- V5. Two price vectors are both projections over one composition.** Mark-to-market and mark-to-mid must be two price vectors applied to the single composition the fold produces; positions must never diverge — only the two money figures may (Chapter 7, P5). One position record forbids the settlement-versus-risk position drift that is a classical reconciliation break.
- V6. Estimates are kept apart from entitlements.** An estimated or model-derived value must never be recorded as, or substituted for, an owed quantity: a mark is a projection, an entitlement is a recorded fact (Chapter 7, P6). The complementary boundary duty — originals preserved, adjusted and derived values recomputed at read — is requirement B4.

16.3 The data boundary

The boundary is the one place the closed system is not closed (Chapter 8). Chapter 8 held it watertight with two devices — the recorded observation and the market data operator — and listed six checks a candidate implementation must pass. A seventh extends the same discipline to the declared terms of a governing agreement, whose provenance is the executed agreement rather than a market feed (Chapter 9). They are the boundary minima.

- B1. Every datum carries provenance.** Every datum on the record must carry its provenance — source, observation time, and basis frame — and no valuation may read a datum that does not (Chapter 8, §8.2). A bare read against a live feed leaves no record and is unreplicable; it must be refused.
- B2. Kinds are registered before they cross.** Every datum's kind must be registered, with the dimension of each component — price, proportional, or quantity — before any datum of that kind crosses; a datum of an unregistered kind must be refused at the door (Chapter 8, §8.3). Registration is immutable: a correction is a new kind plus a retirement of the old, never an edit.
- B3. An operator is declared for every pairing.** For every (registered datum kind, event kind) pair the boundary admits, a market data operator must be declared — the identity operator where a kind is genuinely unaffected, but declared, not assumed. A pairing with no declared operator must be refused, never defaulted to identity (Principle 8.1, Chapter 8). This is adjustment-schedule totality, and it fails closed.
- B4. Originals preserved, adjusted computed at read.** Every observed datum must be preserved on the record exactly as observed; every adjusted value must be recomputed at each read from the original under the declared operators, and every derived quantity recomputed from operator-adjusted inputs (Chapter 8, §8.5–§8.7). An original must never be overwritten by its adjustment.
- B5. The invariance witness holds.** Across every corporate action the operator on the value frame and the move on the quantity must be bound so that total value is invariant across the event; a stale-frame read — a new count against an old price — or a wrong-side-of-ex read must be refused (Chapter 8, §8.6). This is the ingestion-side enforcement of requirement V4.
- B6. Each failure regime has its determined disposition.** Each boundary failure — missing datum (W1), late datum (W2), contradictory sources (W3), unmappable event (W4) — must produce its determined, replayable disposition, a function of log state alone, so replay reproduces it bit-identically; none may improvise or pick silently (Chapter 8, §8.8). In particular a single source for a datum that materially affects valuation is insufficient: such a datum must be drawn from more than one attested source, aggregated by a declared

rule, with divergence beyond a declared threshold producing a flagged output rather than a silent pick (W3).

B7. Declared agreement terms are reconciled at the boundary. Every declared term of a governing agreement — eligibility, valuation percentage, thresholds, the trapping predicate, and the legal-regime bit — is boundary data whose provenance is the executed agreement; it must be reconciled against the counterparty’s own record, and a transcription error repaired by terms amendment plus a compensating rebooking, never a silent edit (Chapter 9; Chapter 8). The regime bit is the worked instance; the duty is general over every declared term.

The boundary rests on two named trust assumptions and no other: TA-KIND, that a kind declaration faithfully renders its publisher’s quotation convention, owned by data governance and detected by the invariance witness, source divergence, and recomputation (Chapter 8); and TA-TERMS, that a declared agreement term faithfully transcribes the executed agreement (B7), owned by the same governance and detected by boundary reconciliation against the counterparty’s own record. Everything else that crosses is recorded, framed, and replayable.

16.4 Verification

An auditor confirms conformance by exercising each group against generated products, events, and histories, on the executable-check surface of Chapter 15.

The machine minima (M1–M7) are liveness and replay properties: generate restart, duplicate-emission, and late-firing histories, and confirm that no event is lost, that every watch is acknowledged or stands overdue, that every late firing reproduces the identical transaction, and that neither machine writes the ledger. The valuation minima (V1–V6) are checkable over generated positions and prices: recompute every NAV from the record and compare to the minor unit, confirm the price function reads state only from the ledger, and confirm that mark-to-market and mark-to-mid fold one composition. The boundary minima (B1–B7) are checkable over generated data, corporate actions, and agreements: confirm that no datum lacks provenance, that no unregistered kind and no undeclared pairing crosses, that every original survives its adjustment, that the invariance witness holds across every generated event, that each of W1–W4 replays bit-identically, and that a perturbed agreement term is caught by boundary reconciliation and repaired by amendment plus rebooking rather than silent edit (Chapters 15 and 14).

Because every requirement here restates a derivation already proved in the body, conformance is not an added burden but the body’s own claims made executable. When they hold, the untrusted machinery is live and replayable, the pricing layer adds no store, and the boundary is closed — so the conservation and reproducibility the rest of the specification proves rest on stated, testable minima.

17 Scope

This chapter discharges the *Scope* section of the constitution.

Scope fixes the boundary of the specification: what the specification covers, and what it reconciles against at its perimeter but never performs. The constitution states the in-scope object in one sentence — the recording, valuation, and lifecycle of trading-book positions held at fair value, and the interface that projects committed activity into settlement. Everything below is that sentence made exact, component by component, together with the explicit list of external authorities that lie outside it.

17.1 In scope

The specification covers the following components, each defined and proved in the chapter named. Nothing in scope lives outside a chapter; nothing below is left to an implementation to invent.

- **The map-then-fold architecture and the projection machinery** that derives every view from the log — Chapter 2.
- **The immutable log and the objects** — units, wallets, balances, the atomic move, the transaction, and the signed coordinate vector (*owned*, *lent*, *posted*) — Chapter 3.
- **The three machines** — the Event Monitor, the Events Executor, and the Transaction Executor — and the single-writer door — Chapter 4.
- **The smart-contract framework**, its event-triggering discipline, and the purity and idempotence requirements — Chapter 5.
- **The three homes** — ProductTerms, UnitStatus, PositionState — and the conditions that render illegal states unrepresentable — Chapter 6.
- **Valuation and profit and loss** — NAV as a projection, path-independent PnL, and deposit neutrality — Chapter 7.
- **Ingestion, corporate actions, and the market data operator** — the recorded observation and the operator intrinsic to each (data-kind, event-kind) pair — Chapter 8.
- **Collateral, margin, and lending** — the regime key, the coverage invariant, and the determination/payment split — Chapter 9.
- **Virtual ledgers, strategies, and the total return swap** — Chapter 10.
- **The settlement interface** — the projection of committed activity into settlement — Chapter 11.
- **Presentation and reporting projections** — every reporting figure read from coordinates, units, and declared terms alone — Chapter 12.
- **Alignment with the Common Domain Model** — shown, never adopted as authority — Chapter 13.
- **The invariant catalogue** and its executable properties — Chapter 14.
- **Testability and the executable-check regime** — Chapter 15.
- **The minimum requirements** any conformant implementation must meet — Chapter 16.

17.2 Out of scope

The following are external authorities. The ledger reconciles against each at its boundary; it performs none of them. Each is a function of some other system, and the ledger's closure holds precisely because it does not absorb them.

- **Price formation** — price discovery and executable pricing. The ledger records the outputs of pricing models and never runs one (Chapter 7; Chapter 16, V1). The discovery of price, and the model that produces it, are external.
- **Legal agreements** — the enforceability and validity of contracts. Every right and obligation a legal contract creates is represented in the ledger as a unit (Chapter 5), but the legal standing of the instrument is decided outside the boundary.

- **The settlement infrastructure itself** — settlement finality, payment-system access, and custodian and CSD connectivity. The ledger says *what* settles; the infrastructure decides how, and whether (Chapter 11).
- **Regulatory submission** — submission gateways and report formatting. Every report is a projection of the record (Chapter 12); its transmission and its per-regime format are external.
- **Reference-data authority** — the mastering of instrument, counterparty, and calendar reference data. The ledger reconciles against it at the boundary; it does not master it.
- **Model governance and retention** — the validation, approval, and retention of pricing and strategy models, and accounting-policy decisions. Reproducing a valuation from recorded inputs and identified prices is unconditional; reproducing a model’s number requires the model itself, whose retention is a governance matter outside the ledger’s scope (the anchor of Chapter 16, V1).

Three companion documents are named in the Exclusions Register as out-of-scope deliverables — expansions the specification points to but does not contain, and not functions the ledger performs: the *Worked-Examples Volume* (register lines E71 and E72), holding the dividend-forecast, special-dividend, spin-off, and elective-merger worked examples; the *SBL Operations Companion* (register line E73), holding the full securities-lending operational lifecycle — locates, buy-ins, SFTR and CSDR discipline, cascade recalls; and the *Track B graphs-interpretter companion* (register line E74), holding the generic term-sheet interpretter, the CDM round-trip, and term-sheet rendering. Each is named so its absence from this specification is deliberate and auditable, never silent.

17.3 The open-problems index and the constitutional-parking rule

Two registers live in this chapter. The first names the design questions the specification leaves open; the second parks any genuine conflict with the constitution.

The open-problems index. The constitution requires that implementation questions be named and left, not silently dropped. Each open design question is named here and carried in full in the *Open-Problems and Escalation Register* companion. The top open risk is close-out and netting algebra: the collateral ruling made per-netting-set claim units first-class (Chapter 9) precisely so that close-out is representable, yet the algebra that nets and closes those units out is not yet designed. It entails, on a counterparty default, valuing each claim at close-out — each priced by the taker — netting the per-netting-set claims and obligations to a single figure, and decoupling recovery from that figure; it is named here so it cannot be improvised. Named alongside it: managed-account fee accrual and NAV attribution, which the constitution’s §6 defers to the detailed specification and which this specification therefore names but does not work; the correction algebra beyond the single compensating transaction; inter-ledger federation and eliminations; the fails-cascade obligation’s full valuation and lifecycle — its mint by the settlement-fail contract, its valuation as a unit, its resolution or supervised write-off, and the accounting of its paired delivery-versus-payment cash leg (Chapter 9); and the remaining items of the companion register. Naming is the whole obligation; none of these is a gap in what the specification does claim.

The constitutional-parking rule. The second register governs conflict with Constitution v1.1 itself.

Principle 17.1 (Constitutional parking). *A genuine conflict between this specification and the constitution — discovered in drafting or in review — is parked in this index with the exact proposed amendment text. It is never silently amended and never fudged. Constitutional*

amendments are the owner's alone, under the constitution's closing Authority and Amendment section; the one-way direction of authority forbids an author from resolving a conflict by editing the constitution. Parking makes the conflict visible and hands the owner a precise, decidable choice.

Current status: the index of parked amendments is empty. The drafting of Chapters 1 through 16 surfaced no conflict with the constitution; every author's completion report filed none, and the drafting of this chapter surfaced none. The index is empty but live: the mechanism stands, numbered and in place, so that any conflict a later revision or review discovers has a home and a fixed procedure rather than an ad hoc one.

17.4 Authority

The direction of authority is one-way. This specification is rated against the constitution, and against nothing else; the constitution is rated against nothing. Every chapter opened by naming the sections it discharges, and this one discharges the constitution's Scope section. Where a future improvement requires departing from the constitution, the constitution is amended first — through the parking rule above and the owner's decision — never the specification bent in silence to fit it, and never the constitution bent to fit the specification.